

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống Website TripXuyenViet bán các chuyến du lịch

Phạm Đức Dũng

dung.pd215265@sis.hust.edu.vn

Ngành Công nghệ thông tin và truyền thông

Giảng viên hướng dẫn: TS. Nguyễn Đức Anh _____

Chữ kí GVHD

Chương trình đào tạo: Công nghệ Thông tin Việt-Pháp

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2025

LỜI CẢM ƠN

Lời đầu tiên, em không biết nói gì hơn ngoài bày tỏ sự biết ơn sâu sắc đến các thầy cô giáo trường Công nghệ Thông tin và Truyền thông - Đại học Bách Khoa Hà Nội đã dạy dỗ cho em kiến thức về các môn đại cương cũng như các môn chuyên ngành, giúp em có được cơ sở lý thuyết vững vàng và tạo điều kiện giúp đỡ em trong suốt quá trình học tập.

Đặc biệt, em xin bày tỏ sự kính trọng và lòng biết ơn sâu sắc nhất đến thầy giáo hướng dẫn **TS. Nguyễn Đức Anh**, thầy là người đã trực tiếp hướng dẫn, giúp đỡ cho em để em có thể hoàn thành đồ án này.

Tiếp theo, em xin được bày tỏ lòng biết ơn của mình với gia đình, bạn bè đã luôn là động lực, hậu phương vững chắc, ủng hộ, giúp đỡ cùng em trong quá trình hoàn thành đồ án tốt nghiệp.

Em xin kính chúc các thầy cô luôn luôn khỏe mạnh và ngày một thành công hơn trên con đường giảng dạy của mình.

Em xin trân trọng cảm ơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Ngành công nghiệp du lịch đang chứng kiến sự phát triển mạnh mẽ, trở thành một trong những lĩnh vực kinh tế trọng điểm đóng góp đáng kể vào GDP toàn cầu. Cùng với sự phát triển này, nhu cầu du lịch của khách hàng ngày càng gia tăng không chỉ về số lượng mà còn về sự đa dạng và tính cá nhân hóa. Mỗi khách hàng có những sở thích, ưu tiên và khả năng tài chính khác nhau, tạo nên những yêu cầu riêng biệt và thách thức lớn đối với các doanh nghiệp cung cấp dịch vụ du lịch.

Trong bối cảnh đó, việc xây dựng một nền tảng thương mại điện tử dạng marketplace cho phép nhiều công ty du lịch cùng tham gia và đăng bán chuyến du lịch trên một hệ thống chung trở nên cần thiết. Điều này giúp khách hàng có thể dễ dàng tiếp cận, so sánh và lựa chọn chuyến du lịch phù hợp nhất với nhu cầu của mình, đồng thời tạo điều kiện cho các doanh nghiệp mở rộng kênh phân phối.

Nhận thấy xu hướng và nhu cầu này, đề tài tập trung phát triển một hệ thống website marketplace du lịch với backend được xây dựng bằng Java Spring Boot, cùng frontend sử dụng ReactJS nhằm mang lại trải nghiệm người dùng thân thiện, hiện đại và mượt mà. Hệ thống cho phép đăng ký và quản lý gian hàng cho nhiều nhà cung cấp, quản lý chuyến du lịch, đơn đặt hàng, thanh toán trực tuyến tích hợp cổng VnPay và bảo mật xác thực qua JWT.

Ứng dụng không chỉ giúp khách hàng dễ dàng tìm kiếm, lọc và đặt chuyến du lịch mà còn cung cấp các công cụ quản lý linh hoạt cho các công ty du lịch, giúp tối ưu hóa quy trình vận hành. Hệ thống được thiết kế theo kiến trúc client-server truyền thống, đảm bảo sự ổn định, bảo mật và khả năng mở rộng trong tương lai.

Mục tiêu của đề tài là xây dựng một giải pháp toàn diện, góp phần đơn giản hóa trải nghiệm du lịch trực tuyến, nâng cao sự hài lòng của khách hàng, đồng thời thúc đẩy phát triển bền vững ngành công nghiệp du lịch thông qua việc áp dụng công nghệ hiện đại và mô hình marketplace hiệu quả.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

The tourism industry has been witnessing rapid growth, becoming one of the key economic sectors significantly contributing to the global GDP. Along with this development, customers' travel demands are increasing not only in quantity but also in diversity and personalization. Each customer has distinct preferences, priorities, and financial capacities, creating unique requirements and significant challenges for tourism service providers.

In this context, building an e-commerce marketplace platform that allows multiple travel companies to participate and list their tours on a common system has become essential. This approach enables customers to easily access, compare, and select the most suitable tours according to their needs, while also providing businesses with opportunities to expand their distribution channels.

Recognizing these trends and demands, this project focuses on developing a travel marketplace website with a backend built on Java Spring Boot and a frontend developed using ReactJS, aiming to deliver a user-friendly, modern, and smooth experience. The system supports registration and management of vendor stores, tour management, order processing, online payments integrated with VnPay, and secure authentication via JWT.

The application not only facilitates customers in searching, filtering, and booking tours but also offers flexible management tools for travel companies to optimize their operational processes. The system is designed with a traditional client-server architecture, ensuring stability, security, and scalability for future expansion.

The objective of this project is to build a comprehensive solution that simplifies the online travel experience, enhances customer satisfaction, and promotes sustainable development of the tourism industry through the application of modern technology and an effective marketplace model.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	1
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	3
2.1 Khảo sát hiện trạng	3
2.2 Tổng quan chức năng	4
2.2.1 Biểu đồ use case tổng quát	4
2.2.2 Biểu đồ use case phân rã Quản lý thông tin người dùng	5
2.2.3 Biểu đồ use case phân rã Quản lý thông tin lịch trình chuyến du lịch	6
2.2.4 Biểu đồ use case phân rã Xem và đặt các chuyến du lịch.....	6
2.2.5 Quy trình nghiệp vụ	7
2.3 Đặc tả chức năng	10
2.3.1 Usecase quản lý người dùng.....	10
2.3.2 Usecase quản lí chuyến du lịch	13
2.3.3 Usecase quản lý đơn đặt chuyến du lịch.....	15
2.3.4 Usecase đặt chuyến du lịch	17
2.3.5 Usecase gửi đánh giá, phản hồi chuyến du lịch đã hoàn thành.....	19
2.4 Yêu cầu phi chức năng	21
2.4.1 Hiệu suất.....	21
2.4.2 Tính khả dụng.....	21
2.4.3 Bảo mật	21

2.4.4 Khả năng mở rộng.....	21
2.4.5 Khả năng bảo trì và quản lý.....	21
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	23
3.1 Frontend – ReactJS, Vite và TailwindCSS.....	23
3.2 Backend – Java Spring Boot	24
3.3 Hệ quản trị cơ sở dữ liệu – PostgreSQL	25
3.4 Nền tảng dịch vụ cơ sở dữ liệu – Supabase	26
3.5 Quản lý hình ảnh – Cloudinary	27
3.6 Tổng kết.....	28
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	29
4.1 Thiết kế kiến trúc.....	29
4.1.1 Lựa chọn kiến trúc phần mềm	29
4.1.2 Thiết kế tổng quan.....	34
4.1.3 Thiết kế chi tiết gói	37
4.2 Thiết kế chi tiết.....	39
4.2.1 Thiết kế giao diện	39
4.2.2 Thiết kế lớp	42
4.2.3 Thiết kế cơ sở dữ liệu	45
4.3 Xây dựng ứng dụng.....	54
4.3.1 Thư viện và công cụ sử dụng.....	54
4.3.2 Kết quả đạt được	54
4.3.3 Minh họa các chức năng chính	55
4.4 Kiểm thử.....	61
4.4.1 Kiểm thử chức năng "Xem danh sách/ chi tiết chuyến du lịch"	61
4.4.2 Kiểm thử chức năng "Tìm kiếm chuyến du lịch"	62
4.4.3 Kiểm thử chức năng "Quản lý chuyến du lịch"	62

4.4.4 Kiểm thử chức năng "Quản lý người dùng"	63
4.4.5 Kết luận	63
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	65
5.1 Xây dựng nền tảng Marketplace Chuyên Du Lịch đa đối tượng	65
5.1.1 Đặt vấn đề	65
5.1.2 Thực hiện giải pháp	65
5.1.3 Kết quả đạt được	66
5.2 Chức năng thanh toán trực tuyến với Cổng thanh toán VnPay	66
5.2.1 Đặt vấn đề	66
5.2.2 Thực hiện giải pháp	67
5.2.3 Kết quả đạt được	69
5.3 Xây dựng và Tích hợp Module Xác thực Phân quyền bằng JWT	69
5.3.1 Phân tích bài toán và bối cảnh ứng dụng	69
5.3.2 Thiết kế và triển khai luồng xác thực với JWT	70
5.3.3 Đánh giá kết quả và lợi ích	72
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	73
6.1 Kết luận	73
6.2 Hướng phát triển	74
TÀI LIỆU THAM KHẢO	76

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	4
Hình 2.2	Biểu đồ use case phân rã: Quản lý thông tin người dùng	5
Hình 2.3	Biểu đồ use case phân rã: Quản lý thông tin lịch trình chuyến du lịch	6
Hình 2.4	Biểu đồ use case phân rã: Xem và đặt các chuyến du lịch . . .	6
Hình 2.5	Biểu đồ quy trình quản lý người dùng(CRUD)	7
Hình 2.6	Biểu đồ quy trình quản lý nội dung chuyến du lịch	8
Hình 2.7	Biểu đồ quy trình quản lý đơn đặt chuyến du lịch	8
Hình 2.8	Biểu đồ quy trình đặt chuyến du lịch	9
Hình 2.9	Biểu đồ quy trình gửi đánh giá phản hồi chuyến du lịch đã hoàn thành (Khách hàng)	10
Hình 4.1	Mô hình kiến trúc 3 tầng	30
Hình 4.2	Mô hình kiến trúc MVC	31
Hình 4.3	So sánh quản lý state khi không sử dụng và sử dụng React Context API	32
Hình 4.4	Sơ đồ kiến trúc hệ thống	34
Hình 4.5	Kiến trúc gói Backend tổng quan	35
Hình 4.6	Kiến trúc gói frontend tổng quan	36
Hình 4.7	Chi tiết gói Backend	37
Hình 4.8	Chi tiết gói Frontend	38
Hình 4.9	Giao diện trang chủ của hệ thống quản lý chuyến du lịch . . .	41
Hình 4.10	Thiết kế lớp TourController	42
Hình 4.11	Thiết kế usecase "Xem danh sách/ chi tiết chuyến du lịch" . . .	43
Hình 4.12	Biểu đồ trình tự usecase "Xem danh sách/ chi tiết chuyến du lịch"	44
Hình 4.13	Cơ sở dữ liệu	45
Hình 4.14	Màn hình trang chủ sau khi đăng nhập vào hệ thống	55
Hình 4.15	Màn hình tìm kiếm chuyến du lịch	56
Hình 4.16	Màn hình chi tiết chuyến du lịch	56
Hình 4.17	Màn hình đặt chuyến du lịch	57
Hình 4.18	Màn hình thực hiện thanh toán	57
Hình 4.19	Màn hình thanh toán thành công	58
Hình 4.20	Màn hình xem đơn hàng	58
Hình 4.21	Màn hình quản lý Admin	59

Hình 4.22	Màn hình quản lý của người quản lý	59
Hình 4.23	Màn hình tạo mới chuyến du lịch	60
Hình 4.24	Màn hình cập nhật lịch trình	60
Hình 4.25	Màn hình quản lý booking	61
Hình 5.1	Giao diện đăng ký đối tác VnPay	68
Hình 5.2	Cấu trúc chi tiết của JWT	70

DANH MỤC BẢNG BIỂU

Bảng 2.1	Bảng thông tin chính cho Use case Quản lý thông tin người dùng	11
Bảng 2.2	Bảng dữ liệu đầu vào cho Use case CRUD Quản lý thông tin người dùng	11
Bảng 2.3	Bảng dữ liệu đầu ra cho Use case Quản lý thông tin người dùng	12
Bảng 2.4	Thông tin chi tiết về Use Case Quản lý chuyến du lịch	13
Bảng 2.5	Dữ liệu đầu vào cho Use Case Quản lý chuyến du lịch	14
Bảng 2.6	Dữ liệu đầu ra cho Use Case Quản lý chuyến du lịch	14
Bảng 2.7	Bảng thông tin chính cho Use case Quản lý đơn đặt chuyến du lịch	15
Bảng 2.8	Bảng dữ liệu đầu vào cho Use case Quản lý đơn đặt chuyến du lịch	16
Bảng 2.9	Bảng dữ liệu đầu ra cho Use case Quản lý đơn đặt chuyến du lịch	16
Bảng 2.10	Thông tin chi tiết về Use Case Đặt chuyến du lịch	17
Bảng 2.11	Dữ liệu đầu vào khi đặt chuyến du lịch	18
Bảng 2.12	Dữ liệu đầu ra khi đặt chuyến du lịch	18
Bảng 2.13	Thông tin chi tiết về Use Case Đánh giá chuyến du lịch	19
Bảng 2.14	Dữ liệu đầu vào cho Use case Đánh giá chuyến du lịch	19
Bảng 2.15	Dữ liệu đầu ra cho Use case Đánh giá chuyến du lịch	20
Bảng 4.1	Mô tả chi tiết bảng users	46
Bảng 4.2	Mô tả chi tiết bảng tour	47
Bảng 4.3	Mô tả chi tiết bảng booking	48
Bảng 4.4	Mô tả chi tiết bảng itinerary	48
Bảng 4.5	Mô tả chi tiết bảng feedback	49
Bảng 4.6	Mô tả chi tiết bảng role	49
Bảng 4.7	Mô tả chi tiết bảng permission	49
Bảng 4.8	Mô tả chi tiết bảng user_role_permission	50
Bảng 4.9	Mô tả chi tiết bảng payment	50
Bảng 4.10	Mô tả chi tiết bảng category	50
Bảng 4.11	Mô tả chi tiết bảng tour_category	51
Bảng 4.12	Mô tả chi tiết bảng qna_and_ans	51
Bảng 4.13	Mô tả chi tiết bảng notification	52
Bảng 4.14	Mô tả chi tiết bảng interest	52

Bảng 4.15	Mô tả chi tiết bảng user_interest	52
Bảng 4.16	Mô tả chi tiết bảng image_path	53
Bảng 4.17	Mô tả chi tiết bảng invalidated_token	53
Bảng 4.18	Danh sách thư viện và công cụ sử dụng	54
Bảng 4.19	Thông tin về ứng dụng	55
Bảng 4.20	Bảng kiểm thử chức năng "Xem danh sách/ chi tiết chuyến du lịch"	62
Bảng 4.21	Bảng kiểm thử chức năng "Tìm kiếm chuyến du lịch"	62
Bảng 4.22	Bảng kiểm thử chức năng "Quản lý chuyến du lịch"	63
Bảng 4.23	Bảng kiểm thử chức năng "Quản lý người dùng của ADMIN"	63

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ĐATN	Đồ án tốt nghiệp
ACID	Các thuộc tính đảm bảo tính toàn vẹn của giao dịch (Atomicity, Consistency, Isolation, Durability)
Admin	Người quản trị hệ thống, có quyền cao nhất trong hệ thống
API	Giao diện lập trình ứng dụng (Application Programming Interface)
BaaS	Dịch vụ Backend (Backend as a Service)
CNTT	Công nghệ thông tin
CRUD	Các thao tác cơ bản trên dữ liệu (Create, Read, Update, Delete)
CSDL	Cơ sở dữ liệu
CSS	Ngôn ngữ định dạng trang web (Cascading Style Sheets)
DTO	Đối tượng truyền dữ liệu (Data Transfer Object)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
JPA	API trừu tượng hóa việc tương tác với CSDL trong Java (Java Persistence API)
MVC	Mô hình kiến trúc Model-View-Controller
ORM	Ánh xạ đối tượng quan hệ (Object-Relational Mapping)
RDBMS	Hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System)
REST	Kiến trúc truyền tải trạng thái đại diện (Representational State Transfer)

Thuật ngữ	Ý nghĩa
SPA	Ứng dụng trang đơn (Single Page Application)
SQL	Ngôn ngữ truy vấn có cấu trúc (Structured Query Language)
SV	Sinh viên
TourManager	Người quản lý chuyến du lịch trên hệ thống
UI	Giao diện người dùng (User Interface)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Ngành du lịch hiện đang trở thành một trong những ngành kinh tế trọng điểm, góp phần thúc đẩy sự phát triển kinh tế - xã hội ở nhiều quốc gia, trong đó có Việt Nam. Sự phát triển này không chỉ đem lại nguồn thu lớn mà còn giúp quảng bá hình ảnh đất nước, con người và nền văn hóa phong phú. Tuy nhiên, trước sự tăng trưởng nhanh chóng của lượng khách du lịch trong và ngoài nước, các công ty du lịch đang phải đối mặt với nhiều thách thức trong việc quản lý, tổ chức và đáp ứng các nhu cầu ngày càng đa dạng và phức tạp của khách hàng một cách hiệu quả và thuận tiện.

Cùng với đó, sự phát triển mạnh mẽ của công nghệ thông tin đã thúc đẩy các nền tảng quản lý và đặt chuyến du lịch trực tuyến trở thành công cụ không thể thiếu cho các doanh nghiệp trong ngành du lịch. Các hệ thống này giúp doanh nghiệp tối ưu hóa quy trình vận hành, giảm thiểu các công việc thủ công và nâng cao trải nghiệm khách hàng. Nhờ đó, việc quản lý thông tin chuyến du lịch, đặt chỗ, thanh toán và theo dõi phản hồi khách hàng trở nên dễ dàng và hiệu quả hơn.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu của đề án là xây dựng một hệ thống marketplace chuyến du lịch trực tuyến, giúp các công ty du lịch dễ dàng quản lý, tổ chức và theo dõi các chuyến du lịch trên cùng một nền tảng. Hệ thống sẽ cung cấp các chức năng chính như quản lý thông tin chuyến du lịch, đặt vé, thanh toán trực tuyến và đánh giá dịch vụ, góp phần nâng cao tính chuyên nghiệp và cải thiện trải nghiệm khách hàng.

Đề án tập trung phát triển ứng dụng web cho người dùng cuối và hệ thống quản lý dữ liệu liên quan. Giao diện được thiết kế trực quan, thân thiện, nhằm giúp khách hàng dễ dàng tìm kiếm và đăng ký dịch vụ. Đồng thời, hệ thống cũng hỗ trợ các công ty du lịch trong việc quản lý linh hoạt và tối ưu quy trình vận hành.

1.3 Định hướng giải pháp

Đề tài hướng tới xây dựng một nền tảng marketplace du lịch với kiến trúc client-server truyền thống. Backend được phát triển bằng Java Spring Boot để xử lý các nghiệp vụ và cung cấp API ổn định, trong khi frontend sử dụng ReactJS nhằm mang đến giao diện người dùng hiện đại, thân thiện và linh hoạt. Hệ thống tích hợp các giải pháp công nghệ như cổng thanh toán VnPay và xác thực bảo mật bằng JWT để đảm bảo tính an toàn và tiện lợi cho người dùng.

Giải pháp này giúp kết nối nhiều nhà cung cấp chuyến du lịch trên cùng một hệ

thống, từ đó khách hàng có thể dễ dàng so sánh, lựa chọn và đặt chuyến du lịch phù hợp với nhu cầu cá nhân. Đồng thời, các công ty du lịch cũng được hỗ trợ tối ưu hóa quy trình kinh doanh và nâng cao chất lượng dịch vụ.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức thành các chương chính như sau.

Chương 2, tôi sẽ tiến hành khảo sát và phân tích yêu cầu của bài toán. Mục đích chính của chương này là khảo sát các website tìm kiếm địa điểm du lịch đang phổ biến hiện nay, phân tích, so sánh ưu nhược điểm của chúng, từ đó giúp xác định được các chức năng chính và phi chức năng của hệ thống.

Trong Chương 3, tôi giới thiệu các công nghệ, công cụ và nền tảng được sử dụng trong dự án.

Tiếp theo chương 4, tôi sẽ tiến hành thiết kế kiến trúc cho ứng dụng, thực hiện kiểm thử và trình bày việc triển khai ứng dụng.

Với Chương 5, bao gồm những đóng góp nổi bật trong khi phát triển ứng dụng, những giải pháp đã được áp dụng để giải quyết bài toán.

Cuối cùng, Chương 6 là phần kết của đồ án, tôi sẽ nêu ra những việc đã đạt được và chưa đạt được của đề tài. Từ đó, đưa ra hướng phát triển trong tương lai.

Tiếp theo, tôi sẽ đi sâu phân tích và trình bày chi tiết những nội dung đã nêu trên trong báo cáo.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Khảo sát hiện trạng

Hiện nay, khác với các nước đã có nền công nghệ thông tin phổ biến và phát triển, tại Việt Nam, đa số các website vẫn là các trang tĩnh với cấu trúc và tổ chức thông tin cố định, ít thay đổi, chủ yếu mang tính giới thiệu công ty và các sản phẩm, dịch vụ. Bên cạnh đó, đã có nhiều website động (dynamic website) với giao diện, bố cục và cách thức quản lý đa dạng, ví dụ như website kinh doanh địa ốc của Công ty Hoàng Quân, website đặt phòng khách sạn trực tuyến của Công ty Thương mại điện tử Việt, hoặc các website của các ngân hàng thương mại và công ty du lịch như SaiGon Tourist, Sinh Cafe.

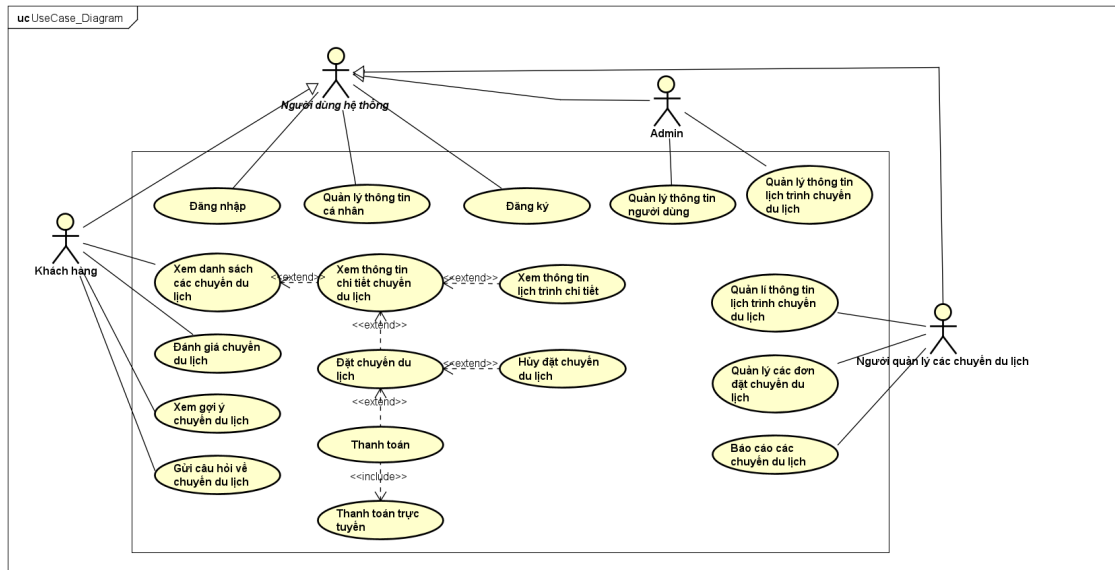
Tuy nhiên, thực tế cho thấy phần lớn các website này do các công ty thiết kế website thuê thực hiện và vận hành. Mặc dù với chi phí ban đầu có thể cao, các website động mang tính chuyên nghiệp và hoạt động ổn định, nhưng việc duy trì hiệu quả vận hành lâu dài còn phụ thuộc rất nhiều vào sự quản lý, cập nhật của chính các công ty sở hữu. Ở Việt Nam, vai trò của quản trị viên website chưa được đánh giá đúng mức, đa số người quản trị chỉ đảm nhận công việc này sau khi hoàn thành các nhiệm vụ khác, khiến việc cập nhật và làm mới thông tin trên website bị xem nhẹ. Điều này dẫn đến việc khách hàng thường xuyên cảm thấy nhàm chán và giảm thói quen truy cập.

Bên cạnh đó, nhu cầu tìm kiếm và so sánh các chuyến du lịch du lịch phù hợp đang là vấn đề được nhiều khách hàng quan tâm. Các hệ thống hiện tại chưa đáp ứng tốt nhu cầu cá nhân hóa và khả năng so sánh hiệu quả, gây khó khăn cho người dùng khi lựa chọn chuyến du lịch phù hợp.

Dựa trên các khảo sát và đánh giá trên, đề tài hướng tới xây dựng một nền tảng marketplace du lịch với kiến trúc client-server truyền thống, giúp kết nối nhiều nhà cung cấp chuyến du lịch trên cùng một hệ thống. Hệ thống sẽ cung cấp các tính năng quản lý thông tin hệ thống và hỗ trợ khách hàng trong việc tìm kiếm, so sánh, và đặt chuyến du lịch phù hợp với sở thích và nhu cầu cá nhân. Việc sử dụng nền tảng này không chỉ mang lại sự tiện lợi, nhanh gọn mà còn giúp khách hàng có cái nhìn tổng quan về các chuyến du lịch du lịch trước khi đưa ra quyết định hợp lý.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát của hệ thống

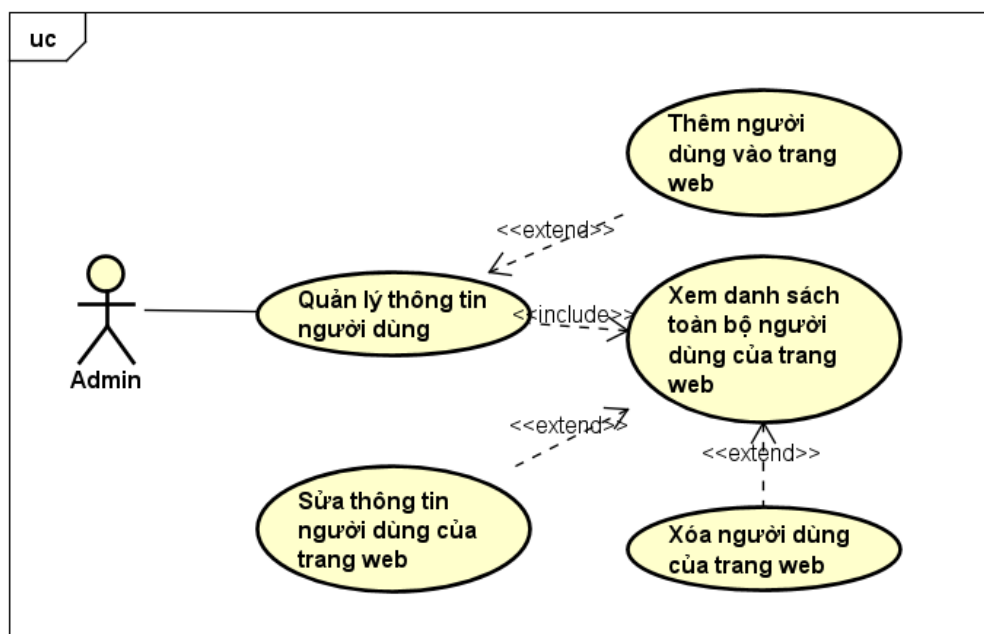
Biểu đồ 2.1 mô tả tổng quan các chức năng chính của hệ thống marketplace chuyến du lịch, bao gồm 4 tác nhân chính tương tác với hệ thống:

- **Khách hàng:** Là người sử dụng dịch vụ chuyến du lịch. Các chức năng chính bao gồm:
 - Đăng nhập, đăng ký tài khoản.
 - Xem danh sách các chuyến du lịch, thông tin chi tiết các chuyến đi.
 - Đặt chuyến du lịch, hủy chuyến, và thanh toán (bao gồm cả thanh toán trực tuyến).
 - Đánh giá chuyến du lịch, xem gợi ý và gửi câu hỏi về các chuyến đi.
- **Người quản lý các chuyến du lịch:** Chịu trách nhiệm quản lý thông tin các chuyến du lịch. Bao gồm:
 - Quản lý lịch trình chuyến du lịch: tạo mới, cập nhật.
 - Quản lý đơn đặt chuyến: xem, xử lý.
 - Xem báo cáo các chuyến du lịch.
- **Admin (Quản trị viên):** Có quyền cao nhất trong hệ thống.
 - Quản lý người dùng và thông tin người dùng.
 - Xem danh sách chuyến du lịch.

- Quản lý thông tin lịch trình (gồm cả thêm, sửa, xóa).
- **Người dùng hệ thống (vai trò trung gian):** Bao gồm các chức năng dùng chung như:
 - Đăng nhập, đăng ký tài khoản.
 - Quản lý thông tin cá nhân.

Biểu đồ use case tổng quát trên giúp hình dung rõ mối quan hệ giữa người dùng và các chức năng chính trong hệ thống, từ đó làm nền tảng cho việc thiết kế chức năng chi tiết và giao diện người dùng.

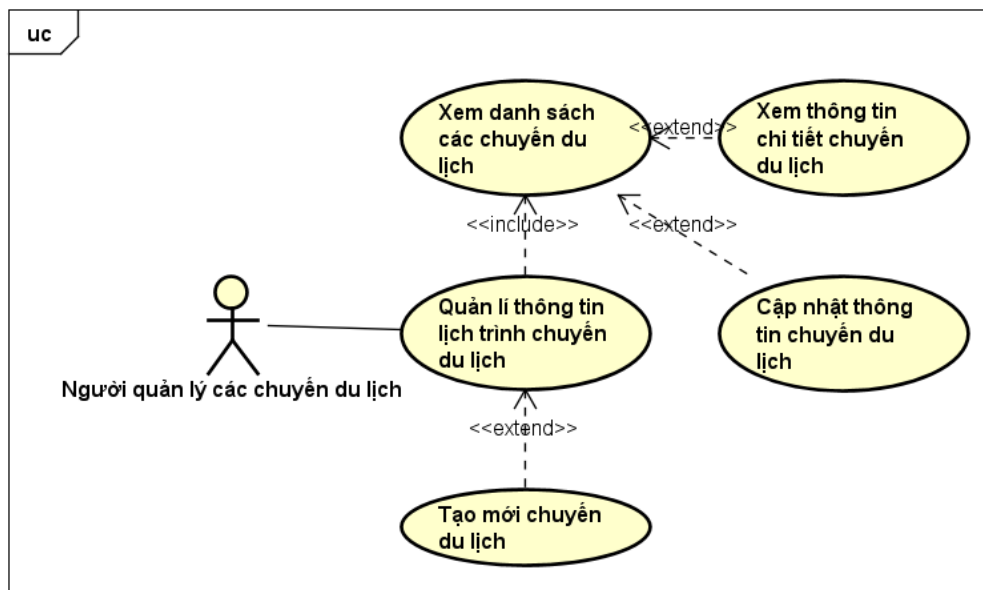
2.2.2 Biểu đồ use case phân rã Quản lý thông tin người dùng



Hình 2.2: Biểu đồ use case phân rã: Quản lý thông tin người dùng

Biểu đồ 2.2 mô tả cách quản trị viên thực hiện các thao tác quản trị người dùng trên hệ thống. Thông qua Use Case chính là "Quản lý thông tin người dùng", Admin có thể xem danh sách, thêm mới, chỉnh sửa và xóa người dùng của hệ thống. Các chức năng này cho phép quản trị viên kiểm soát hiệu quả cơ sở dữ liệu người dùng, đảm bảo tính chính xác, bảo mật và cập nhật thông tin kịp thời.

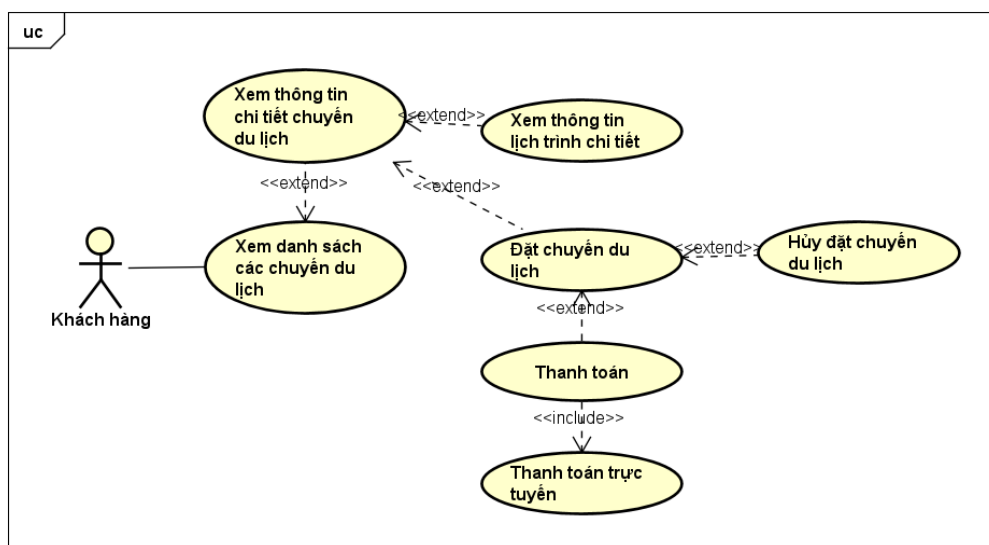
2.2.3 Biểu đồ use case phân rã Quản lý thông tin lịch trình chuyến du lịch



Hình 2.3: Biểu đồ use case phân rã: Quản lý thông tin lịch trình chuyến du lịch

Biểu đồ 2.3 mô tả việc người quản lý thực hiện các thao tác để tổ chức và cập nhật nội dung chuyến đi. Use Case chính là "Quản lý thông tin lịch trình chuyến du lịch", từ đó người quản lý có thể thực hiện các hành động như tạo mới chuyến đi, cập nhật thông tin, xem danh sách và xem chi tiết từng lịch trình cụ thể. Mô hình này giúp chuẩn hóa quy trình tổ chức chuyến du lịch và nâng cao hiệu quả vận hành hệ thống.

2.2.4 Biểu đồ use case phân rã Xem và đặt các chuyến du lịch



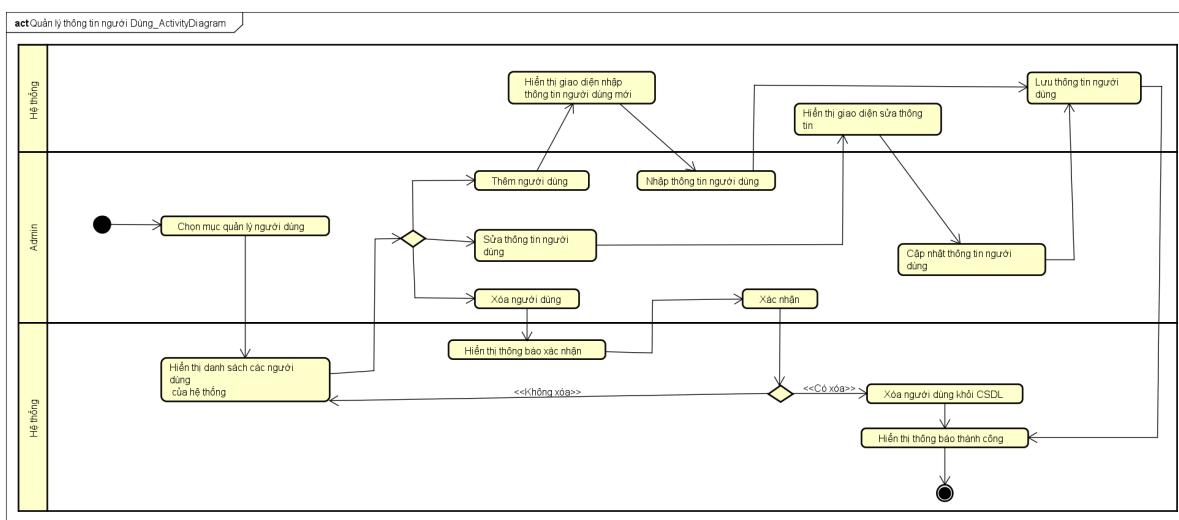
Hình 2.4: Biểu đồ use case phân rã: Xem và đặt các chuyến du lịch

Biểu đồ 2.4 mô tả quy trình khách hàng tương tác với hệ thống để tra cứu và đặt chuyến du lịch. Use Case chính là "Xem và đặt các chuyến du lịch", từ đó khách hàng có thể thực hiện các thao tác như xem thông tin các chuyến đi, xem chi tiết lịch trình, thực hiện đặt chuyến du lịch, thanh toán hoặc hủy đặt. Các chức năng này được phân tách rõ ràng giúp khách hàng dễ dàng tiếp cận thông tin và thực hiện các hành động theo nhu cầu du lịch cá nhân.

2.2.5 Quy trình nghiệp vụ

Trong phân hệ này, chúng ta sẽ xem xét tới một số quy trình chính dựa trên các usecase của từng tác nhân có trong hệ thống. Chi tiết về các hành động trong các quy trình này được mô hình hóa trong các mục con của từng quy trình.

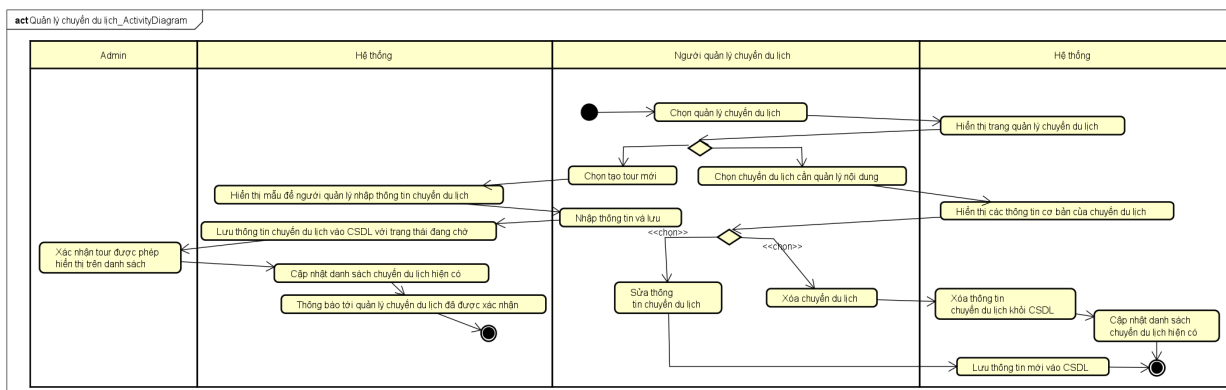
a, Quy trình quản lý người dùng (CRUD)



Hình 2.5: Biểu đồ quy trình quản lý người dùng(CRUD)

Hình 2.5 mô tả quá trình quản lý tài khoản của tất cả người dùng trên hệ thống của Admin bằng cách thêm mới, chỉnh sửa hoặc xóa tài khoản. Các thay đổi liên quan đến người dùng sẽ được hệ thống cập nhật và lưu trữ vào cơ sở dữ liệu, đảm bảo quản lý hiệu quả thông tin người dùng.

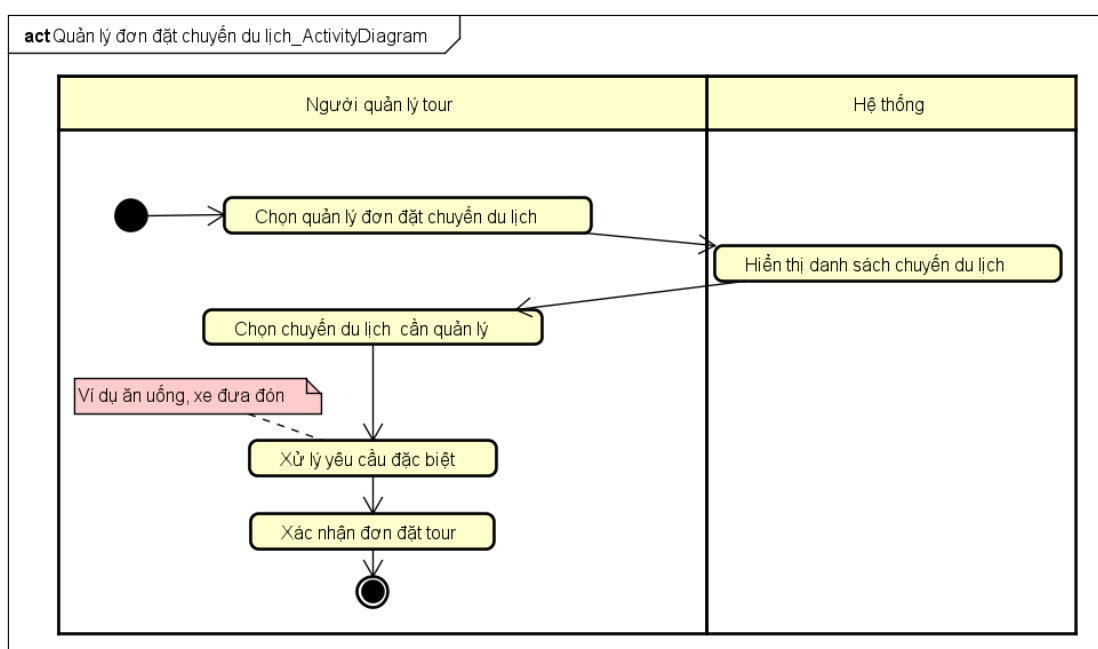
b, Quy trình quản lý nội dung chuyến du lịch (Người quản lý chuyến du lịch)



Hình 2.6: Biểu đồ quy trình quản lý nội dung chuyến du lịch

Hình ?? mô tả quá trình Người quản lý chuyến du lịch quản lý các chuyến du lịch bằng cách chọn chức năng "Quản lý chuyến du lịch". Tại đây, người quản lý có thể tạo mới, chỉnh sửa hoặc xóa thông tin chuyến du lịch. Khi chọn "Tạo chuyến du lịch mới", hệ thống sẽ hiển thị mẫu nhập thông tin để người quản lý điền và lưu vào cơ sở dữ liệu (CSDL). Nếu cần chỉnh sửa thông tin chuyến du lịch đã có, hệ thống cho phép sửa và cập nhật dữ liệu. Ngoài ra, người quản lý có thể xóa các chuyến du lịch không cần thiết, và hệ thống sẽ cập nhật danh sách chuyến du lịch hiện có.

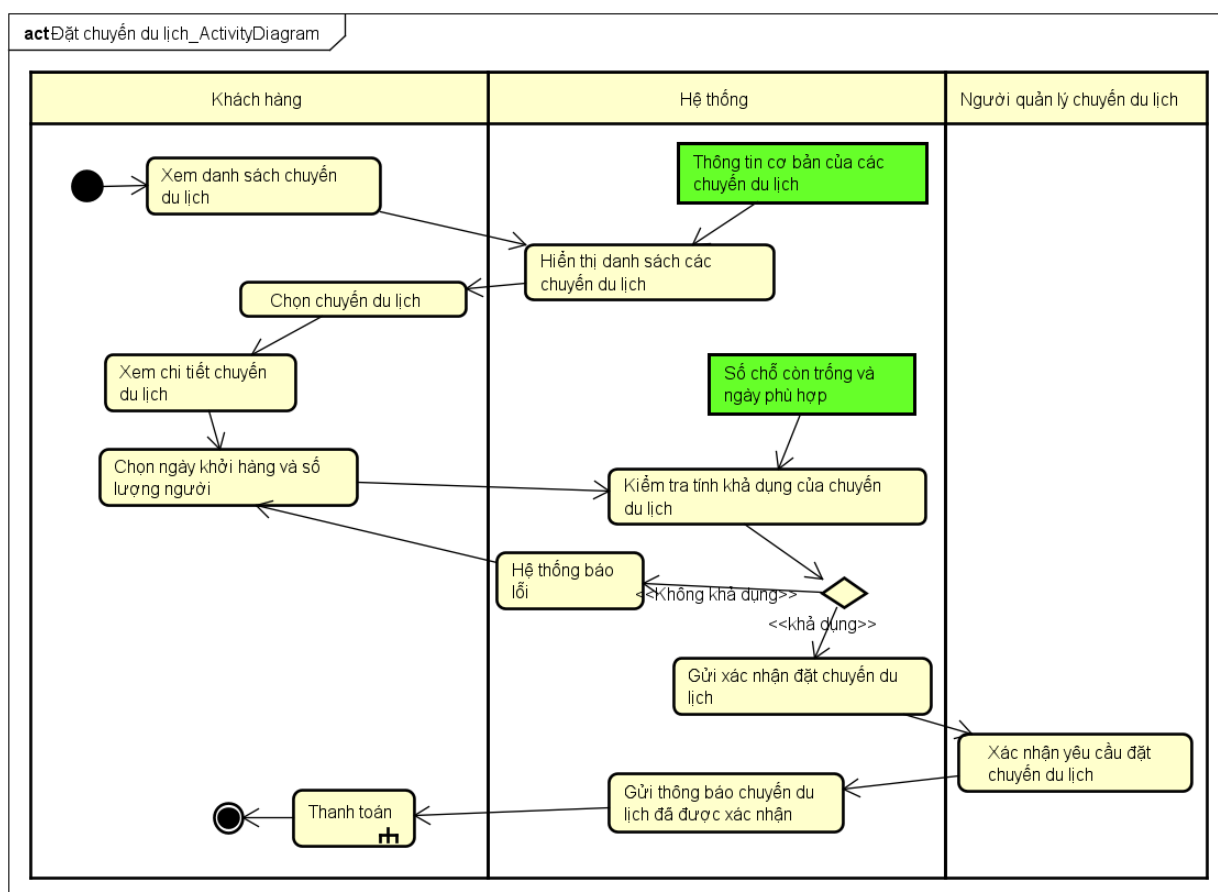
c, Quy trình quản lý đơn đặt chuyến du lịch



Hình 2.7: Biểu đồ quy trình quản lý đơn đặt chuyến du lịch

Hình 2.7 mô tả quá trình quản lý đơn đặt chuyến du lịch của người quản lý. Họ có thể xem danh sách các chuyến du lịch đã đặt, xác nhận yêu cầu đặt từ khách hàng và xử lý các yêu cầu đặc biệt từ khách hàng, như sắp xếp ăn uống hoặc xe đưa đón. Sau khi xử lý, hệ thống cập nhật trạng thái .

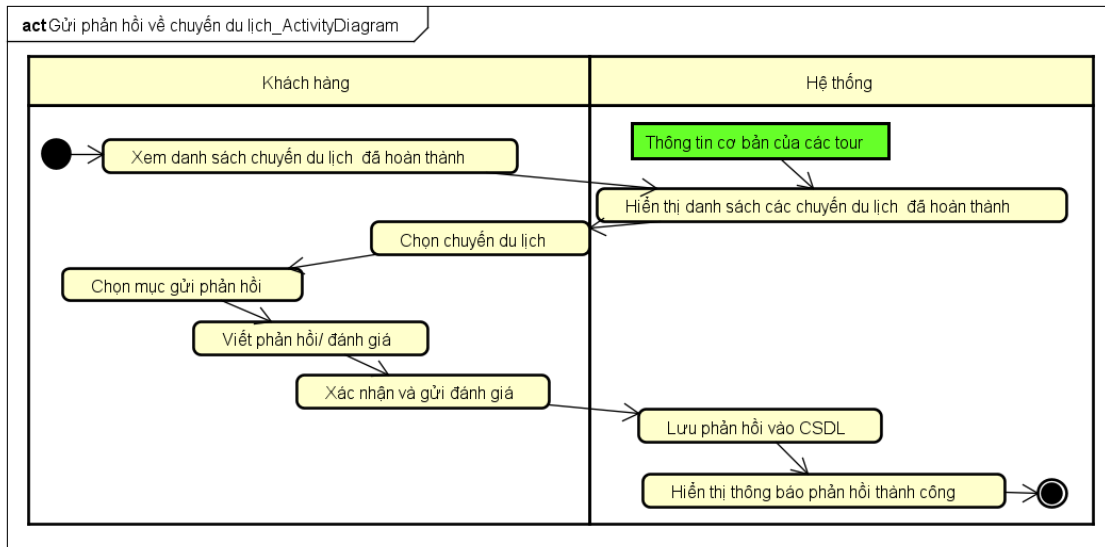
d, Quy trình đặt chuyến du lịch (khách hàng)



Hình 2.8: Biểu đồ quy trình đặt chuyến du lịch

Hình 2.8 mô tả quá trình đặt chuyến du lịch của khách hàng. Sau khi đăng nhập vào hệ thống, khách hàng có thể truy cập mục "Danh sách chuyến du lịch" để xem các chuyến du lịch hiện có. Hệ thống sẽ hiển thị thông tin chuyến du lịch, bao gồm tên chuyến du lịch, mô tả, và người phụ trách. Khách hàng có thể chọn chuyến du lịch mà họ quan tâm và sau đó chọn ngày khởi hành và số lượng người tham gia. Hệ thống sẽ kiểm tra tính khả dụng của chuyến du lịch (chỗ trống và ngày phù hợp) và gửi xác nhận đặt chuyến du lịch nếu chuyến du lịch khả dụng. Sau khi người quản lý xác nhận yêu cầu đặt của khách hàng, họ sẽ tiếp tục quy trình với bước thanh toán để hoàn tất đặt chuyến du lịch.

e, Quy trình gửi đánh giá phản hồi chuyến du lịch đã hoàn thành (Khách hàng)



Hình 2.9: Biểu đồ quy trình gửi đánh giá phản hồi chuyến du lịch đã hoàn thành (Khách hàng)

Hình 2.8 mô tả quá trình gửi đánh giá phản hồi về chuyến du lịch đã hoàn thành của khách hàng. Họ xem danh sách chuyến du lịch đã hoàn thành, chọn chuyến du lịch muốn đánh giá, và viết phản hồi. Sau khi xác nhận và gửi đánh giá, hệ thống lưu phản hồi vào cơ sở dữ liệu và hiển thị thông báo gửi phản hồi thành công cho khách hàng.

2.3 Đặc tả chức năng

2.3.1 Usecase quản lý người dùng

Mã Use case	UC001	Tên Use case	Quản lý thông tin người dùng
Tác nhân	Admin		
Tiền điều kiện	Admin đã truy cập hệ thống		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Admin	Truy cập mục quản lý thông tin người dùng
	2	Hệ thống	Hiển thị danh sách người dùng
	3	Admin	Chọn người dùng muốn chỉnh sửa
	4	Hệ thống	Hiển thị form chỉnh sửa thông tin người dùng
	5	Admin	Cập nhật thông tin và nhấn "Lưu"
	6	Hệ thống	Kiểm tra tính hợp lệ của thông tin và lưu lại
	7	Hệ thống	Thông báo cập nhật thành công tới Admin
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	6a	Hệ thống	Thông báo lỗi nếu thông tin không hợp lệ, yêu cầu chỉnh sửa
	6b	Hệ thống	Thông báo lỗi nếu không kết nối được cơ sở dữ liệu
Hậu điều kiện	Thông tin người dùng được cập nhật thành công và lưu trữ trong hệ thống		

Bảng 2.1: Bảng thông tin chính cho Use case Quản lý thông tin người dùng

Dữ liệu đầu vào					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Tên người dùng	Tên của người dùng cần cập nhật	Có	Không rỗng	"Nguyễn Văn A"
2	Email	Địa chỉ email của người dùng	Có	Định dạng email hợp lệ	example@1.com
3	Vai trò	Vai trò của người dùng (Admin, User, etc.)	Có	Chỉ chấp nhận các vai trò được định nghĩa	"User"

Bảng 2.2: Bảng dữ liệu đầu vào cho Use case CRUD Quản lý thông tin người dùng

Dữ liệu đầu ra					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Thông báo cập nhật	Xác nhận kết quả cập nhật thông tin người dùng	Có	"Thành công" hoặc "Thất bại"	"Cập nhật thành công"
2	Thông báo lỗi (nếu có)	Lý do cập nhật thất bại	Không	Chỉ hiển thị khi cập nhật thất bại	"Email đã tồn tại"

Bảng 2.3: Bảng dữ liệu đầu ra cho Use case Quản lý thông tin người dùng

2.3.2 Usecase quản lí chuyến du lịch

Mã Use case	UC02	Tên Use case	Quản lý chuyến du lịch
Tác nhân	Người quản lý chuyến du lịch		
Tiền điều kiện	Người quản lý đã đăng nhập và có quyền quản lý chuyến du lịch		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Người quản lý	Truy cập mục quản lý chuyến du lịch
	2	Hệ thống	Hiển thị danh sách chuyến du lịch hiện có
	3	Người quản lý	Chọn "Tạo chuyến du lịch mới" hoặc chọn chuyến du lịch cần chỉnh sửa
	4	Hệ thống	Hiển thị form nhập thông tin hoặc thông tin cơ bản của chuyến du lịch
	5	Người quản lý	Nhập hoặc chỉnh sửa thông tin và lưu lại
	6	Hệ thống	Lưu thông tin vào CSDL với trạng thái đang chờ
	7	Admin	Xác nhận chuyến du lịch
	8	Hệ thống	Cập nhật lên trang chủ danh sách chuyến du lịch
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	7a	Admin	Không xác nhận chuyến du lịch, tour về trạng thái bị từ chối, không hiển thị trên trang chủ
	6a	Hệ thống	Thông báo lỗi nếu không kết nối được cơ sở dữ liệu
Hậu điều kiện	Chuyến du lịch được tạo mới hoặc cập nhật thành công và có sẵn trong hệ thống		

Bảng 2.4: Thông tin chi tiết về Use Case Quản lý chuyến du lịch

Dữ liệu đầu vào					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Tên chuyến du lịch	Tên của chuyến du lịch	Có	Không rỗng	"chuyến du lịch Hà Nội"
2	Giá chuyến du lịch	Chi phí cho mỗi khách hàng tham gia	Có	Số nguyên dương	500000
3	Ngày khởi hành	Ngày bắt đầu của chuyến du lịch	Có	Định dạng ngày hợp lệ	"01/01/2025"
4	Thông tin mô tả	Mô tả nội dung của chuyến du lịch	Có	Chuỗi văn bản không rỗng	"Khám phá thành phố Hà Nội"
5	Thông tin lịch trình	Lịch trình chi tiết cho chuyến du lịch	Không	Chuỗi văn bản	"Ngày 1: Thăm Hồ Gươm..."

Bảng 2.5: Dữ liệu đầu vào cho Use Case Quản lý chuyến du lịch

Dữ liệu đầu ra					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Thông báo lưu thay đổi	Xác nhận kết quả lưu nội dung chuyến du lịch	Có	"Thành công" hoặc "Thất bại"	"Cập nhật thành công"
2	Thông báo lỗi (nếu có)	Lý do lưu nội dung thất bại	Không	Chỉ hiển thị khi có lỗi	"Tên chuyến du lịch không hợp lệ"

Bảng 2.6: Dữ liệu đầu ra cho Use Case Quản lý chuyến du lịch

2.3.3 Usecase quản lý đơn đặt chuyến du lịch

Mã Use case	UC03	Tên Use case	Quản lý đơn đặt chuyến du lịch
Tác nhân	Người quản lý chuyến du lịch		
Tiền điều kiện	Có đơn đặt chuyến du lịch cần được quản lý		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Người quản lý	Chọn quản lý đơn đặt chuyến du lịch
	2	Hệ thống	Hiển thị danh sách chuyến du lịch có đơn đặt
	3	Người quản lý	Chọn chuyến du lịch cần quản lý đơn đặt và xử lý các yêu cầu đặc biệt
	4	Hệ thống	Cập nhật trạng thái đơn đặt chuyến du lịch
	5	Người quản lý	Ghi chú bổ sung nếu cần và kết thúc quản lý
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	2a	Hệ thống	Thông báo lỗi nếu không kết nối được cơ sở dữ liệu
	3a	Người quản lý	Không xử lý được yêu cầu đặc biệt, từ chối đơn đặt chuyến du lịch, đơn đặt sẽ bị hủy và thông báo cho người dùng
Hậu điều kiện	Trạng thái đơn đặt chuyến du lịch được cập nhật và lưu trữ		

Bảng 2.7: Bảng thông tin chính cho Use case Quản lý đơn đặt chuyến du lịch

Dữ liệu đầu vào					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Yêu cầu đặc biệt	Các yêu cầu đặc biệt như ăn uống, xe đưa đón	Không	Chuỗi văn bản hoặc không có	"Yêu cầu đồ ăn chay"
2	Trạng thái đơn đặt	Trạng thái hiện tại của đơn đặt	Có	Trạng thái hợp lệ (Pending, Confirmed, etc.)	"Confirmed"

Bảng 2.8: Bảng dữ liệu đầu vào cho Use case Quản lý đơn đặt chuyến du lịch

Dữ liệu đầu ra					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Xác nhận cập nhật trạng thái	Thông báo xác nhận thay đổi trạng thái đơn đặt	Có	"Thành công" hoặc "Thất bại"	"Cập nhật thành công"

Bảng 2.9: Bảng dữ liệu đầu ra cho Use case Quản lý đơn đặt chuyến du lịch

2.3.4 Usecase đặt chuyến du lịch

Mã Use case	UC04	Tên Use case	Đặt chuyến du lịch
Tác nhân	Người dùng (Khách hàng)		
Tiền điều kiện	Khách hàng đã đăng nhập thành công và đã xem danh sách các chuyến du lịch.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Người dùng	Chọn chuyến du lịch cần đặt từ danh sách chuyến du lịch
	2	Hệ thống	Hiển thị thông tin chi tiết của chuyến du lịch (ngày khởi hành, số chỗ trống, giá)
	3	Người dùng	Chọn ngày khởi hành và số lượng người tham gia
	4	Hệ thống	Kiểm tra tính khả dụng của chuyến du lịch và gửi yêu cầu đặt chuyến du lịch nếu chuyến du lịch khả dụng
	5	Người quản lý chuyến du lịch	Xác nhận đặt chuyến du lịch
	6	Hệ thống	Gửi thông báo cho người dùng chuyến du lịch đã được xác nhận và vui lòng chuyển sang quy trình thanh toán
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	4a	Hệ thống	Thông báo lỗi: chuyến du lịch không khả dụng vào ngày và số lượng người yêu cầu
	5a	Người quản lý chuyến du lịch	Không xác nhận đặt chuyến du lịch, đơn đặt chuyến du lịch về trạng thái bị hủy, thông báo về cho người dùng
Hậu điều kiện	Chuyến du lịch được đặt thành công và thông tin đặt chuyến du lịch chuyển đến quy trình thanh toán.		

Bảng 2.10: Thông tin chi tiết về Use Case Đặt chuyến du lịch

Dữ liệu đầu vào					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Ngày khởi hành	Ngày khách hàng chọn để tham gia chuyến du lịch	Có	Phải là ngày còn khả dụng	15/12/2024
2	Số lượng người	Số người tham gia trong một chuyến du lịch	Có	Phải là số nguyên dương và không vượt quá số chỗ trống	3

Bảng 2.11: Dữ liệu đầu vào khi đặt chuyến du lịch

Dữ liệu đầu ra					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Xác nhận đặt chuyến du lịch	Thông báo xác nhận chuyến du lịch đã được đặt thành công	Có	Nội dung thông báo xác nhận	"Đặt chuyến du lịch thành công"
2	Thông báo lỗi (nếu có)	Thông báo lỗi khi chuyến du lịch không khả dụng hoặc có lỗi khác	Không	Chỉ hiển thị khi đặt chuyến du lịch không thành công	"Chuyến du lịch không khả dụng vào ngày đã chọn"

Bảng 2.12: Dữ liệu đầu ra khi đặt chuyến du lịch

2.3.5 Usecase gửi đánh giá, phản hồi chuyến du lịch đã hoàn thành

Mã Use case	UC05	Tên Use case	Đánh giá chuyến du lịch
Tác nhân	Người dùng (Khách hàng)		
Tiền điều kiện	Chuyến du lịch đã hoàn thành và khách hàng đã tham gia chuyến đi		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Người dùng	Truy cập mục đánh giá chuyến du lịch đã hoàn thành
	2	Người dùng	Chọn chuyến du lịch muốn đánh giá
	3	Người dùng	Nhập nội dung đánh giá và chọn điểm đánh giá (sao)
	4	Hệ thống	Lưu đánh giá vào cơ sở dữ liệu
	5	Hệ thống	Thông báo đánh giá thành công tới người dùng
Hậu điều kiện	Đánh giá được lưu trữ và hiển thị trên hệ thống		

Bảng 2.13: Thông tin chi tiết về Use Case Đánh giá chuyến du lịch

Dữ liệu đầu vào					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Nội dung đánh giá	Phản hồi của khách hàng về chuyến du lịch	Có	Chuỗi văn bản không rỗng	"Chuyến du lịch tuyệt vời, rất hài lòng!"
2	Điểm đánh giá (Sao)	Số điểm đánh giá chuyến du lịch	Có	Điểm từ 1 đến 5 sao	5

Bảng 2.14: Dữ liệu đầu vào cho Use case Đánh giá chuyến du lịch

Dữ liệu đầu ra					
STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	Thông báo gửi phản hồi	Xác nhận phản hồi thành công	Có	"Thành công" hoặc "Thất bại"	"Phản hồi thành công"
2	Thông báo lỗi (nếu có)	Lý do phản hồi không thành công	Không	Chỉ hiển thị khi phản hồi không thành công	"Không thể lưu phản hồi"

Bảng 2.15: Dữ liệu đầu ra cho Use case Đánh giá chuyến du lịch

2.4 Yêu cầu phi chức năng

Phần này xác định các yêu cầu phi chức năng nhằm đảm bảo hệ thống marketplace du lịch hoạt động ổn định, hiệu quả, an toàn và đáp ứng tốt trải nghiệm người dùng trong kiến trúc client-server truyền thống.

2.4.1 Hiệu suất

Website cần có thời gian phản hồi nhanh, với trang chủ và các trang thông tin chính phải tải trong vòng dưới 3 giây để tối ưu trải nghiệm người dùng. Khi có nhiều người dùng truy cập đồng thời (từ 500 đến 1000 người dùng cùng lúc), hệ thống vẫn phải duy trì tốc độ xử lý ổn định, không bị gián đoạn hay quá tải. Các thao tác tìm kiếm, lọc và so sánh chuyến du lịch phải được thực hiện trong thời gian không quá 2 giây, nhằm đảm bảo tính tương tác mượt mà.

2.4.2 Tính khả dụng

Hệ thống phải hoạt động liên tục 24/7, không bị gián đoạn để đảm bảo việc đặt chuyến du lịch của khách hàng không bị ảnh hưởng, đặc biệt trong các mùa du lịch cao điểm. Ngoài ra, hệ thống cần có cơ chế dự phòng, sao lưu dữ liệu định kỳ hàng ngày và khả năng phục hồi nhanh chóng khi có sự cố xảy ra, nhằm giảm thiểu mất mát dữ liệu và đảm bảo tính liên tục kinh doanh cho các nhà cung cấp.

2.4.3 Bảo mật

Hệ thống phải bảo vệ nghiêm ngặt thông tin cá nhân và dữ liệu giao dịch của người dùng, ngăn chặn truy cập trái phép và các hành vi tấn công mạng. Tính toàn vẹn dữ liệu cần được đảm bảo để tránh bị thay đổi hoặc hỏng hóc. Hệ thống sử dụng các cơ chế xác thực và ủy quyền người dùng dựa trên tiêu chuẩn bảo mật hiện đại (như JWT), nhằm phân quyền rõ ràng, chỉ cho phép những người dùng được phép truy cập và thực hiện các chức năng tương ứng.

2.4.4 Khả năng mở rộng

Thiết kế hệ thống theo kiến trúc module và phân tán để dễ dàng mở rộng khi lượng người dùng tăng hoặc số lượng chuyến du lịch nhiều lên. Cơ sở hạ tầng phải hỗ trợ cân bằng tải, phân phối tài nguyên hiệu quả cả về phần mềm và phần cứng, đảm bảo trải nghiệm người dùng không bị giảm sút khi lưu lượng truy cập đột ngột tăng cao, phù hợp với đặc thù marketplace có nhiều nhà cung cấp và khách hàng cùng tương tác.

2.4.5 Khả năng bảo trì và quản lý

Mã nguồn hệ thống phải rõ ràng, có tài liệu chi tiết kèm theo để dễ dàng bảo trì và cập nhật tính năng mới. Hệ thống quản trị (CMS) thiết kế thân thiện, trực quan, cho phép các nhà quản trị dễ dàng quản lý thông tin chuyến du lịch, đơn hàng, và

dữ liệu khách hàng mà không đòi hỏi kiến thức kỹ thuật chuyên sâu. Điều này giúp giảm chi phí vận hành và tăng tính linh hoạt trong quản lý.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Ở Chương 2, tôi đã khảo sát và phân tích yêu cầu hệ thống marketplace du lịch. Trong chương này, tôi sẽ trình bày tổng quan các công nghệ, nền tảng được áp dụng để giải quyết các yêu cầu đã nêu, đồng thời phân tích lý do lựa chọn và so sánh các công nghệ tương tự.

3.1 Frontend – ReactJS, Vite và TailwindCSS

Phần giao diện người dùng của hệ thống được xây dựng bằng ReactJS – một thư viện JavaScript phổ biến hỗ trợ phát triển các ứng dụng web động thông qua cơ chế component hóa. Việc sử dụng React giúp tối ưu khả năng tái sử dụng mã nguồn và tổ chức giao diện theo hướng cấu trúc rõ ràng. Để tăng tốc quá trình build và cải thiện hiệu suất, dự án sử dụng thêm công cụ bundler **Vite**, cho phép khởi động nhanh và cập nhật hot module hiệu quả trong môi trường phát triển.

Một số ưu điểm nổi bật của ReactJS bao gồm:

- **Component hóa:** Toàn bộ giao diện được chia thành các component độc lập, mỗi component chịu trách nhiệm hiển thị và xử lý logic của riêng mình. Điều này giúp tái sử dụng mã nguồn hiệu quả và tăng tính tổ chức trong dự án.
- **Virtual DOM:** React sử dụng Virtual DOM – một bản sao ảo của DOM thực tế – để theo dõi thay đổi trạng thái và cập nhật giao diện một cách tối ưu. Khi có sự thay đổi, React sẽ chỉ cập nhật các phần tử cần thiết thay vì làm mới toàn bộ cây DOM, giúp cải thiện hiệu suất đáng kể.
- **Cập nhật dữ liệu linh hoạt:** React hỗ trợ quản lý trạng thái thông qua cơ chế `useState`, `useReducer`, và các thư viện như Redux hoặc Context API, giúp xử lý các luồng dữ liệu phức tạp trong ứng dụng một cách rõ ràng và hiệu quả.
- **Khả năng mở rộng cao:** Với hệ sinh thái phong phú, React dễ dàng tích hợp với các thư viện và framework khác như React Router, Axios, Redux Toolkit, Material UI, giúp mở rộng tính năng mà không làm ảnh hưởng đến kiến trúc tổng thể.
- **Thân thiện với SEO khi kết hợp backend:** Mặc dù React là thư viện hướng client-side, nhưng khi kết hợp với backend như Spring Boot, hệ thống có thể hỗ trợ tối ưu hóa SEO thông qua việc tạo sitemap, cấu trúc URL rõ ràng và sử dụng các kỹ thuật như prerendering hoặc SSR thông qua các công cụ hỗ trợ. Điều này giúp cải thiện khả năng hiển thị của trang web trên các công cụ tìm kiếm.

- **Cộng đồng phát triển lớn:** React có một cộng đồng rất mạnh và tài liệu đầy đủ, giúp việc học tập, trao đổi và giải quyết lỗi trở nên thuận tiện. Nhiều thư viện hiện đại đều tương thích tốt với React, tạo điều kiện cho việc phát triển nhanh chóng và duy trì dễ dàng.

Với những đặc điểm này, ReactJS là lựa chọn hàng đầu trong việc xây dựng các ứng dụng web dạng SPA (Single Page Application), đặc biệt trong những dự án yêu cầu hiệu suất cao, khả năng tái sử dụng và mở rộng lâu dài.

Về mặt trình bày giao diện, hệ thống áp dụng **Tailwind CSS**, một framework CSS dạng utility-first, giúp lập trình viên xây dựng giao diện nhất quán và linh hoạt chỉ bằng cách kết hợp các lớp tiện ích sẵn có. Điều này góp phần giảm thiểu mã CSS dư thừa và tăng tốc quá trình thiết kế.

Việc điều hướng giữa các trang được thực hiện bằng **React Router v6**, cho phép xây dựng các route rõ ràng và linh hoạt, giúp cải thiện trải nghiệm người dùng trong ứng dụng dạng SPA (Single Page Application). Ngoài ra, thư viện **Axios** được sử dụng để giao tiếp với backend thông qua các API, hỗ trợ gửi và nhận dữ liệu một cách thuận tiện và bảo mật.

Để xử lý biểu mẫu và quản lý trạng thái đầu vào, hệ thống tích hợp **React Hook Form**, một thư viện nhẹ nhưng mạnh mẽ giúp đơn giản hóa quá trình xác thực, kiểm soát và gửi dữ liệu form.

Tổ hợp công nghệ này không chỉ giúp xây dựng giao diện hiện đại, hiệu suất cao mà còn đảm bảo khả năng mở rộng và bảo trì dễ dàng trong các giai đoạn phát triển sau này.

3.2 Backend – Java Spring Boot

Java Spring Boot là một framework phát triển ứng dụng mã nguồn mở được xây dựng trên nền tảng Java, hỗ trợ phát triển các ứng dụng web và dịch vụ RESTful hiệu quả. Spring Boot nổi bật với khả năng khởi tạo nhanh, cấu hình tối giản và hỗ trợ mạnh mẽ cho việc xây dựng các API có khả năng mở rộng cao, đáp ứng nhu cầu của các hệ thống phân tán và client-server truyền thống.

Framework này cung cấp nhiều tính năng hiện đại như Dependency Injection (DI), cơ chế quản lý giao dịch, bảo mật với Spring Security, và tích hợp dễ dàng với các hệ quản trị cơ sở dữ liệu phổ biến. Spring Boot hỗ trợ xây dựng các microservices cũng như các ứng dụng backend monolithic, cho phép xử lý hàng ngàn yêu cầu đồng thời với hiệu suất ổn định.

Các lựa chọn thay thế cho Spring Boot bao gồm Node.js với Express hoặc ASP.NET Core. Tuy nhiên, Spring Boot được ưu tiên do hệ sinh thái Java rộng

lớn, hỗ trợ mạnh mẽ về bảo mật, khả năng mở rộng, cũng như tính ổn định trong các môi trường doanh nghiệp. Ngoài ra, cộng đồng phát triển rộng và tài liệu phong phú giúp việc bảo trì và phát triển trở nên thuận tiện.

3.3 Hệ quản trị cơ sở dữ liệu – PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) mã nguồn mở, được phát triển từ năm 1986 tại Đại học California, Berkeley. Đây là hệ quản trị cơ sở dữ liệu mạnh mẽ, có khả năng xử lý dữ liệu có cấu trúc phức tạp với hiệu suất và độ tin cậy cao.

Được thiết kế để quản lý và lưu trữ dữ liệu, PostgreSQL cho phép người dùng truy vấn, thao tác và quản lý dữ liệu một cách hiệu quả và an toàn. PostgreSQL là một trong những hệ quản trị cơ sở dữ liệu phổ biến nhất trên thế giới và được sử dụng rộng rãi trong các doanh nghiệp.

Các tính năng nổi bật của PostgreSQL bao gồm:

- **Quản lý dữ liệu:** PostgreSQL hỗ trợ đầy đủ các thao tác CRUD (Create, Read, Update, Delete) trên các bảng và mối quan hệ, đồng thời cung cấp khả năng thao tác dữ liệu nâng cao với các kiểu dữ liệu phong phú như JSON, XML, hstore và mảng.
- **Bảo mật dữ liệu:** Hệ thống cho phép mã hóa kết nối, sử dụng chính sách kiểm soát truy cập dựa trên vai trò (Role-based access control), và hỗ trợ các phương thức xác thực như MD5, SCRAM-SHA-256 để đảm bảo an toàn thông tin.
- **Quản lý giao dịch:** PostgreSQL tuân thủ nghiêm ngặt các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo độ tin cậy của các giao dịch trong môi trường nhiều người dùng đồng thời.
- **Chỉ mục và hiệu suất:** Hệ quản trị này hỗ trợ nhiều loại chỉ mục như B-tree, Hash, GiST, GIN và SP-GiST giúp tối ưu tốc độ truy vấn trong các tình huống dữ liệu lớn, đồng thời cho phép lập kế hoạch thực thi truy vấn thông minh.
- **Sao lưu và phục hồi:** PostgreSQL hỗ trợ cơ chế sao lưu toàn bộ cơ sở dữ liệu hoặc theo điểm thời gian (Point-in-time Recovery – PITR), giúp bảo vệ dữ liệu khỏi các sự cố hoặc thao tác sai.
- **Mở rộng và tùy biến:** Cho phép người dùng định nghĩa thêm các hàm tùy chỉnh (User-defined functions), trigger và các *extension* như PostGIS (xử lý dữ liệu không gian), pgAudit (kiểm toán), giúp mở rộng khả năng xử lý dữ liệu chuyên sâu.

Một số hệ quản trị cơ sở dữ liệu tương tự có thể sử dụng thay thế như MySQL, MariaDB hoặc Oracle. Tuy nhiên, PostgreSQL được lựa chọn nhờ khả năng mở rộng tốt, tính ổn định cao, hiệu năng xử lý truy vấn mạnh và cộng đồng phát triển lớn, giúp dễ dàng tìm tài liệu và hỗ trợ khi cần thiết.

3.4 Nền tảng dịch vụ cơ sở dữ liệu – Supabase

Supabase là một nền tảng mã nguồn mở thuộc mô hình Backend-as-a-Service (BaaS), cung cấp giải pháp toàn diện cho việc xây dựng backend mà không cần thiết lập máy chủ riêng. Supabase được phát triển trên nền PostgreSQL, cho phép người dùng tạo và quản lý cơ sở dữ liệu, đồng thời tự động sinh ra API RESTful và GraphQL dựa trên schema có sẵn.

Supabase cung cấp nhiều tính năng hỗ trợ phát triển backend một cách nhanh chóng và hiệu quả. Các tính năng chính bao gồm:

- **Tự động sinh API:** Supabase tự động tạo API RESTful và GraphQL từ schema cơ sở dữ liệu PostgreSQL mà không cần lập trình thủ công.
- **Quản lý cơ sở dữ liệu trực quan:** Giao diện trực quan cho phép tạo bảng, thiết lập quan hệ, chỉ mục, truy vấn SQL và thực hiện migration một cách dễ dàng.
- **Xác thực và phân quyền:** Cung cấp hệ thống xác thực người dùng với email/mật khẩu, OAuth, magic link, kết hợp với Row Level Security để kiểm soát quyền truy cập ở cấp độ từng dòng dữ liệu.
- **Realtime:** Hỗ trợ cập nhật dữ liệu thời gian thực giữa server và client thông qua WebSocket, phù hợp với các ứng dụng cần đồng bộ liên tục.
- **Lưu trữ tệp (Storage):** Cho phép tải lên, quản lý và phân phối tệp tin, hỗ trợ tạo URL công khai hoặc có thời hạn, đồng thời tích hợp với hệ thống xác thực để bảo vệ tài nguyên.
- **Khả năng mở rộng:** Hệ thống được thiết kế có thể self-host hoặc triển khai trên nền tảng cloud, phù hợp cho các ứng dụng cần kiểm soát hạ tầng và dữ liệu.
- **Mã nguồn mở:** Supabase được phát triển trên triết lý mã nguồn mở hoàn toàn, giúp người dùng dễ dàng kiểm tra, tùy chỉnh và triển khai theo nhu cầu thực tế.

So với các nền tảng như Firebase hay Hasura, Supabase nổi bật với tính mở, linh hoạt và khả năng self-host nếu cần.

3.5 Quản lý hình ảnh – Cloudinary

Cloudinary là một dịch vụ quản lý hình ảnh dựa trên nền tảng đám mây, cung cấp giải pháp toàn diện từ tải lên, lưu trữ, xử lý, tối ưu hóa đến phân phối hình ảnh. Cloudinary cho phép người dùng dễ dàng upload ảnh trực tiếp từ trình duyệt hoặc ứng dụng mà không cần cấu hình phức tạp.

Dịch vụ này hỗ trợ xây dựng các URL động để thực hiện các thao tác xử lý hình ảnh như thay đổi kích thước, cắt xén, điều chỉnh màu sắc một cách linh hoạt và tự động. Ngoài ra, Cloudinary còn tích hợp nhiều API và các plugin giúp tương tác dễ dàng với các framework và hệ quản trị cơ sở dữ liệu phổ biến.

Các tính năng nổi bật mà **Cloudinary** cung cấp bao gồm:

- **Tạo URL để xử lý hình ảnh:** Cloudinary cho phép tạo các URL động giúp thực hiện các thao tác chuyển đổi hình ảnh như cắt, phóng to, thu nhỏ... một cách linh hoạt.
- **Tích hợp với view engine của Rails:** Cung cấp các helper cho framework **Rails**, hỗ trợ chèn và thay đổi hình ảnh ngay trong **view** một cách dễ dàng.
- **Gói API wrappers:** Bao gồm các API wrappers cho phép người dùng thực hiện các thao tác như tải ảnh, quản lý thư viện hình ảnh, cập nhật thông tin hoặc xóa ảnh.
- **Tải ảnh từ trình duyệt:** Cloudinary cung cấp plugin **jQuery** cho phép tải ảnh trực tiếp từ trình duyệt mà không cần qua backend, giúp tối ưu hiệu suất xử lý phía client.
- **Tích hợp Active Record:** Hệ thống hỗ trợ kết nối với **Active Record**, giúp lưu trữ và thao tác hình ảnh gắn liền với cơ sở dữ liệu một cách trực quan, thuận tiện.
- **Plugin CarrierWave:** Hỗ trợ tích hợp plugin **CarrierWave**, giúp các ứng dụng Ruby dễ dàng thao tác, lưu trữ và hiển thị ảnh động bộ với các gem phổ biến.
- **Phân phối hình ảnh qua CDN:** Hình ảnh được phân phối qua **CDN** toàn cầu, giúp tăng tốc độ truy cập và giảm độ trễ khi người dùng truy cập từ nhiều khu vực địa lý khác nhau.
- **Công cụ chuyển đổi dữ liệu:** Cloudinary cung cấp các công cụ hỗ trợ di chuyển ảnh giữa các dịch vụ, hệ thống hoặc môi trường lưu trữ khác nhau, đảm bảo tính liên tục và dễ bảo trì.

3.6 Tổng kết

Trong chương này, tôi đã trình bày các công nghệ được sử dụng để xây dựng hệ thống marketplace du lịch, phù hợp với yêu cầu đã phân tích ở Chương 2. Các công nghệ lựa chọn đảm bảo hiệu suất, bảo mật và khả năng mở rộng.

Backend sử dụng Spring Boot để phát triển API linh hoạt. Giao diện được xây dựng bằng ReactJS kết hợp với Vite, Tailwind CSS và DaisyUI nhằm tối ưu trải nghiệm người dùng. Dữ liệu được lưu trữ trên Supabase PostgreSQL, trong khi Cloudinary hỗ trợ quản lý hình ảnh.

Những công nghệ này có tính ứng dụng cao, dễ bảo trì, và phù hợp với phạm vi triển khai của đề án cá nhân.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

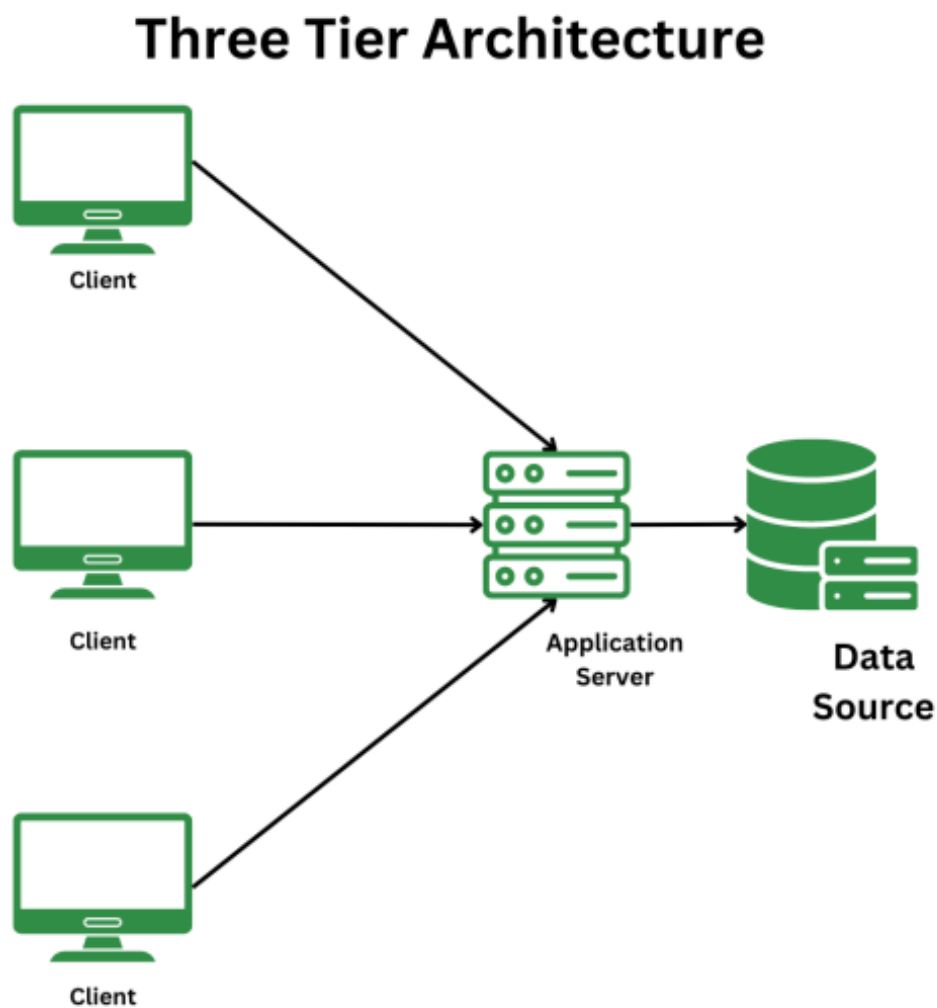
Đối với ứng dụng của mình, tôi đã chọn **kiến trúc ba lớp (Three-tier architecture)** để tổ chức và triển khai các chức năng. Kiến trúc này chia ứng dụng thành ba lớp độc lập, mỗi lớp đảm nhận một vai trò riêng biệt: tầng trình bày (**Presentation Tier**), tầng nghiệp vụ (**Application Tier**) và tầng dữ liệu (**Data Tier**). Đây là một kiến trúc phổ biến và dễ dàng mở rộng, bảo trì.

- **Tầng trình bày (Presentation Tier):** Tầng này đảm nhận việc giao tiếp với người dùng cuối. Các thành phần trong tầng này bao gồm giao diện người dùng (UI) như các trang web hoặc ứng dụng di động, nơi người dùng sẽ thực hiện các thao tác và nhận lại kết quả từ hệ thống. Trong hệ thống của tôi, tầng này được triển khai bằng **ReactJS** kết hợp với HTML, CSS và JavaScript để tạo ra giao diện người dùng dễ sử dụng.
- **Tầng nghiệp vụ (Application Tier):** Đây là lớp xử lý các logic nghiệp vụ của ứng dụng, nơi các yêu cầu từ tầng trình bày được xử lý và chuyển tới tầng dữ liệu khi cần thiết. Trong ứng dụng của tôi, tầng nghiệp vụ sẽ chứa các lớp và module xử lý các yêu cầu như tính toán, xác thực và thực hiện các quy trình nghiệp vụ. Các lớp này có thể là các **Service classes** hoặc các **Business Logic classes**, nơi mà logic cốt lõi của ứng dụng được triển khai.
- **Tầng dữ liệu (Data Tier):** Tầng này chịu trách nhiệm lưu trữ và truy xuất dữ liệu từ cơ sở dữ liệu hoặc các nguồn dữ liệu khác. Trong ứng dụng của tôi, tầng này sử dụng các công nghệ cơ sở dữ liệu Postgresql để lưu trữ thông tin về người dùng, sản phẩm, đơn hàng, v.v. Các lớp trong tầng này bao gồm các **Data Access Object (DAO)** hoặc **Repository**, giúp truy vấn, cập nhật và quản lý dữ liệu một cách hiệu quả.

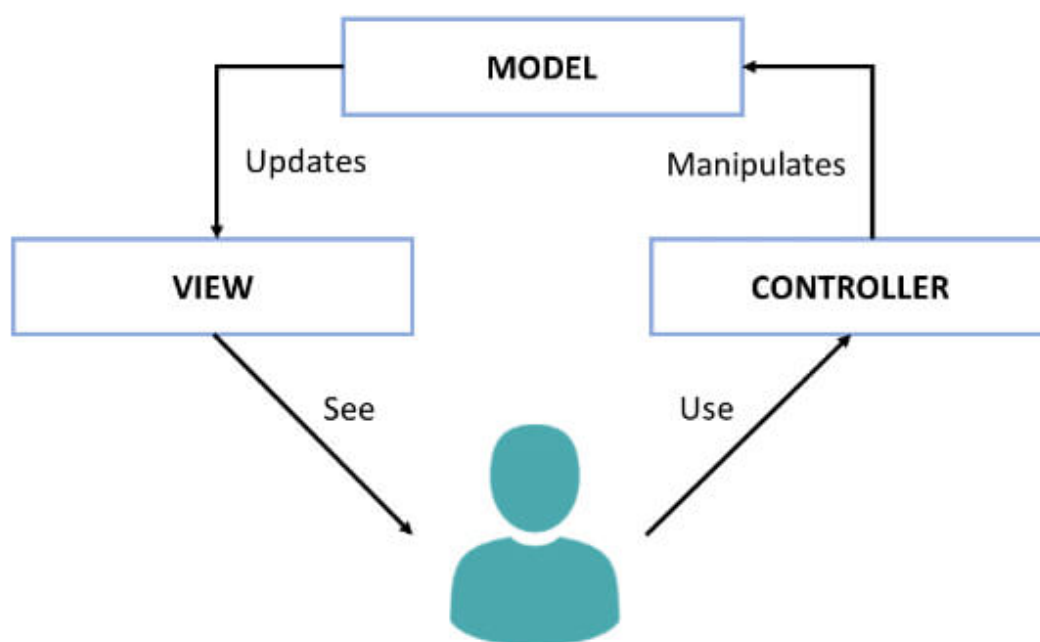
Việc lựa chọn kiến trúc ba lớp giúp tôi dễ dàng tổ chức các thành phần của hệ thống một cách rõ ràng và có thể thay đổi, nâng cấp hoặc bảo trì từng tầng mà không ảnh hưởng tới các tầng khác. Trong ứng dụng của mình, tôi cũng đã áp dụng các nguyên lý của kiến trúc ba lớp vào việc tổ chức mã nguồn, giúp cho quá trình phát triển và bảo trì trở nên dễ dàng hơn.

Mặc dù kiến trúc ba lớp là một mô hình khá cơ bản và phổ biến, tôi cũng đã áp dụng một số cải tiến để phù hợp hơn với yêu cầu của hệ thống. Ví dụ, trong tầng nghiệp vụ, tôi sử dụng một số **design pattern** như **Singleton** hoặc **Factory** để quản

lý các đối tượng và dịch vụ một cách hiệu quả hơn, đồng thời tăng tính mở rộng của ứng dụng.



Hình 4.1: Mô hình kiến trúc 3 tầng

a, Mô hình tầng nghiệp vụ Application Tier: MVC**Hình 4.2:** Mô hình kiến trúc MVC

Mô hình **MVC** (Model-View-Controller) là một mô hình thiết kế phần mềm phổ biến trong phát triển ứng dụng web. Nó tách biệt logic ứng dụng thành ba phần chính: **Model**, **View** và **Controller**, giúp tăng tính tương tác và linh hoạt.

Trong ứng dụng sử dụng **Java Spring Boot**, mô hình MVC được áp dụng thông qua việc tách biệt các thành phần của ứng dụng, mỗi phần đảm nhận một vai trò riêng biệt:

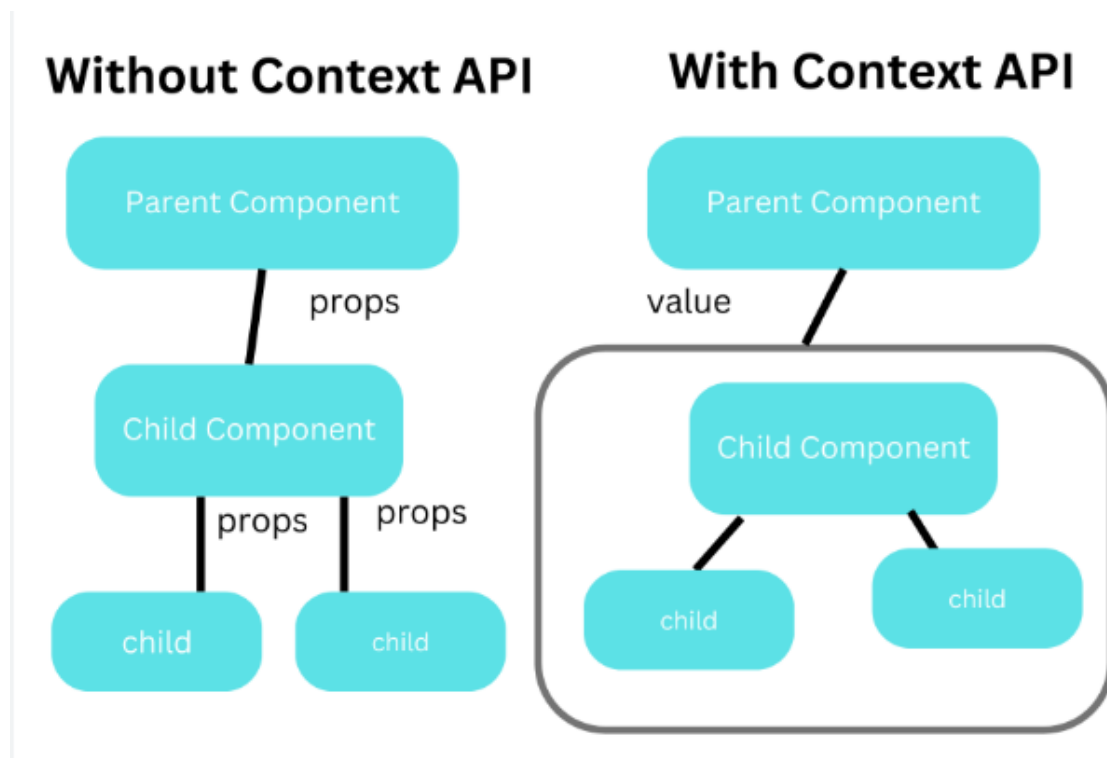
- **Model:** Đại diện cho dữ liệu của ứng dụng. Trong Spring Boot, các lớp Entity trong thư mục `entity` đại diện cho các đối tượng trong cơ sở dữ liệu. Các lớp này sử dụng các thư viện ORM như **JPA** (Java Persistence API) để tương tác với cơ sở dữ liệu và quản lý các model.
- **View:** Là phần giao diện người dùng mà người dùng cuối có thể tương tác. Trong ứng dụng Spring Boot, phần View có thể được xử lý bằng các công nghệ như **Thymeleaf**, **Angular** hoặc **ReactJS**. Tuy nhiên, nếu sử dụng Spring Boot cùng các thư viện frontend, chúng ta có thể kết hợp với ReactJS để hiển thị dữ liệu từ Controller.
- **Controller:** Đảm nhận vai trò điều khiển luồng điều hướng và xử lý yêu cầu từ người dùng. Trong Spring Boot, các lớp Controller nằm trong thư mục `controller` và sử dụng các annotation như `@RestController` hoặc `@Controller` để xử lý các endpoint HTTP. Controller nhận các yêu cầu từ

người dùng, gọi các phương thức trong Service hoặc Repository để truy vấn và xử lý dữ liệu, sau đó trả về kết quả cho View hiển thị.

Áp dụng mô hình MVC trong Spring Boot giúp tách biệt các tầng trong ứng dụng, tăng tính tái sử dụng mã nguồn, giảm độ phức tạp và làm cho việc phát triển và bảo trì ứng dụng dễ dàng hơn. Mô hình MVC trong Spring Boot giúp duy trì sự linh hoạt, dễ dàng mở rộng và quản lý các lớp khác nhau của hệ thống trong quá trình phát triển ứng dụng web.

Phía **Server** sẽ tập trung phát triển Model-Controller, trong khi phần **View** sẽ được xử lý ở phía **Client** (với ReactJS hoặc Thymeleaf).

b, Mô hình tầng trình bày Presentation Tier: React Context API và Hooks



Hình 4.3: So sánh quản lý state khi không sử dụng và sử dụng React Context API

Hình 4.3 so sánh sự khác biệt giữa cách quản lý state trong React khi không sử dụng Context API (bên trái) và khi sử dụng Context API (bên phải). Ở mô hình không có Context API, dữ liệu được truyền qua nhiều cấp component thông qua props, gây khó khăn trong việc bảo trì và mở rộng. Ngược lại, với Context API, các component con có thể truy cập trực tiếp giá trị từ component cha mà không cần truyền qua nhiều tầng, giúp quản lý state hiệu quả và rõ ràng hơn trong các ứng dụng React lớn. Mô hình quản lý state trong ứng dụng được xây dựng dựa trên **React Context API** và **React Hooks**, đây là cách tiếp cận hiện đại và được khuyến

ngiht bởi **React team** để quản lý state trong ứng dụng React.

Mô hình này dựa trên ba khái niệm chính: **Context**, **Provider** và **Consumer**.

- **Context**: Đại diện cho một container chứa state toàn cục của ứng dụng. Nó được tạo ra thông qua `React.createContext()` và có thể chứa bất kỳ giá trị JavaScript nào.
- **Provider**: Là một component cung cấp giá trị cho Context. Nó bao bọc các component con và cung cấp cho chúng quyền truy cập vào state thông qua Context.
- **Consumer**: Là các component con có thể đọc giá trị từ Context thông qua `useContext` Hook hoặc `Context.Consumer` component.

Mô hình này tạo ra một luồng dữ liệu một chiều (one-way data flow) trong ứng dụng React. Khi state cần được cập nhật, các component có thể sử dụng các Hooks như `useState`, `useReducer` để thay đổi state, và các component khác sẽ tự động cập nhật khi state thay đổi.

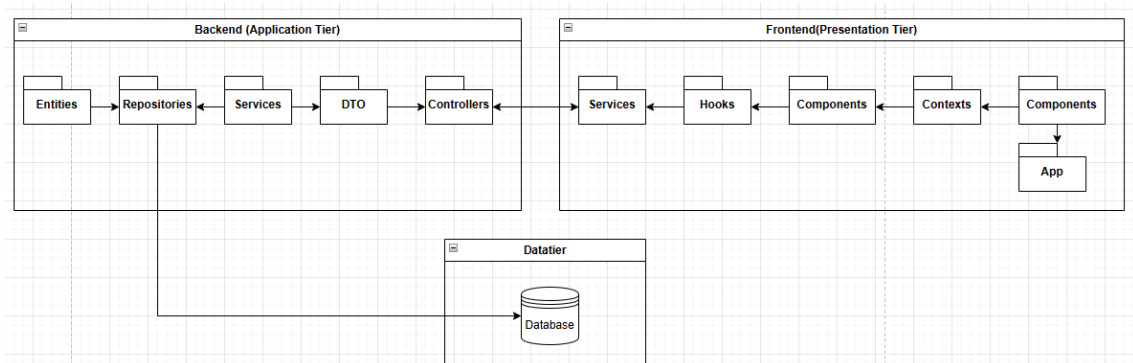
Áp dụng mô hình **Context API** và **Hooks** trong React giúp quản lý state ứng dụng một cách hiệu quả và dễ dàng. Nó giúp tránh **prop drilling** (truyền props qua nhiều cấp component) và tạo điểm truy cập duy nhất vào state toàn cục. Mô hình này cũng tạo điều kiện thuận lợi cho việc tái sử dụng logic và tách biệt concerns trong ứng dụng.

Ngoài ra, ứng dụng của tôi cũng sử dụng **React Router** để xử lý routing giữa các trang và **Axios** để gọi API, giúp việc tương tác với server trở nên dễ dàng và hiệu quả hơn.

c, Mô hình tầng dữ liệu Data Tier: PostgreSQL trên Supabase

Trong mô hình tầng dữ liệu, tôi sử dụng **PostgreSQL trên Supabase** để lưu trữ dữ liệu của ứng dụng. **Supabase** là một nền tảng backend-as-a-service (BaaS) sử dụng **PostgreSQL** làm cơ sở dữ liệu chính, giúp quản lý dữ liệu dễ dàng và an toàn.

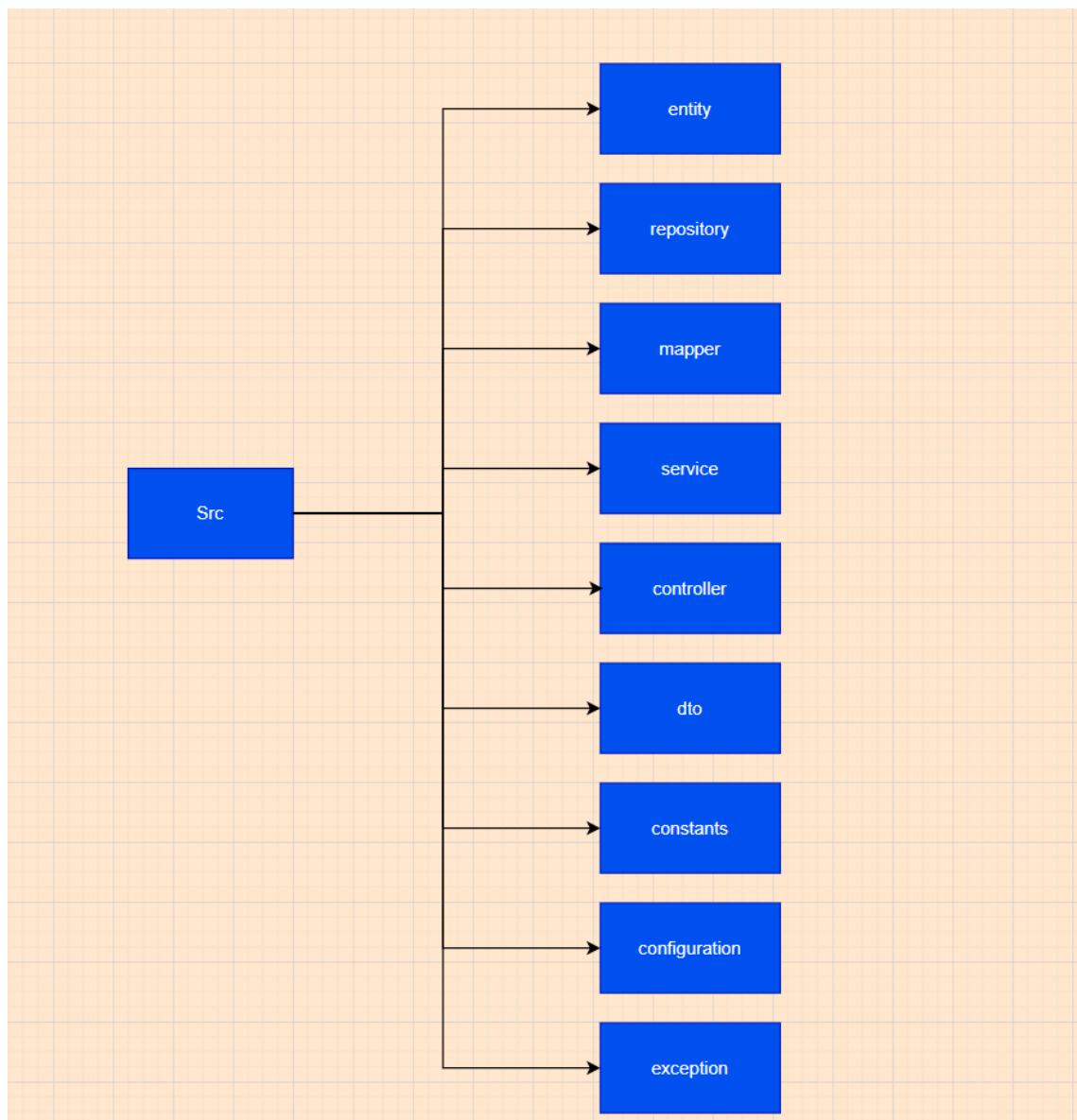
4.1.2 Thiết kế tổng quan



Hình 4.4: Sơ đồ kiến trúc hệ thống

Hình 4.4 mô tả sơ đồ kiến trúc hệ thống dự án của tôi. Dự án được xây dựng dựa trên kiến trúc ba tầng (**Three-tier architecture**), bao gồm các tầng **Frontend** (Presentation Tier), **Backend** (Application Tier) và **Data Tier** (Data Tier). Mỗi tầng đảm nhận một vai trò riêng biệt và kết nối với nhau thông qua các giao thức và API cụ thể.

Backend (Application Tier):



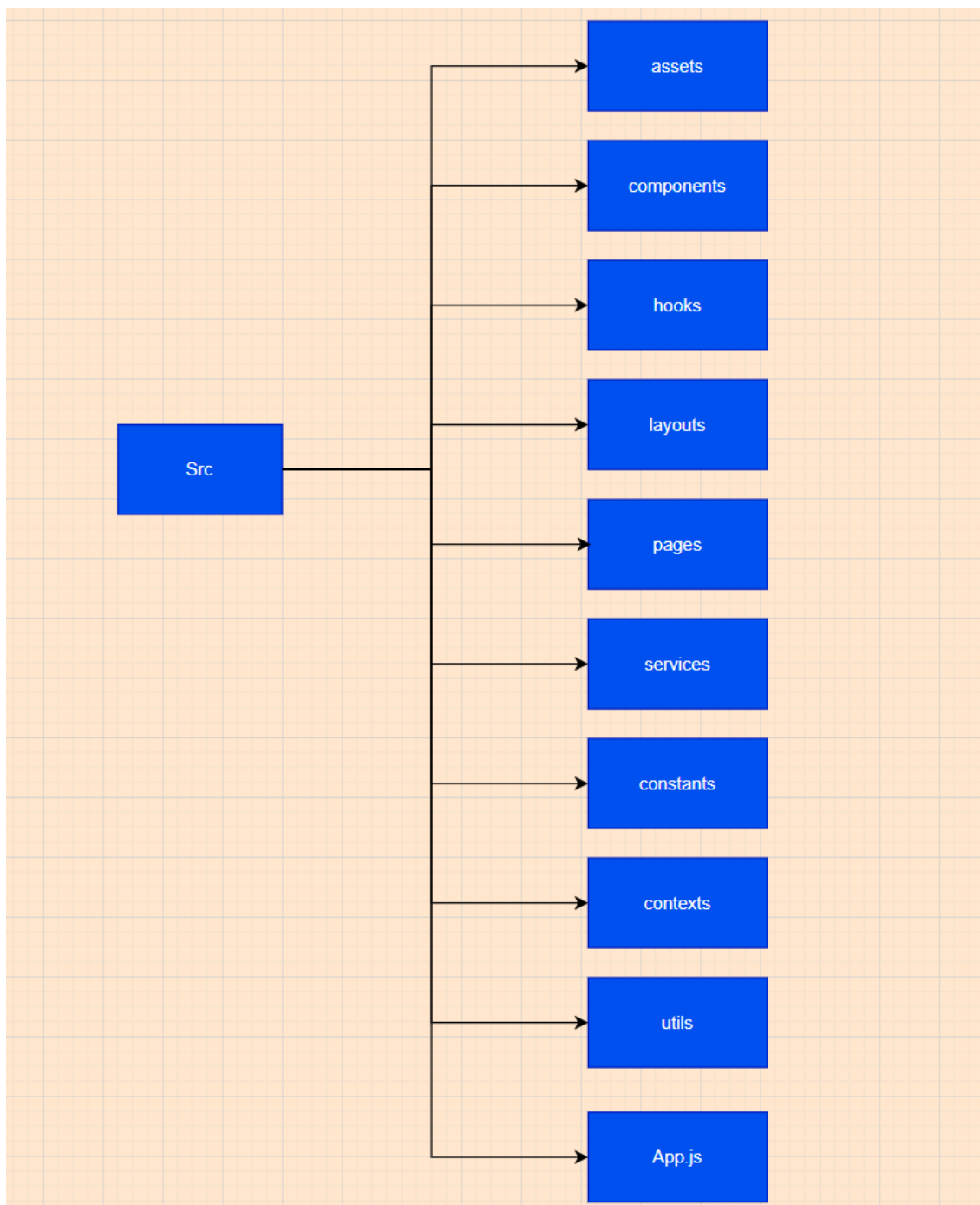
Hình 4.5: Kiến trúc gói Backend tổng quan

Hình 4.5 thể hiện kiến trúc Backend tổng quan của dự án. Dự án sử dụng **Spring Boot** cho phần **Backend**. Các thành phần chính trong **Backend** bao gồm:

- **Controllers:** Tiếp nhận các yêu cầu từ **Frontend**, sau đó gọi đến các phương thức tương ứng trong **Service** để thực hiện các thao tác xử lý nghiệp vụ. Kết quả được trả về cho **Frontend**.
- **Service:** Chứa các logic nghiệp vụ của ứng dụng, tương tác với **Repositories** để truy vấn hoặc cập nhật dữ liệu.
- **Repositories:** Chịu trách nhiệm truy cập và thao tác với cơ sở dữ liệu thông qua **JPA/Hibernate**.
- **Entities:** Định nghĩa cấu trúc dữ liệu, phản ánh cấu trúc của các bảng và mối quan hệ trong cơ sở dữ liệu.

- **DTO (Data Transfer Object)**: Dùng để truyền dữ liệu giữa các tầng, giúp tách biệt **Entity** với dữ liệu trả về cho **Client**.

Frontend (Presentation Tier)



Hình 4.6: Kiến trúc gói frontend tổng quan

Hình 4.6 thể hiện kiến trúc Frontend tổng quan của dự án. Dự án sử dụng **ReactJS** với cấu trúc được tổ chức theo các thành phần chính sau:

- **App**: Là điểm khởi đầu của ứng dụng, chứa cấu hình routing và các provider

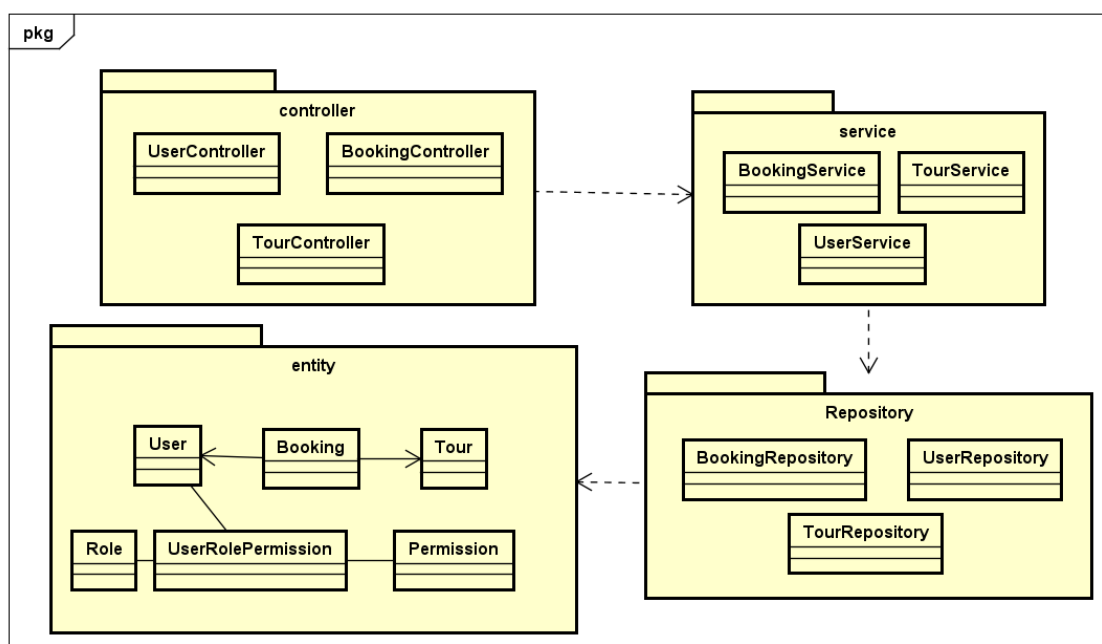
chính.

- **Layouts:** Định nghĩa cấu trúc giao diện chung cho các loại trang khác nhau:
- **Pages:** Chứa các component trang, mỗi trang được bọc trong một layout phù hợp và sử dụng các components để xây dựng giao diện.
- **Components:** Xây dựng giao diện người dùng có thể tái sử dụng, được tổ chức theo:
- **Contexts:** Quản lý state toàn cục cho ứng dụng:
- **Hooks:** Dùng để tái sử dụng logic, quản lý state cục bộ hoặc gọi API:
- **Services:** Xử lý việc gọi API tới Backend và xử lý dữ liệu trả về:
- **API:** Cấu hình và định nghĩa các endpoint.
- **Utils:** Chứa các hàm tiện ích và helpers được sử dụng xuyên suốt ứng dụng.

Data Tier - Database: Sử dụng **PostgreSQL** để lưu trữ dữ liệu cấu trúc của ứng dụng.

4.1.3 Thiết kế chi tiết gói

a, Thiết kế chi tiết gói Backend



Hình 4.7: Chi tiết gói Backend

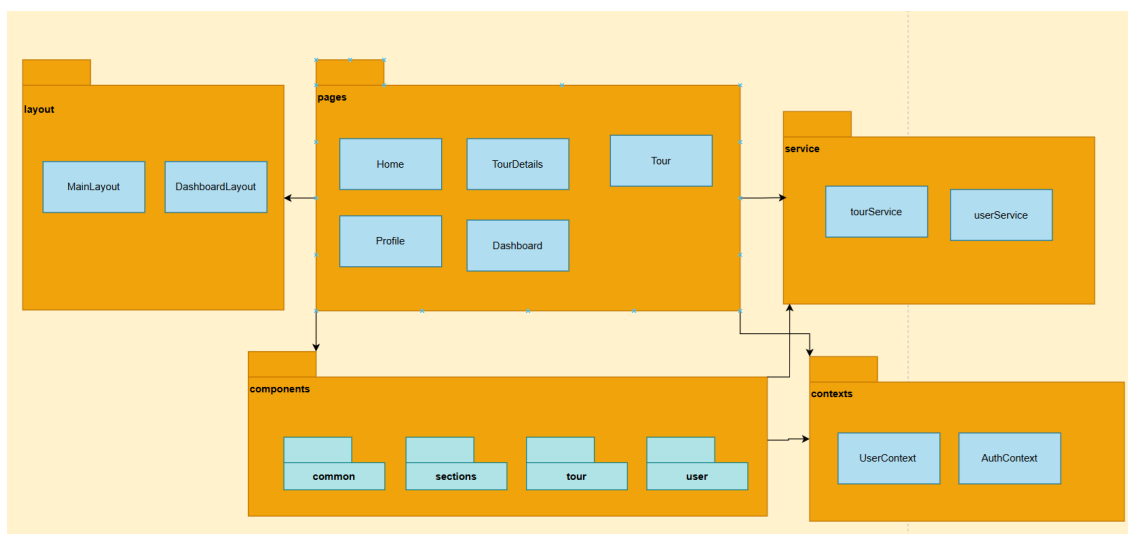
Hình 4.7 thể hiện mối quan hệ giữa các package chính của Backend. Mối quan hệ giữa các package trong gói Backend như sau:

- **Controller** tiếp nhận các yêu cầu từ phía client, sau đó gọi các phương thức

tương ứng trong **Service** để xử lý logic nghiệp vụ.

- **Service** đóng vai trò trung gian, xử lý các logic nghiệp vụ chính của hệ thống. **Service** sẽ gọi tới **Repository** để truy xuất hoặc cập nhật dữ liệu trong cơ sở dữ liệu, đồng thời thao tác với các **Entity** để thực hiện các nghiệp vụ cần thiết.
- **Repository** chịu trách nhiệm truy cập, lưu trữ và truy vấn dữ liệu từ cơ sở dữ liệu, làm việc trực tiếp với các **Entity**.
- **Entity** định nghĩa cấu trúc dữ liệu, ánh xạ các bảng trong cơ sở dữ liệu và thể hiện các mối quan hệ giữa các đối tượng như **User**, **Booking**, **Tour**, **Role**, **Permission**, **UserRolePermission**, v.v.
- Các **Entity** như **User**, **Role**, **Permission** liên kết với nhau thông qua bảng trung gian **UserRolePermission** để quản lý phân quyền. **Booking** liên kết với **User** (khách hàng) và **Tour** để quản lý thông tin đặt chuyến du lịch.
- **Service** và **Repository** đóng vai trò hỗ trợ, cung cấp dữ liệu và logic cho **Controller** cũng như các thành phần khác trong hệ thống.

b, Thiết kế chi tiết gói frontend



Hình 4.8: Chi tiết gói Frontend

Hình 4.8 thể hiện mối quan hệ giữa các package chính trong gói Frontend như sau:

- **Pages** sử dụng **Layout** để định hình giao diện, sử dụng **Components** để xây dựng nội dung, gọi **Service** để lấy/gửi dữ liệu, và sử dụng **Contexts** để truy cập hoặc cập nhật state toàn cục.
- **Components** cũng có thể sử dụng **Service** và **Contexts** nếu cần thiết.

- **Service** và **Contexts** đóng vai trò hỗ trợ, cung cấp dữ liệu và state cho các thành phần khác.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống được thiết kế nhằm tối ưu trải nghiệm người dùng trên các thiết bị máy tính, máy tính bảng, điện thoại. Giao diện đảm bảo:

- **Dễ sử dụng:** Hiển thị đầy đủ thông tin cần thiết, sử dụng biểu tượng (icon) và thanh điều hướng trực quan.
- **Tương tác mượt mà:** Giao diện phản hồi nhanh chóng với các thao tác như đặt chuyến du lịch, xem chi tiết chuyến du lịch, quản lý tài khoản, quản lý booking, v.v.
- **Thích ứng với dữ liệu lớn:** Với danh sách chuyến du lịch hoặc booking dài, hệ thống hỗ trợ phân trang hoặc cuộn vô hạn.

Các trang của hệ thống đều tuân theo bố cục chung gồm 3 phần chính:

1. Header (Thanh tiêu đề):

- Hiển thị logo và tên hệ thống.
- Thanh tìm kiếm chuyến du lịch (nếu có).
- Nút đăng nhập/đăng xuất, hiển thị thông tin người dùng.
- Nút mở menu điều hướng trên thiết bị nhỏ.

2. Footer (Menu điều hướng):

- Chứa các mục như: Trang chủ, Quản lý chuyến du lịch, Quản lý booking, Quản lý người dùng (Admin), Thống kê, Cài đặt.
- Footer sẽ luôn hiển thị ở dưới cùng của trang và có thể cuộn được.

3. Main Content (Nội dung chính):

- Hiển thị nội dung của từng trang như danh sách chuyến du lịch, chi tiết chuyến du lịch, form đặt chuyến du lịch, quản lý booking, bảng quản trị người dùng.
- Đối với trang có nhiều dữ liệu (danh sách chuyến du lịch, booking, user), hỗ trợ tìm kiếm, lọc, sắp xếp.

Màu sắc giao diện

- **Màu chủ đạo:** Xanh dương đậm (#2629aa) – thể hiện sự chuyên nghiệp, tin cậy và hiện đại.

- **Màu hover:** Xanh dương nhạt (#5888f0) – dùng cho hiệu ứng hover trên các nút.
- **Màu chữ chính:** Đen (#000).
- **Màu nền:** Trắng (#fff).
- **Lưu ý:** Các màu thông báo như thành công, cảnh báo, lỗi... không thấy định nghĩa rõ trong CSS, nếu có sử dụng thêm các màu này bạn nên bổ sung vào biến màu.

Thiết kế nút (Button)

- **Nút thông thường:** Sử dụng màu nền xanh dương đậm (#2629aa), chữ trắng, bo góc nhẹ, padding 0.5rem, tối thiểu 100px.
- **Nút viền:** Viền và chữ màu xanh dương đậm, nền trắng.
- **Nút phụ:** Viền và chữ màu xám đậm (#333 hoặc #222), nền trắng.
- **Nút hover:** Đổi sang màu xanh dương nhạt (#5888f0).

Thông điệp phản hồi

- **Thông báo nhanh:** Xuất hiện ở góc dưới bên phải màn hình, tự động ẩn sau 5 giây.
- **Thông báo xác nhận:** Hiện thị dạng modal với tiêu đề, nội dung, nút xác nhận/hủy.

Phông chữ

- **Font mặc định:** Arial, sans-serif – hiện đại, phổ biến, tối ưu cho web.

Giao diện các trang chính

1. Trang đăng nhập:

- **Input:** Email, mật khẩu.
- **Nút:** "Đăng nhập", "Quên mật khẩu".
- **Chế độ lỗi:** Hiện thị thông báo đỏ nếu sai thông tin.

2. Trang chủ:

- Form tìm kiếm chuyến du lịch, lọc chuyến du lịch theo địa điểm, thời gian, giá.
- Hiện thị danh sách chuyến du lịch nổi bật, chuyến du lịch mới nhất.
- Mỗi chuyến du lịch hiện thị thông tin cơ bản: tên, địa điểm, giá, ngày khởi hành, số chỗ còn lại.

3. Trang chi tiết chuyến du lịch:

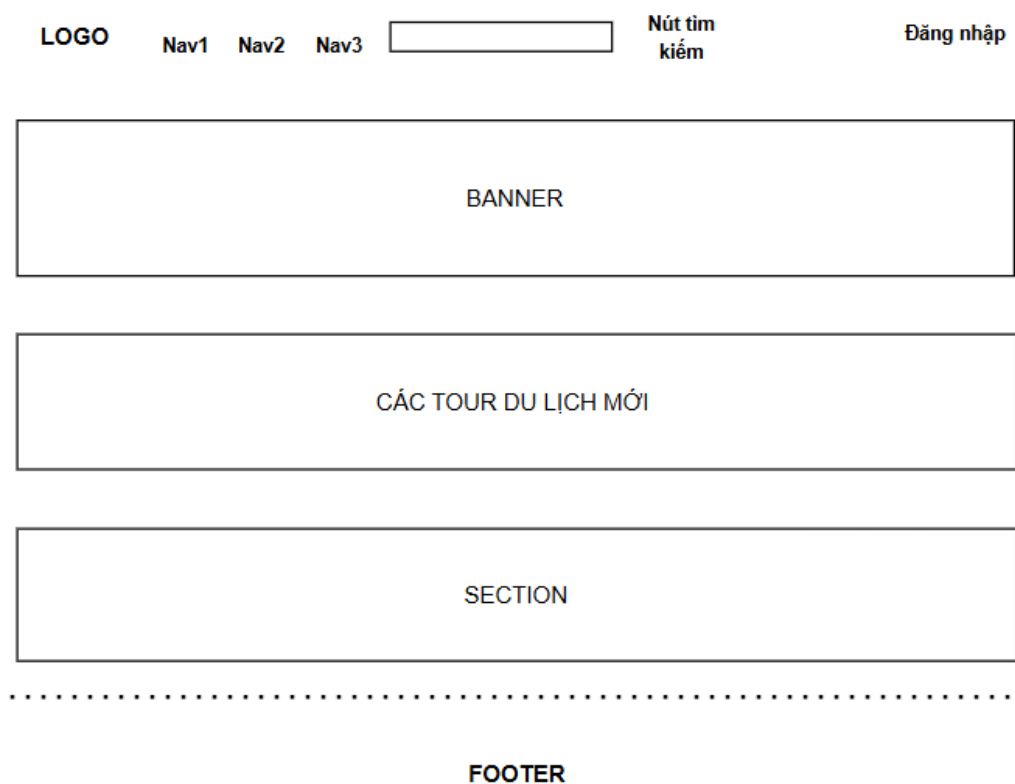
- Hiển thị đầy đủ thông tin về chuyến du lịch: mô tả, lịch trình, hình ảnh, giá, đánh giá, số chỗ còn lại.
- Nút đặt chuyến du lịch, hiển thị form đặt chuyến du lịch khi người dùng đăng nhập.

4. Trang quản lý booking (người dùng):

- Hiển thị danh sách các booking của người dùng.
- Cho phép xem chi tiết, hủy booking.

5. Trang quản trị (Admin Dashboard):

- Quản lý danh sách người dùng, chuyến du lịch, booking.
- Thống kê số lượng chuyến du lịch, booking, người dùng, doanh thu theo ngày/tháng.
- Chức năng duyệt, chỉnh sửa, xóa chuyến du lịch, quản lý quyền người dùng.

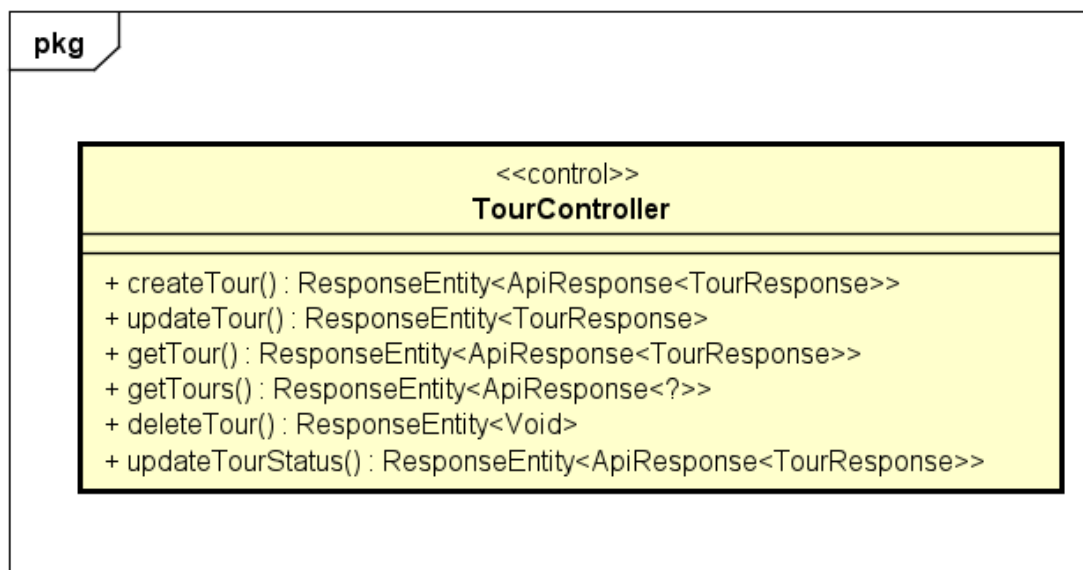


Hình 4.9: Giao diện trang chủ của hệ thống quản lý chuyến du lịch

Hình 4.9 là hình minh họa giao diện trang chủ của hệ thống quản lý chuyến du lịch

4.2.2 Thiết kế lớp

a, Lớp TourController



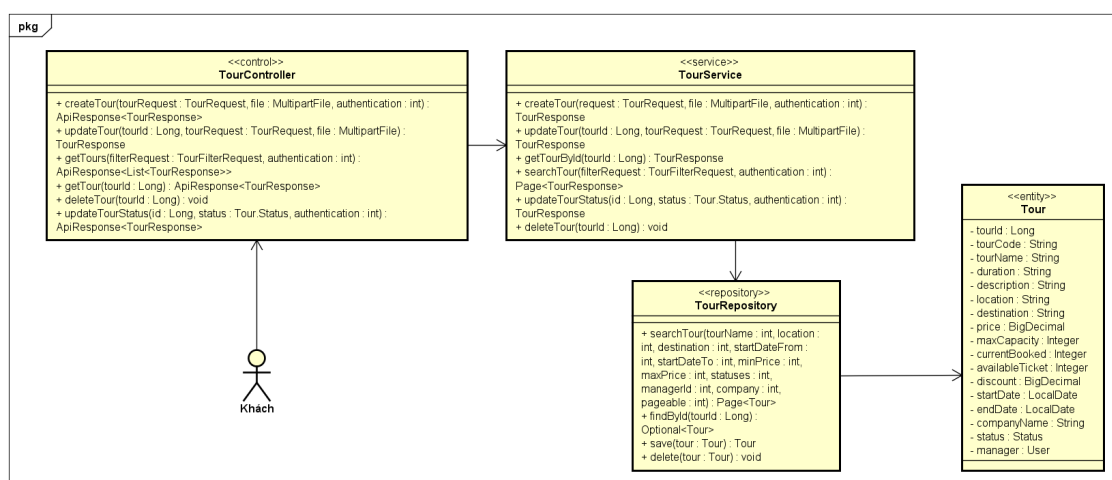
Hình 4.10: Thiết kế lớp TourController

Hình 4.10 là thiết kế lớp ToursController dành cho người dùng. Lớp này có các phương thức và mục đích ý nghĩa cụ thể như sau:

- **createTour():** Phương thức này dùng để tạo một chuyến du lịch mới. Nó nhận vào dữ liệu chuyến du lịch từ người dùng và tệp ảnh cho banner chuyến du lịch, sau đó gọi dịch vụ để xử lý và trả về kết quả dưới dạng TourResponse trong ApiResponse. Phương thức này yêu cầu người dùng có quyền của Admin hoặc Tour Manager.
- **updateTour():** Phương thức này dùng để cập nhật thông tin của một chuyến du lịch đã có. Nó nhận vào dữ liệu chuyến du lịch từ người dùng và tệp ảnh mới cho banner chuyến du lịch nếu có, sau đó gọi dịch vụ để xử lý và trả về kết quả dưới dạng TourResponse trong ApiResponse. Phương thức này yêu cầu người dùng có quyền của Admin hoặc Tour Manager.
- **getTours():** Phương thức này dùng để tìm kiếm và trả về danh sách các chuyến du lịch dựa trên bộ lọc từ phía người dùng (như tên chuyến du lịch, địa điểm, giá cả, ngày bắt đầu, v.v.). Phương thức này trả về kết quả dưới dạng ApiResponse chứa danh sách các chuyến du lịch và thông tin phân trang.
- **getTour():** Phương thức này dùng để lấy chi tiết của một chuyến du lịch theo tourId. Phương thức này sẽ gọi dịch vụ để lấy thông tin chuyến du lịch từ cơ sở dữ liệu và trả về thông tin đó dưới dạng TourResponse trong ApiResponse.

- **deleteTour():** Phương thức này dùng để xóa một chuyến du lịch theo `tourId`. Sau khi xác thực quyền của người dùng, chuyến du lịch sẽ được xóa khỏi cơ sở dữ liệu và trả về mã trạng thái HTTP 204 No Content để chỉ ra rằng chuyến du lịch đã bị xóa thành công.
- **updateTourStatus():** Phương thức này dùng để cập nhật trạng thái của một chuyến du lịch, ví dụ như duyệt hoặc từ chối chuyến du lịch. Phương thức này nhận vào `tourId` và trạng thái mới cho chuyến du lịch, sau đó trả về thông tin của chuyến du lịch đã được cập nhật trạng thái trong `TourResponse`.

b, Thiết kế usecase "Xem danh sách/ chi tiết chuyến du lịch"



Hình 4.11: Thiết kế usecase "Xem danh sách/ chi tiết chuyến du lịch"

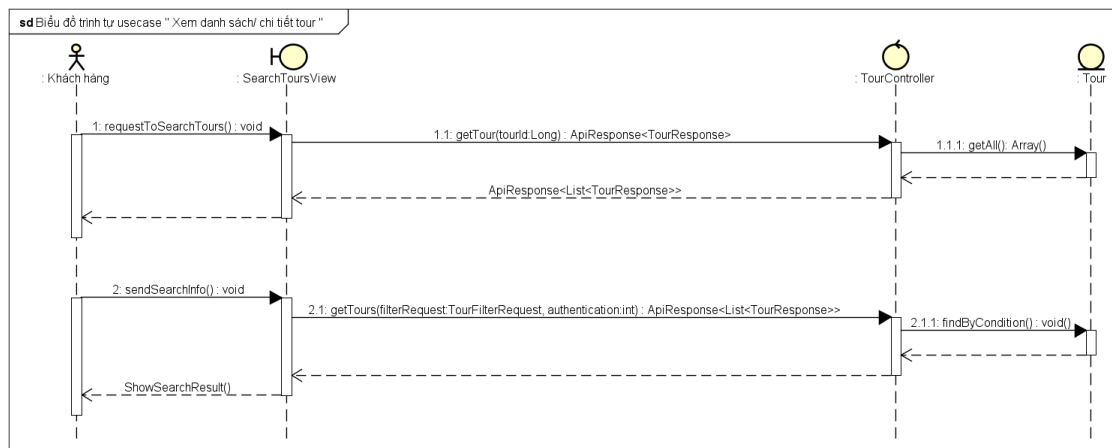
Biểu đồ thiết kế usecase trong Hình 4.11 mô tả kiến trúc tầng lớp (layered architecture) khi người dùng thực hiện thao tác xem danh sách hoặc chi tiết các chuyến du lịch.

- **TourController:** Là lớp tiếp nhận yêu cầu HTTP từ phía người dùng (ví dụ: lấy danh sách chuyến du lịch, xem chi tiết chuyến du lịch, tạo mới, cập nhật, xóa chuyến du lịch...). Lớp này đóng vai trò điều phối, gọi tới các phương thức phù hợp trong `TourService`.
- **TourService:** Thực hiện xử lý nghiệp vụ liên quan đến chuyến du lịch như: xác thực, phân quyền, xử lý logic kinh doanh, và chuyển tiếp yêu cầu truy vấn dữ liệu xuống lớp `TourRepository`.
- **TourRepository:** Là lớp tương tác trực tiếp với cơ sở dữ liệu, thực hiện các truy vấn như tìm kiếm danh sách chuyến du lịch theo điều kiện, tìm chuyến du lịch theo ID, lưu hoặc xóa bản ghi chuyến du lịch.
- **Tour (Entity):** Đại diện cho bảng dữ liệu chuyến du lịch trong hệ thống. Bao gồm các thuộc tính như `tourId`, `tourName`, `location`, `price`, `startDate`, `endDate`,

status, và mối quan hệ với User (manager).

- **Khách:** Là tác nhân chính tương tác với hệ thống. Người dùng cuối có thể gửi yêu cầu xem danh sách hoặc chi tiết chuyến du lịch thông qua các chức năng trên giao diện.

c, Biểu đồ trình tự usecase "Xem danh sách/ chi tiết chuyến du lịch"



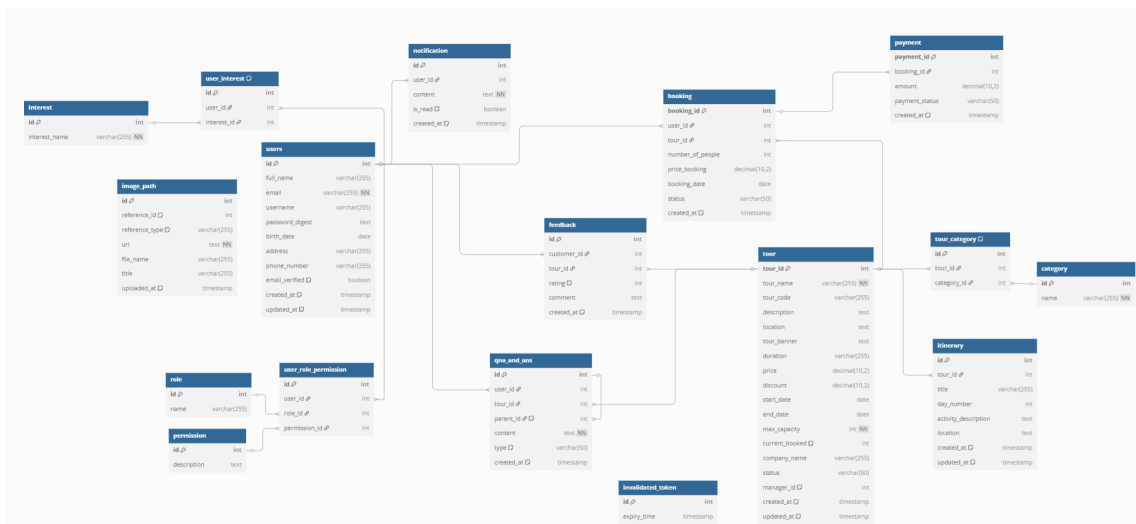
Hình 4.12: Biểu đồ trình tự usecase "Xem danh sách/ chi tiết chuyến du lịch"

Hình 4.12 mô tả trình tự tương tác giữa các đối tượng trong hệ thống khi người dùng thực hiện thao tác xem danh sách hoặc chi tiết chuyến du lịch.

- **Khách hàng:** Là tác nhân khởi tạo yêu cầu tìm kiếm chuyến du lịch thông qua giao diện người dùng.
- **SearchToursView:** Thành phần giao diện trung gian tiếp nhận hành vi người dùng, thực hiện gửi yêu cầu đến TourController.
- **TourController:** Là thành phần thuộc tầng điều khiển (controller), tiếp nhận các yêu cầu từ giao diện, gọi tới các phương thức xử lý bên trong hệ thống như getTour() hoặc getTours().
- **Tour:** Đóng vai trò như lớp thực thể hoặc mô hình dữ liệu, chịu trách nhiệm truy vấn dữ liệu thô hoặc đối tượng cụ thể từ cơ sở dữ liệu (thông qua repository).
- **Diễn biến:**
 - Người dùng yêu cầu hiển thị danh sách chuyến du lịch → requestToSearchTours() được gọi.
 - SearchToursView gửi yêu cầu đến TourController.getTour() để lấy dữ liệu.

- TourController gọi đến phương thức truy xuất dữ liệu từ lớp Tour, thông qua các phương thức như getAll() hoặc findByCondition() để truy xuất chuyến du lịch theo điều kiện lọc.
- Dữ liệu được trả ngược về và hiển thị kết quả bằng phương thức ShowSearchResult().

4.2.3 Thiết kế cơ sở dữ liệu



Hình 4.13: Cơ sở dữ liệu

Hình 4.13 là mô tả chung về cơ sở dữ liệu dự án của tôi. Sau đây là mô tả chi tiết các trường dữ liệu của từng bảng trong cơ sở dữ liệu.

a, Bảng users

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID định danh duy nhất cho mỗi người dùng.
2	full_name	VARCHAR(255)	Không	Không	Họ và tên đầy đủ của người dùng.
3	email	VARCHAR (255)	Có	UNIQUE	Địa chỉ email, dùng để đăng nhập và liên lạc.
4	username	VARCHAR (255)	Có	UNIQUE	Tên đăng nhập duy nhất của người dùng.
5	password_digest	TEXT	Có	Không	Chuỗi mật khẩu đã được mã hóa.
6	birth_date	DATE	Không	Không	Ngày sinh của người dùng.
7	address	VARCHAR (255)	Không	Không	Địa chỉ của người dùng.
8	phone_number	VARCHAR (255)	Không	Không	Số điện thoại của người dùng.
9	email_verified	BOOLEAN	Có	DEFAULT FALSE	Trạng thái xác thực email.
10	created_at	TIMESTAMP	Có	DEFAULT now()	Thời gian tạo tài khoản.
11	updated_at	TIMESTAMP	Có	DEFAULT now()	Thời gian cập nhật thông tin lần cuối.

Bảng 4.1: Mô tả chi tiết bảng users

b, Bảng tour

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	tour_id	SERIAL	Có	Khóa chính	ID định danh duy nhất cho mỗi chuyến du lịch.
2	tour_name	VARCHAR (255)	Có	Không	Tên của chuyến du lịch.
3	tour_code	VARCHAR (255)	Không	UNIQUE	Mã định danh cho chuyến du lịch, dễ nhận biết.
4	description	TEXT	Không	Không	Mô tả chi tiết về nội dung chuyến du lịch.
5	price	DECIMAL (10,2)	Có	Không	Giá gốc của chuyến du lịch.
6	max _capacity	INT	Có	Không	Số lượng khách tối đa của chuyến du lịch.
7	start_date	DATE	Có	Không	Ngày khởi hành của chuyến du lịch.
8	end_date	DATE	Có	Không	Ngày kết thúc của chuyến du lịch.
9	created_at	TIMESTAMP	Có	DEFAULT now()	Thời gian tạo thông tin chuyến du lịch.
10	updated_at	TIMESTAMP	Có	DEFAULT now()	Thời gian cập nhật chuyến du lịch lần cuối.

Bảng 4.2: Mô tả chi tiết bảng tour

c, Bảng booking

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	booking_id	SERIAL	Có	Khóa chính	ID định danh duy nhất cho mỗi đơn đặt chuyến du lịch.
2	user_id	INT	Có	Khóa ngoại (users.id)	ID của người dùng thực hiện đặt chuyến du lịch.
3	tour_id	INT	Có	Khóa ngoại (tour.tour_id)	ID của chuyến du lịch được đặt.
4	number_of_people	INT	Có	Không	Số lượng người tham gia trong đơn đặt.
5	price_booking	DECIMAL (10,2)	Có	Không	Tổng giá trị của đơn đặt tại thời điểm đặt.
6	booking_date	DATE	Có	Không	Ngày thực hiện đặt chuyến du lịch.
7	status	VARCHAR (50)	Có	Không	Trạng thái của đơn đặt (pending, confirmed, cancelled).
8	created_at	TIMESTAMP	Có	DEFAULT now()	Thời gian tạo đơn đặt.

Bảng 4.3: Mô tả chi tiết bảng booking

d, Bảng itinerary

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID định danh duy nhất của mục lịch trình.
2	tour_id	INT	Có	Khóa ngoại (tour.tour_id)	ID của chuyến du lịch chứa lịch trình này.
3	title	VARCHAR (255)	Có	Không	Tiêu đề cho hoạt động trong ngày.
4	day_number	INT	Có	Không	Lịch trình cho ngày thứ mấy trong chuyến du lịch.
5	activity_description	TEXT	Có	Không	Mô tả chi tiết các hoạt động.

Bảng 4.4: Mô tả chi tiết bảng itinerary

e, Bảng feedback

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất cho mỗi phản hồi.
2	customer_id	INT	Có	Khóa ngoại (users.id)	ID của khách hàng gửi phản hồi.
3	tour_id	INT	Có	Khóa ngoại (tour.tour_id)	ID của chuyến du lịch được nhận phản hồi.
4	rating	INT	Có	CHECK (1-5)	Điểm đánh giá (từ 1 đến 5 sao).
5	comment	TEXT	Không	Không	Nội dung bình luận, nhận xét chi tiết.
6	created_at	TIMESTAMP	Có	DEFAULT now()	Thời điểm gửi phản hồi.

Bảng 4.5: Mô tả chi tiết bảng feedback

f, Bảng role

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID định danh duy nhất cho mỗi vai trò.
2	name	VARCHAR (255)	Có	UNIQUE	Tên của vai trò (ví dụ: admin, user, manager).

Bảng 4.6: Mô tả chi tiết bảng role

g, Bảng permission

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID định danh duy nhất cho mỗi quyền hạn.
2	description	TEXT	Không	Không	Mô tả chi tiết về quyền hạn (ví dụ: "Được phép xóa chuyến du lịch").

Bảng 4.7: Mô tả chi tiết bảng permission

h, Bảng user_role_permission

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID của bản ghi phân quyền.
2	user_id	INT	Có	Khóa ngoại (users.id)	ID của người dùng được gán quyền.
3	role_id	INT	Có	Khóa ngoại (role.id)	ID của vai trò được gán.
4	permission_id	INT	Có	Khóa ngoại (permission.id)	ID của quyền hạn được gán.

Bảng 4.8: Mô tả chi tiết bảng user_role_permission

i, Bảng payment

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất của giao dịch thanh toán.
2	booking_id	INT	Có	Khóa ngoại (booking.id)	ID của đơn đặt hàng được thanh toán.
3	amount	DECIMAL (10,2)	Có	Không	Số tiền của giao dịch.
4	payment_status	VARCHAR (50)	Có	Không	Trạng thái thanh toán (succeeded, failed, pending).
5	created_at	TIMESTAMP	Có	DEFAULT now()	Thời điểm tạo giao dịch.

Bảng 4.9: Mô tả chi tiết bảng payment

j, Bảng category

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất của danh mục.
2	name	VARCHAR (255)	Có	UNIQUE	Tên danh mục (ví dụ: Du lịch biển, Du lịch mạo hiểm).

Bảng 4.10: Mô tả chi tiết bảng category

k, Bảng tour_category

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID của bản ghi.
2	tour_id	INT	Có	Khóa ngoại (tour.tour_id)	ID của chuyến du lịch.
3	category_id	INT	Có	Khóa ngoại (category.id)	ID của danh mục.

Bảng 4.11: Mô tả chi tiết bảng tour_category

l, Bảng ques_and_ans

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất của câu hỏi/trả lời.
2	user_id	INT	Có	Khóa ngoại (users.id)	ID người đăng.
3	tour_id	INT	Có	Khóa ngoại (tour.tour_id)	ID chuyến du lịch được hỏi đến.
4	parent_id	INT	Không	Khóa ngoại (qna_and_ans.id)	ID của câu hỏi gốc (nếu đây là một câu trả lời).
5	content	TEXT	Có	Không	Nội dung câu hỏi hoặc câu trả lời.
6	type	VARCHAR (50)	Có	Không	Phân loại là "question" hoặc "answer".
7	created_at	TIMESTAMP	Có	DEFAULT now()	Thời điểm đăng.

Bảng 4.12: Mô tả chi tiết bảng qna_and_ans

m, Bảng notification

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	BIGINT	Có	Khóa chính	ID duy nhất của thông báo.
2	user_id	BIGINT	Có	Khóa ngoại (users.id)	ID người nhận thông báo.
3	content	TEXT	Có	Không	Nội dung của thông báo.
4	created_at	TIMESTAMP	Có	DEFAULT now()	Thời điểm tạo thông báo.
5	is_read	BOOLEAN	Có	DEFAULT FALSE	Trạng thái đã đọc hay chưa.
6	tour_id	BIGINT	Có	Khóa ngoại (tour.tour_id)	ID tour liên quan đến thông báo.

Bảng 4.13: Mô tả chi tiết bảng notification

n, Bảng interest

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất của sở thích.
2	interest_name	VARCHAR (255)	Có	UNIQUE	Tên sở thích (ví dụ: Leo núi, Tắm biển).

Bảng 4.14: Mô tả chi tiết bảng interest

o, Bảng user_interest

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID của bản ghi.
2	user_id	INT	Có	Khóa ngoại (users.id)	ID của người dùng.
3	interest_id	INT	Có	Khóa ngoại (interest.id)	ID của sở thích.

Bảng 4.15: Mô tả chi tiết bảng user_interest

p, Bảng image_path

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID duy nhất của hình ảnh.
2	reference_id	INT	Có	Không	ID của đối tượng được liên kết (tour, user).
3	reference_type	VARCHAR (255)	Có	Không	Tên bảng của đối tượng được liên kết ("tour", "user").
4	url	TEXT	Có	Không	Đường dẫn URL đến hình ảnh.
5	title	VARCHAR (255)	Không	Không	Tiêu đề hoặc mô tả ngắn cho ảnh.
6	uploaded_at	TIMESTAMP	Có	DEFAULT now()	Thời gian tải ảnh lên.

Bảng 4.16: Mô tả chi tiết bảng image_path

q, Bảng invalidated_token

STT	Tên trường	Kiểu dữ liệu	Bắt buộc	Ràng buộc	Mô tả
1	id	SERIAL	Có	Khóa chính	ID của token.
2	expiry_time	TIMESTAMP	Có	Không	Thời gian hết hạn của token đã bị vô hiệu hóa (dùng cho JWT blacklist).

Bảng 4.17: Mô tả chi tiết bảng invalidated_token

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Mục đích	Công cụ/Thư viện	Địa chỉ URL
IDE lập trình	Visual Studio Code 1.85.x	https://code.visualstudio.com/
Quản lý mã nguồn	Git	https://git-scm.com/
Quản lý dự án Java	Maven 3.9.x	https://maven.apache.org/
Ngôn ngữ backend	Java 21	https://www.oracle.com/java/technologies/downloads/
Framework backend	Spring Boot 3.2.x	https://spring.io/projects/spring-boot
ORM backend	Spring Data JPA	https://spring.io/projects/spring-data-jpa
ORM backend	Hibernate	https://hibernate.org/
Thư viện mapping	MapStruct 1.5.5	https://mapstruct.org/
Thư viện annotation	Lombok 1.18.30	https://projectlombok.org/
Thư viện bảo mật	Spring Security	https://spring.io/projects/spring-security
Thư viện gửi mail	Spring Boot Starter Mail	https://spring.io/projects/spring-boot
Thư viện cloud media	Cloudinary 1.33.0	https://cloudinary.com/
Cơ sở dữ liệu	PostgreSQL 15.x	https://www.postgresql.org/
Thư viện frontend	React 19.x	https://react.dev/
Thư viện router FE	React Router DOM 7.5.x	https://reactrouter.com/
Thư viện gọi API FE	Axios 1.9.x	https://axios-http.com/
Thư viện UI FE	Lucide React 0.503.x	https://lucide.dev/
Thư viện form FE	React Hook Form 7.56.x	https://react-hook-form.com/
Thư viện carousel FE	React Responsive Carousel 3.2.x	https://www.npmjs.com/package/react-responsive-carousel
Thư viện toast FE	React Toastify 11.0.x	https://fkhadra.github.io/react-toastify/
Thư viện CSS	TailwindCSS 4.1.x	https://tailwindcss.com/
Công cụ build FE	Vite 6.3.x	https://vitejs.dev/
Công cụ kiểm thử API	Postman 10.x	https://www.postman.com/

Bảng 4.18: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Sau khi hoàn thiện hệ thống, người dùng có thể thực hiện các chức năng sau:

Ứng dụng của tôi là một **Marketplace Chuyến Du Lịch**, nơi người dùng có thể duyệt và tìm kiếm các chuyến du lịch theo nhiều tiêu chí khác nhau như điểm đến, thời gian du lịch, mức giá, loại hình chuyến du lịch (phượt, nghỉ dưỡng, văn hóa, v.v.). Hệ thống sẽ hiển thị danh sách các chuyến du lịch phù hợp với tiêu chí tìm kiếm và cung cấp đầy đủ thông tin chi tiết về từng chuyến du lịch, bao gồm lịch trình, giá vé, hướng dẫn viên, các dịch vụ kèm theo và hình ảnh minh họa.

Khách hàng có thể duyệt và tìm kiếm các chuyến du lịch theo nhiều tiêu chí như điểm đến, thời gian khởi hành, mức giá, loại hình chuyến du lịch (phượt, nghỉ dưỡng, văn hóa, v.v.). Hệ thống sẽ hiển thị danh sách các chuyến du lịch phù hợp và cung cấp đầy đủ thông tin chi tiết như lịch trình, giá vé, hướng dẫn viên, dịch vụ đi kèm và hình ảnh minh họa. Khi đặt chuyến du lịch, khách hàng có thể gửi kèm các yêu cầu đặc biệt dành riêng cho chuyến đi, chẳng hạn như yêu cầu ăn uống riêng, phương tiện đưa đón hoặc các nhu cầu cá nhân khác; những yêu cầu này sẽ được người quản lý chuyến du lịch tiếp nhận và xử lý.

Trong khi đó, người quản lý chuyến du lịch sẽ chịu trách nhiệm tạo mới, cập nhật và quản lý thông tin các chuyến du lịch trên hệ thống, đồng thời tiếp nhận, xử lý các đơn đặt chuyến du lịch cũng như các yêu cầu đặc biệt từ khách hàng.

Ứng dụng cũng hỗ trợ người dùng đánh giá các chuyến du lịch du lịch, cho phép chấm điểm và nhận xét về chất lượng dịch vụ, giá trị chuyến du lịch, sự hài lòng với lịch trình. Các đánh giá này được lưu trữ và hiển thị trên các trang chuyến du lịch, giúp cộng đồng có cái nhìn khách quan khi chọn chuyến du lịch.

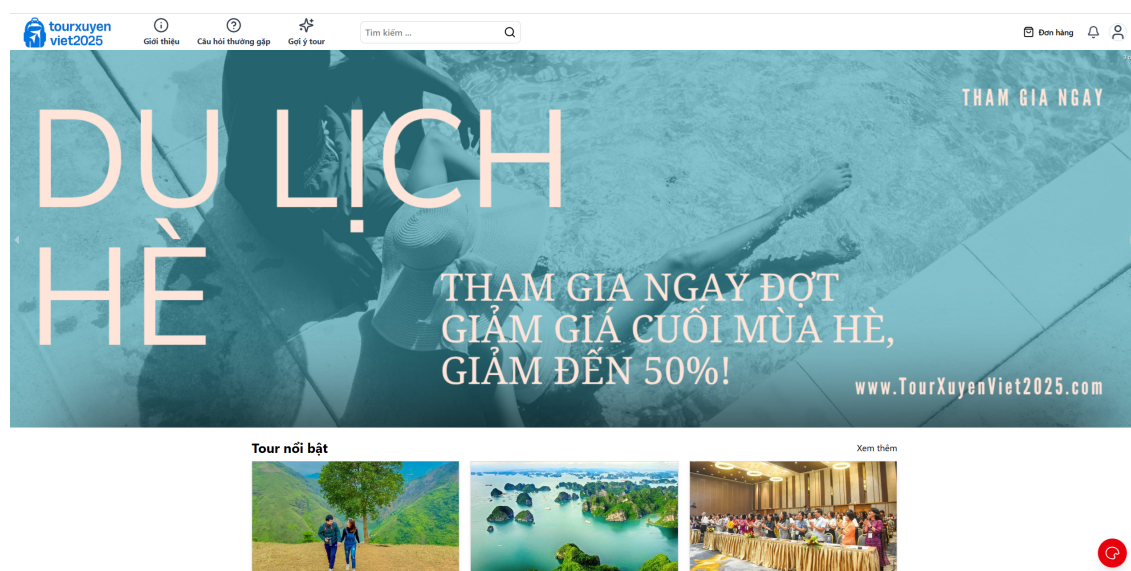
Vai trò quản trị cao nhất thuộc về Admin, người có quyền kiểm soát toàn bộ hệ thống, bao gồm giám sát hoạt động, quản lý người dùng và phân quyền truy cập.

Việc phân tách rõ ràng các chức năng theo từng vai trò giúp hệ thống vận hành hiệu quả, đồng thời đảm bảo tính bảo mật và trải nghiệm người dùng tối ưu.

Số dòng code	14.605 dòng
Số gói	2 gói
Số lớp	112 lớp
Dung lượng toàn bộ mã nguồn	54 MB

Bảng 4.19: Thông tin về ứng dụng

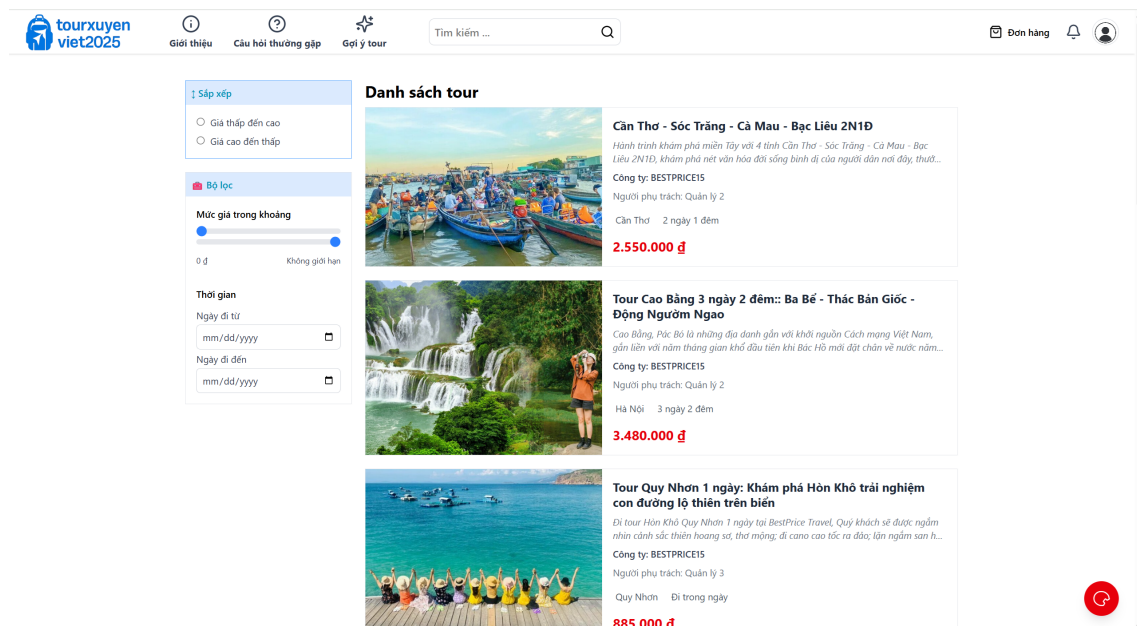
4.3.3 Minh họa các chức năng chính



Hình 4.14: Màn hình trang chủ sau khi đăng nhập vào hệ thống

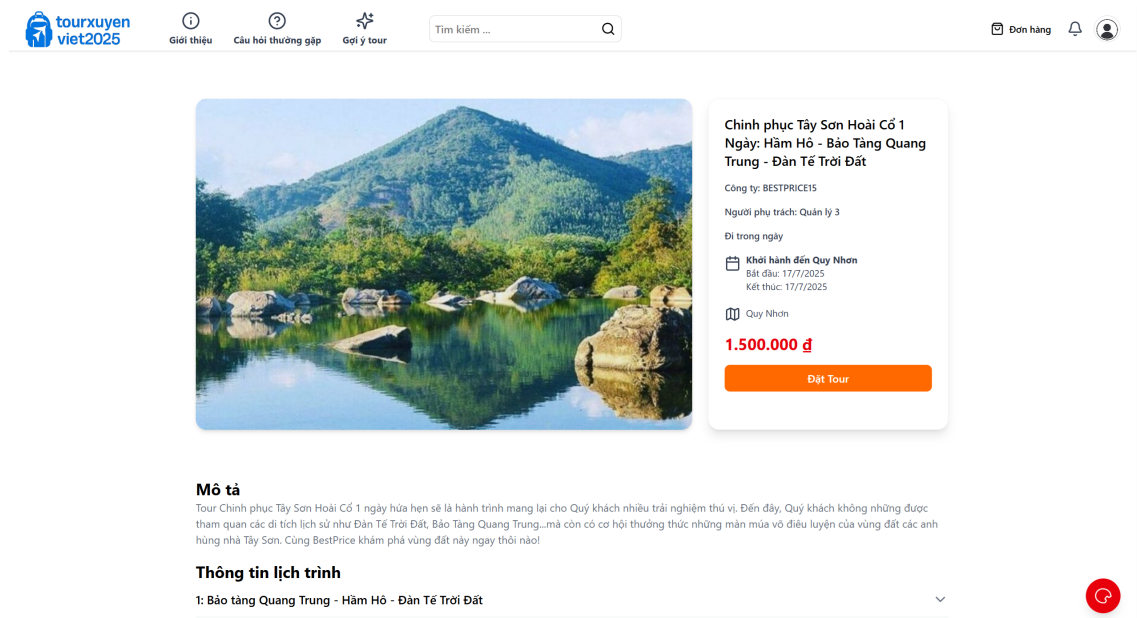
Hình 4.14 mô tả màn hình trang chủ sau khi đăng nhập, thanh header sẽ có các mục để truy cập đầy đủ tính năng của trang web như đơn hàng, quản lý thông tin cá nhân và thông báo liên quan đến chuyến du lịch.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



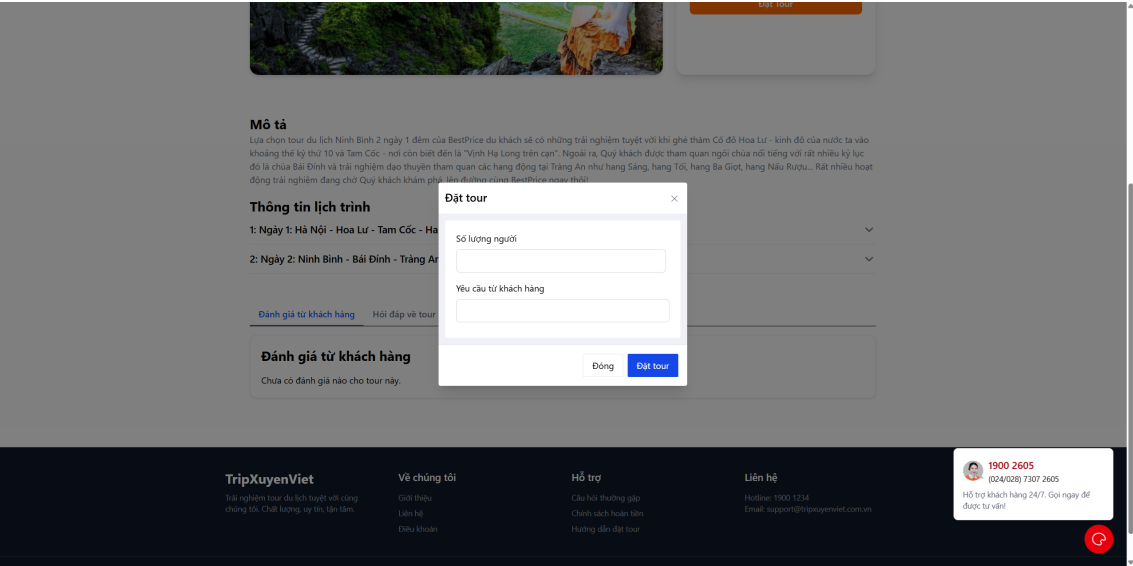
Hình 4.15: Màn hình tìm kiếm chuyến du lịch

Hình 4.15 mô tả chức năng tìm kiếm theo địa điểm du lịch, theo tên chuyến du lịch. Người dùng cũng có thể thực hiện việc sắp xếp theo giá tăng dần, giảm dần, cũng như lọc chuyến du lịch theo giá.



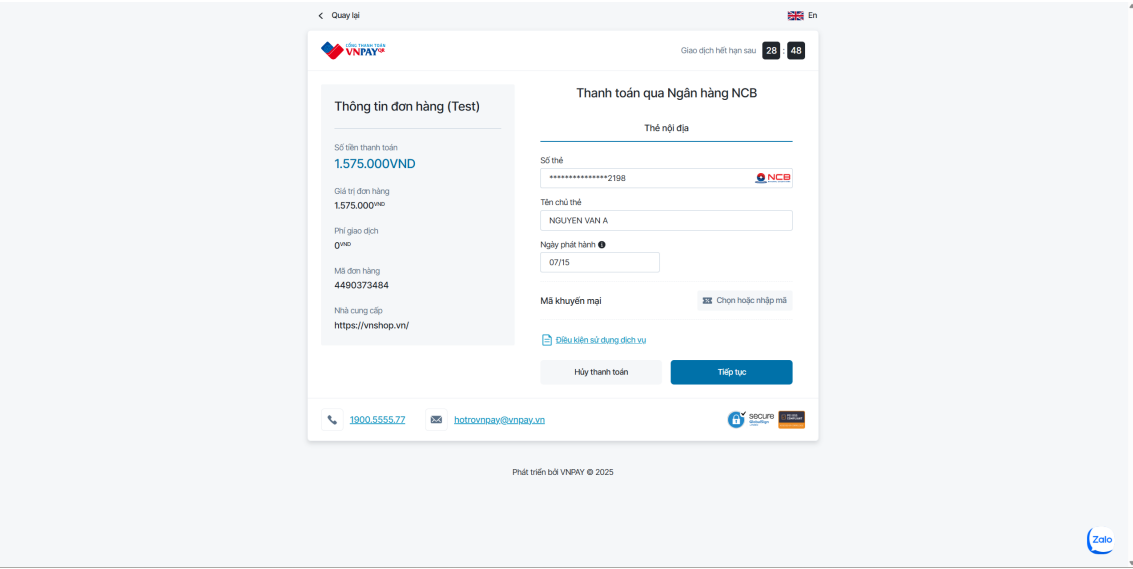
Hình 4.16: Màn hình chi tiết chuyến du lịch

Hình 4.16 mô tả màn hình chi tiết của một chuyến du lịch, nơi người dùng có thể xem thông tin chi tiết về chuyến du lịch, như mô tả, lịch trình, giá và đánh giá từ các khách hàng trước đó.



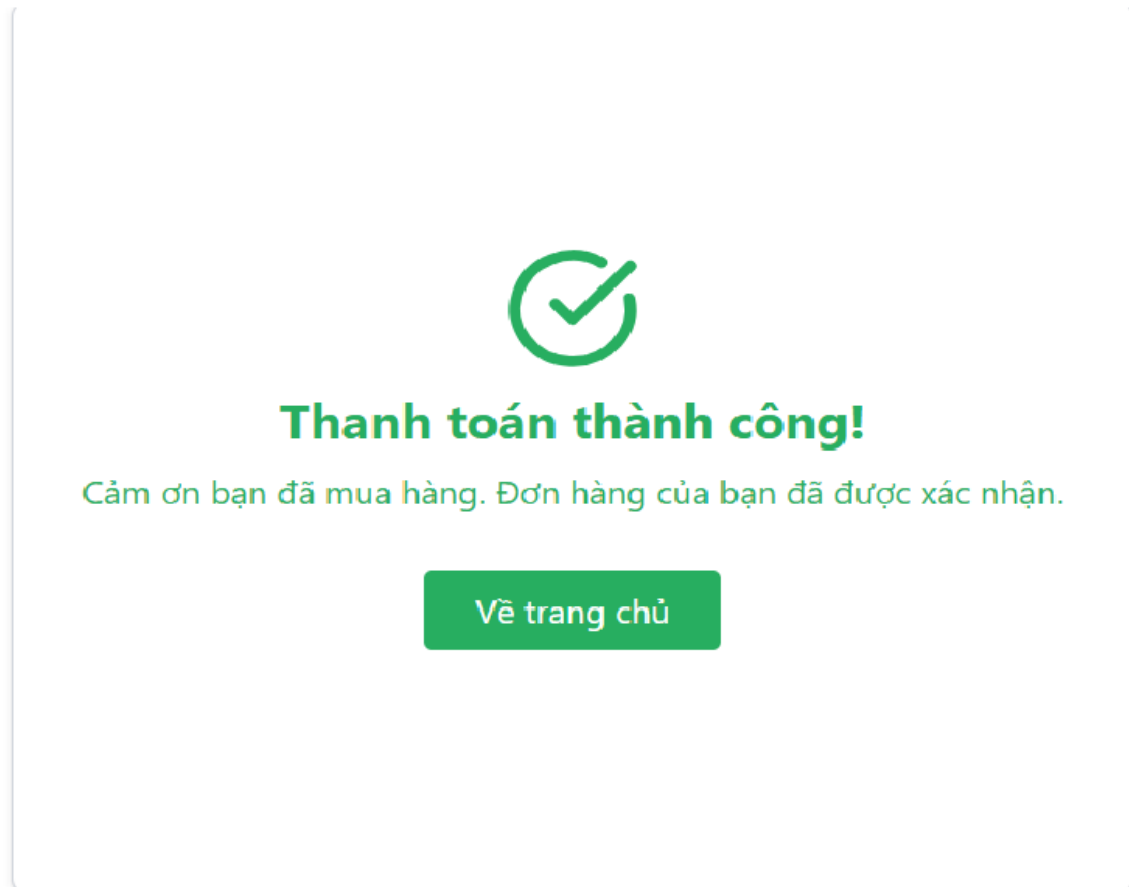
Hình 4.17: Màn hình đặt chuyến du lịch

Hình 4.17 mô tả màn hình đặt chuyến du lịch, nơi người dùng có thể chọn số lượng người tham gia và điền các yêu cầu đặc biệt, sau đó tiến hành đặt chuyến du lịch.



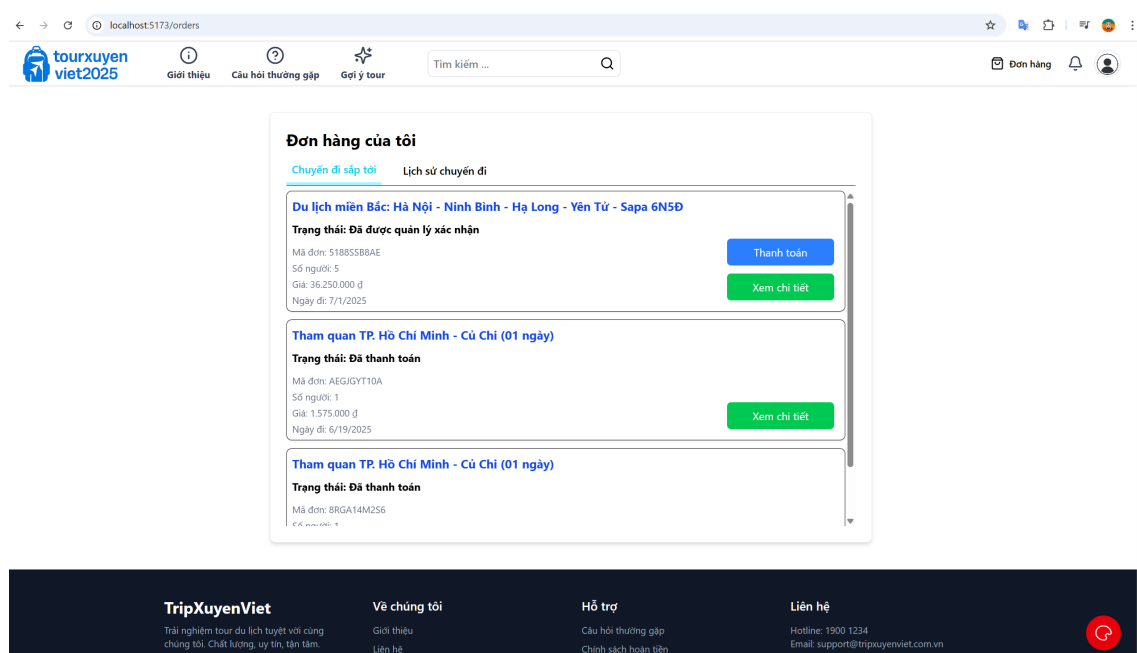
Hình 4.18: Màn hình thực hiện thanh toán

Hình 4.18 mô tả màn hình thực hiện thanh toán, nơi người dùng có thể điền thông tin thẻ thanh toán và hoàn tất quá trình thanh toán cho chuyến du lịch đã chọn.



Hình 4.19: Màn hình thanh toán thành công

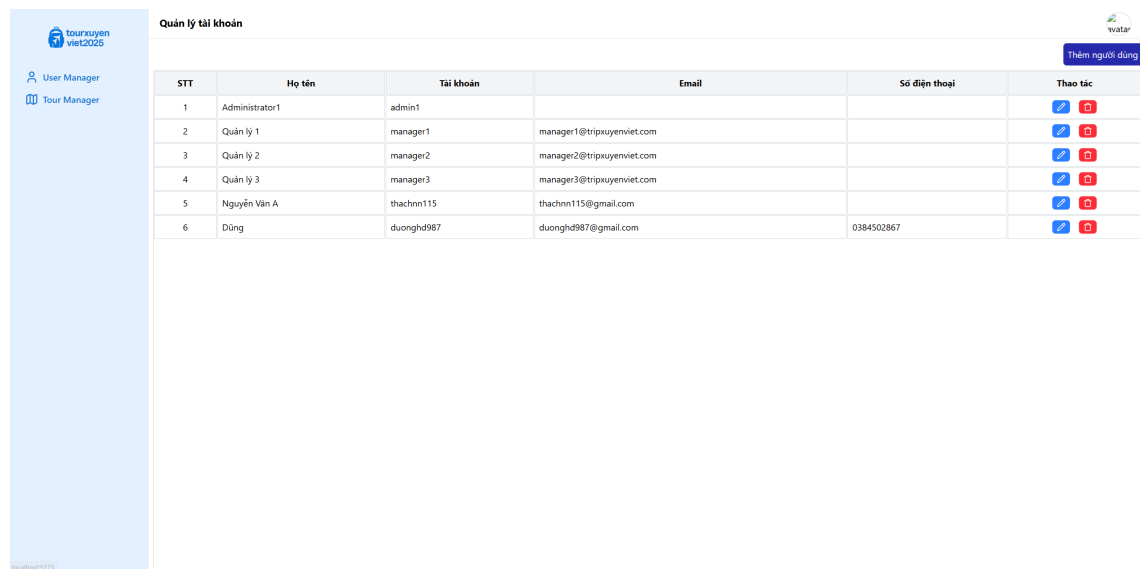
Hình 4.19 mô tả màn hình thông báo thanh toán thành công, xác nhận việc thanh toán đã hoàn tất.



Hình 4.20: Màn hình xem đơn hàng

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

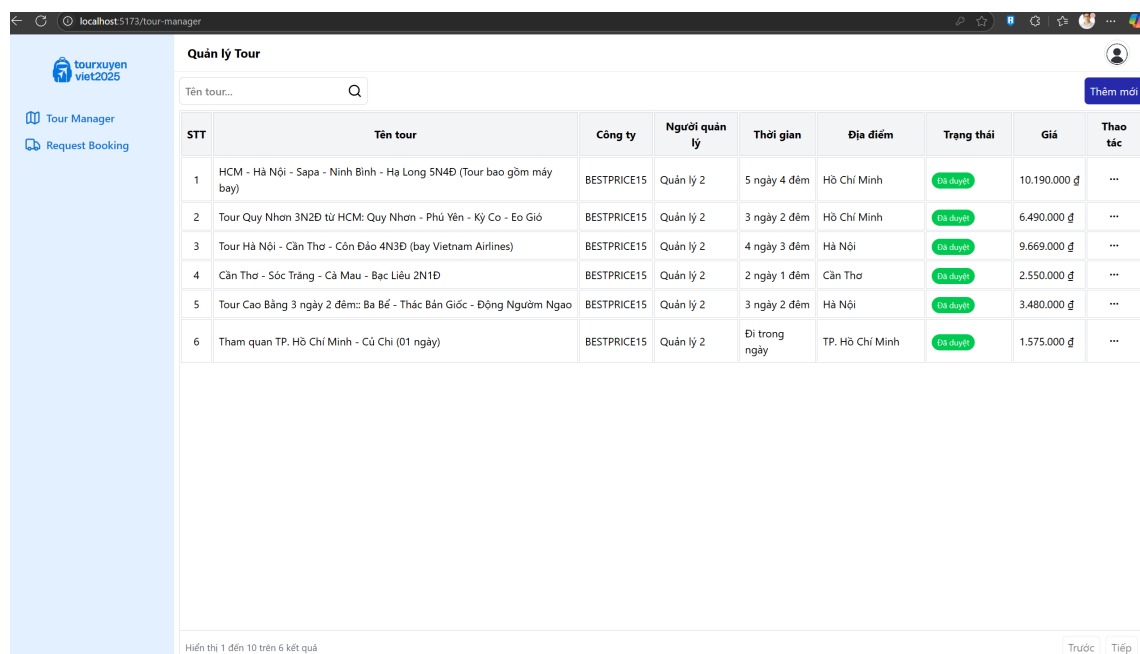
Hình 4.20 mô tả màn hình xem đơn hàng, nơi người dùng có thể theo dõi các thông tin liên quan đến các đơn đặt chuyến du lịch của mình và trạng thái của đơn.



STT	Họ tên	Tài khoản	Email	Số điện thoại	Thao tác
1	Administrator1	admin1			
2	Quản lý 1	manager1	manager1@tripsuyenviet.com		
3	Quản lý 2	manager2	manager2@tripsuyenviet.com		
4	Quản lý 3	manager3	manager3@tripsuyenviet.com		
5	Nguyễn Văn A	thachnn115	thachnn115@gmail.com		
6	Dũng	duonghd987	duonghd987@gmail.com	0384502867	

Hình 4.21: Màn hình quản lý Admin

Hình 4.21 mô tả màn hình quản lý Admin, nơi quản trị viên có thể quản lý các chuyến du lịch và người dùng trong hệ thống.



STT	Tên tour	Công ty	Người quản lý	Thời gian	Địa điểm	Trạng thái	Giá	Thao tác
1	HCM - Hà Nội - Sapa - Ninh Bình - Hạ Long 5N4Đ (Tour bao gồm máy bay)	BESTPRICE15	Quản lý 2	5 ngày 4 đêm	Hồ Chí Minh	Đã duyệt	10.190.000 đ	...
2	Tour Quy Nhơn 3N2Đ từ HCM: Quy Nhơn - Phú Yên - Kỳ Co - Eo Gió	BESTPRICE15	Quản lý 2	3 ngày 2 đêm	Hồ Chí Minh	Đã duyệt	6.490.000 đ	...
3	Tour Hà Nội - Cần Thơ - Côn Đảo 4N3Đ (bay Vietnam Airlines)	BESTPRICE15	Quản lý 2	4 ngày 3 đêm	Hà Nội	Đã duyệt	9.669.000 đ	...
4	Cần Thơ - Sóc Trăng - Cà Mau - Bạc Liêu 2N1Đ	BESTPRICE15	Quản lý 2	2 ngày 1 đêm	Cần Thơ	Đã duyệt	2.550.000 đ	...
5	Tour Cao Bằng 3 ngày 2 đêm: Ba Bể - Thác Bản Giốc - Động Ngườm Ngao	BESTPRICE15	Quản lý 2	3 ngày 2 đêm	Hà Nội	Đã duyệt	3.480.000 đ	...
6	Tham quan TP. Hồ Chí Minh - Củ Chi (01 ngày)	BESTPRICE15	Quản lý 2	Đi trong ngày	TP. Hồ Chí Minh	Đã duyệt	1.575.000 đ	...

Hình 4.22: Màn hình quản lý của người quản lý

Hình 4.22 mô tả màn hình quản lý của người quản lý, nơi họ có thể quản lý các chuyến du lịch và các đơn đặt chuyến du lịch mà họ quản lý.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

The screenshot shows the 'Tạo mới Tour' (Create New Tour) form. The form is divided into several sections:

- Ảnh** (Image): A placeholder for the tour image with the text 'Click để tải ảnh'.
- Thông tin chung** (General Information):
 - Mã Tour (Tour Code): Text input field.
 - Tên Tour (Tour Name): Text input field.
 - Thời lượng tour (Tour Duration): Text input field.
 - Công ty (Company): Text input field.
 - Địa điểm (Location): Text input field.
 - Điểm đến (Destination): Text input field.
 - Mô tả (Description): Text input field.
- Thông tin lịch trình** (Itinerary Information):
 - Giá (VNĐ) (Price (VND)): Text input field.
 - Giảm giá (VNĐ) (Discount (VND)): Text input field.
 - Sức chứa (Capacity): Text input field.
 - Quản lý (Management): Text input field.
 - Quản lý 1 (Management 1): Text input field.
 - Ngày bắt đầu (Start Date): Date picker (mm/dd/yyyy).
 - Ngày kết thúc (End Date): Date picker (mm/dd/yyyy).

The form has 'Đóng' (Close) and 'Tạo' (Create) buttons at the bottom right.

Hình 4.23: Màn hình tạo mới chuyến du lịch

Hình 4.23 mô tả màn hình tạo mới chuyến du lịch, nơi quản trị viên có thể điền thông tin chi tiết về chuyến du lịch mới và lưu vào hệ thống.

The screenshot shows the 'Danh sách lịch trình' (Itinerary List) form. The form displays a table of itineraries with the following columns:

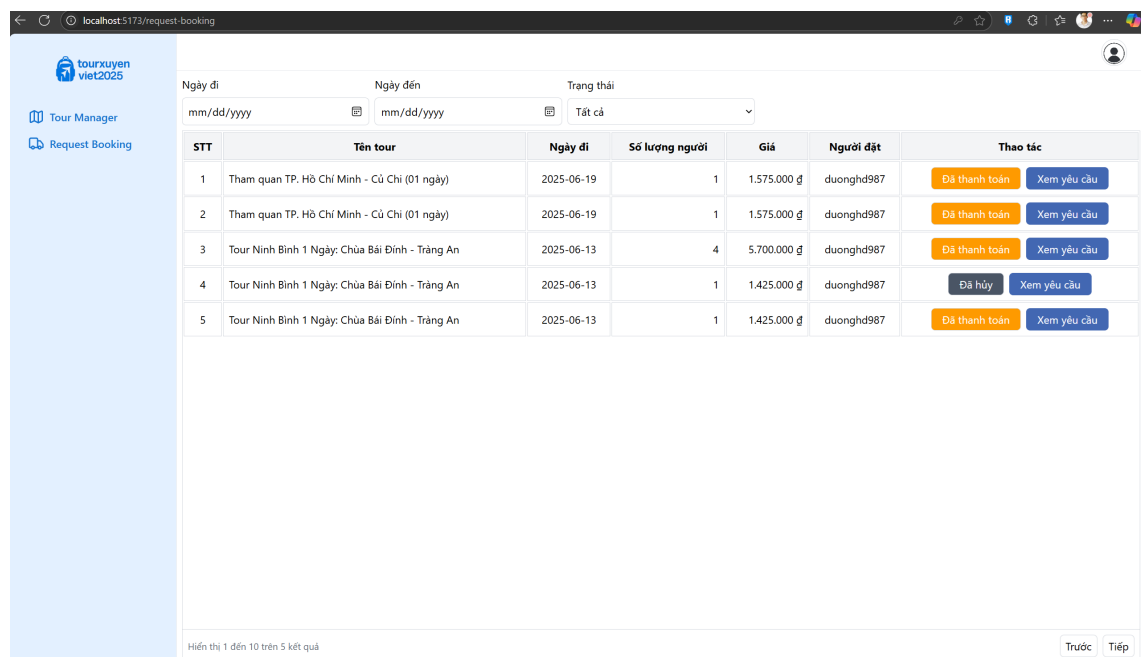
STT	Tên lịch trình	Ngày lịch trình	Địa điểm	Thao tác
1	Hà Nội - Tây Xưa - Mỏm Cá Heo - Cây Cỏ Đơn - Thảo Nguyên Tây Xưa - Đình Gió	1	Hà Nội - Tây Xưa - Mỏm Cá Heo - Cây Cỏ Đơn - Thảo Nguyên Tây Xưa - Đình Gió	...
2	Sông Lũng Khổng Long Hàng Đông - Hồ Suối Chiếu - Hà Nội	2	Sông Lũng Khổng Long Hàng Đông - Hồ Suối Chiếu - Hà Nội	...

The form has 'Thêm lịch trình' (Add Itinerary) button at the top right and 'Trước' (Previous) and 'Tiếp' (Next) buttons at the bottom right.

Hình 4.24: Màn hình cập nhật lịch trình

Hình 4.24 mô tả màn hình cập nhật lịch trình chuyến du lịch, nơi quản trị viên có thể chỉnh sửa lịch trình các chuyến du lịch hiện có.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.25: Màn hình quản lý booking

Hình 4.25 mô tả màn hình quản lý booking, nơi người quản lý chuyến du lịch có thể theo dõi và quản lý các booking chuyến du lịch của người dùng.

4.4 Kiểm thử

Ở bước kiểm thử, em sẽ tiến hành sử dụng kỹ thuật kiểm thử hộp đen để kiểm tra các chức năng trong hệ thống.

4.4.1 Kiểm thử chức năng "Xem danh sách/ chi tiết chuyến du lịch"

Các case cho chức năng "Xem danh sách/ chi tiết chuyến du lịch" được mô tả ở bảng sau đây:

STT	Test Case	Đầu vào	Kết quả mong đợi	Trạng thái
1	Xem danh sách chuyến du lịch	Bấm vào xem thêm ở trang chủ	Hiển thị kết quả các chuyến du lịch hiện có trên trang web	Pass
2	Xem chi tiết chuyến du lịch	Chọn chuyến du lịch trong danh sách	Hiển thị chi tiết chuyến du lịch với lịch trình và giá chuyến du lịch	Pass
3	Sắp xếp chuyến du lịch theo giá	Nhấn vào nút sắp xếp "Giá: Từ thấp đến cao"	Hiển thị các chuyến du lịch theo giá từ thấp đến cao	Pass
4	Xem danh sách chuyến du lịch khi không có chuyến du lịch nào	Không có chuyến du lịch nào trong hệ thống	Hiển thị thông báo "Không có chuyến du lịch nào"	Pass

Bảng 4.20: Bảng kiểm thử chức năng "Xem danh sách/ chi tiết chuyến du lịch"

4.4.2 Kiểm thử chức năng "Tìm kiếm chuyến du lịch"

Các case cho chức năng "Tìm kiếm chuyến du lịch" được mô tả ở bảng sau đây:

STT	Test Case	Đầu vào	Kết quả mong đợi	Trạng thái
1	Tìm kiếm chuyến du lịch theo tên chuyến du lịch	Nhập từ khóa "chuyến du lịch Tết" vào ô tìm kiếm	Hiển thị các chuyến du lịch có tên "chuyến du lịch Tết"	Pass
2	Tìm kiếm chuyến du lịch không có kết quả	Nhập từ khóa "Lễ hội ánh sáng" vào ô tìm kiếm	Hiển thị thông báo không tìm thấy kết quả	Pass
3	Tìm kiếm chuyến du lịch với từ khóa không đầy đủ	Nhập từ khóa "chuyến du lịch T" vào ô tìm kiếm	Hiển thị tất cả các chuyến du lịch có "chuyến du lịch T" trong tên	Pass

Bảng 4.21: Bảng kiểm thử chức năng "Tìm kiếm chuyến du lịch"

4.4.3 Kiểm thử chức năng "Quản lý chuyến du lịch"

Các case cho chức năng "Quản lý chuyến du lịch" được mô tả ở bảng sau đây:

STT	Test Case	Đầu vào	Kết quả mong đợi	Trạng thái
1	Tạo chuyến du lịch mới	Nhập thông tin chuyến du lịch mới và nhấn "Tạo chuyến du lịch"	Hiển thị chuyến du lịch mới trong danh sách chuyến du lịch	Pass
2	Cập nhật thông tin chuyến du lịch	Chỉnh sửa thông tin chuyến du lịch và nhấn "Cập nhật"	Hiển thị thông tin chuyến du lịch đã được cập nhật	Pass
3	Xóa chuyến du lịch	Chọn chuyến du lịch và nhấn "Xóa chuyến du lịch"	chuyến du lịch bị xóa khỏi hệ thống	Pass
4	Duyệt chuyến du lịch	Chọn chuyến du lịch và nhấn "Duyệt chuyến du lịch"	chuyến du lịch được thay đổi trạng thái thành "Approved"	Pass
5	Tạo chuyến du lịch mới thiếu thông tin	Nhập thông tin không đầy đủ và nhấn "Tạo chuyến du lịch"	Hiển thị thông báo "Thông tin không đầy đủ"	Pass

Bảng 4.22: Bảng kiểm thử chức năng "Quản lý chuyến du lịch"

4.4.4 Kiểm thử chức năng "Quản lý người dùng"

Các case cho chức năng "Quản lý người dùng" được mô tả ở bảng sau đây:

STT	Test Case	Đầu vào	Kết quả mong đợi	Trạng thái
1	Thêm người dùng mới	Nhập thông tin người dùng mới và nhấn "Thêm người dùng"	Hiển thị người dùng mới trong danh sách người dùng	Pass
2	Cập nhật thông tin người dùng	Chỉnh sửa thông tin người dùng và nhấn "Cập nhật"	Hiển thị thông tin người dùng đã được cập nhật	Pass
3	Xóa người dùng	Chọn người dùng và nhấn "Xóa người dùng"	Người dùng bị xóa khỏi hệ thống	Pass
4	Thêm người dùng thiếu thông tin	Nhập thông tin không đầy đủ và nhấn "Thêm người dùng"	Hiển thị thông báo "Thông tin không đầy đủ"	Pass

Bảng 4.23: Bảng kiểm thử chức năng "Quản lý người dùng của ADMIN"

4.4.5 Kết luận

Số lượng các trường hợp kiểm thử: 16 trường hợp kiểm thử (trung bình 3 trường hợp cho mỗi chức năng).

Tất cả các chức năng trên đã được kiểm thử với các tình huống đầu vào khác nhau để đảm bảo rằng hệ thống hoạt động chính xác theo yêu cầu và mô tả. Mỗi

test case đã được kiểm tra đầy đủ và kết quả mong đợi đã đạt được trong tất cả các trường hợp kiểm thử.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Xây dựng nền tảng Marketplace Chuyển Du Lịch đa đối tượng

5.1.1 Đặt vấn đề

Trong bối cảnh ngành du lịch tại Việt Nam đang ngày càng phát triển, nhiều công ty lữ hành và tổ chức du lịch đã chuyển mình sang mô hình kinh doanh trực tuyến. Tuy nhiên, phần lớn các nền tảng hiện nay vẫn hoạt động độc lập, mỗi đơn vị cung cấp chuyến du lịch đều xây dựng website riêng biệt, gây khó khăn cho người dùng khi muốn so sánh và lựa chọn các chuyến du lịch phù hợp.

Việc thiếu một hệ thống marketplace tập trung dẫn đến sự phân mảnh trong thông tin, thiếu minh bạch về giá cả và chất lượng dịch vụ, đồng thời gây tốn kém trong việc xây dựng và vận hành hệ thống riêng cho từng đơn vị. Do đó, việc xây dựng một nền tảng marketplace dùng chung – nơi tập hợp các công ty du lịch khắp cả nước để đăng tải và quản lý chuyến du lịch – là một giải pháp có tính chiến lược, mang lại lợi ích cho cả người dùng lẫn nhà cung cấp dịch vụ.

5.1.2 Thực hiện giải pháp

Giải pháp đề xuất là xây dựng một hệ thống ****Marketplace Chuyển Du Lịch**** tập trung, hỗ trợ đa đối tượng thông qua cơ chế phân quyền rõ ràng gồm ba vai trò chính:

- **Admin:** Là người quản trị hệ thống, có toàn quyền kiểm soát các hoạt động trên nền tảng. Admin có thể phê duyệt chuyến du lịch, kiểm duyệt nội dung, quản lý người dùng và xử lý báo cáo, phản hồi.
- **Tour Manager (Người quản lý chuyến du lịch):** Là đại diện từ các công ty du lịch khác nhau trên toàn quốc. Mỗi Tour Manager có quyền đăng ký tài khoản, tạo và quản lý các chuyến du lịch của riêng đơn vị mình, bao gồm thêm/sửa chuyến du lịch, cập nhật lịch trình, theo dõi đơn đặt chuyến du lịch, và phản hồi các yêu cầu đặc biệt từ khách hàng.
- **Khách hàng (User):** Là người dùng cuối của hệ thống. Họ có thể duyệt và tìm kiếm các chuyến du lịch theo điểm đến, mức giá, thời gian,... Ngoài ra, người dùng còn có thể thực hiện các thao tác như đặt chuyến du lịch, thanh toán online, đánh giá chuyến đi, thêm ghi chú cá nhân, và gửi yêu cầu đặc biệt (ăn uống, di chuyển, lịch trình cá nhân hóa) để Tour Manager xem xét trước khi khởi hành.

Cơ chế phân quyền này được xây dựng dựa trên hệ thống xác thực JWT kết hợp với cấu trúc role-based authorization của Spring Security, đảm bảo mỗi người dùng

chỉ được phép thực hiện các chức năng tương ứng với vai trò được gán.

5.1.3 Kết quả đạt được

Mô hình marketplace chuyên du lịch tập trung mang lại nhiều lợi ích nổi bật:

- **Tạo sân chơi chung cho doanh nghiệp:** Giúp các công ty lữ hành, đặc biệt là các đơn vị vừa và nhỏ, dễ dàng tiếp cận khách hàng mà không cần đầu tư hệ thống riêng.
- **Tăng trải nghiệm người dùng:** Người dùng có thể so sánh và lựa chọn chuyến du lịch từ nhiều nhà cung cấp khác nhau ngay trên một nền tảng duy nhất.
- **Tối ưu chi phí vận hành:** Hệ thống dùng chung giúp tiết kiệm chi phí phát triển, bảo trì và vận hành so với việc xây dựng nhiều website độc lập.
- **Thúc đẩy minh bạch và cạnh tranh lành mạnh:** Các chuyến du lịch được hiển thị công khai với đánh giá thực tế từ người dùng, tạo niềm tin và tăng cường chất lượng dịch vụ của các nhà cung cấp.
- **Dễ dàng mở rộng:** Mô hình có thể mở rộng quy mô để tích hợp thêm các đơn vị cung cấp dịch vụ liên quan như khách sạn, vận chuyển, nhà hàng, tạo thành hệ sinh thái du lịch hoàn chỉnh.

5.2 Chức năng thanh toán trực tuyến với Cổng thanh toán VnPay

5.2.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 và xu hướng chuyển đổi số đang diễn ra mạnh mẽ trên toàn cầu, hành vi của người tiêu dùng trong lĩnh vực thương mại điện tử đã có những thay đổi sâu sắc. Đối với ngành du lịch, một trong những lĩnh vực có tốc độ số hóa nhanh nhất, việc cung cấp một trải nghiệm liền mạch và toàn diện cho khách hàng không còn là một lợi thế, mà đã trở thành một yêu cầu bắt buộc. Khách hàng hiện đại không chỉ tìm kiếm thông tin chuyến du lịch trực tuyến mà còn kỳ vọng có thể hoàn tất toàn bộ quy trình đặt dịch vụ—from lựa chọn, tùy chỉnh đến thanh toán—ngay trên nền tảng web hoặc ứng dụng di động một cách nhanh chóng, an toàn và tiện lợi.

Sự thiếu vắng một hệ thống thanh toán trực tuyến tích hợp sẽ tạo ra một rào cản lớn trong hành trình của khách hàng, dẫn đến tỷ lệ từ bỏ giỏ hàng cao, làm giảm hiệu quả kinh doanh và bỏ lỡ các cơ hội doanh thu tiềm năng. Hơn nữa, việc xử lý thanh toán theo phương thức truyền thống còn tiềm ẩn nhiều rủi ro về sai sót, đòi hỏi nhiều nỗ lực trong công tác đối soát và gây khó khăn cho việc quản lý dòng tiền của doanh nghiệp.

Trước thực trạng đó, việc lựa chọn và tích hợp một cổng thanh toán điện tử uy

tín là một quyết định mang tính chiến lược. VnPay, với vị thế là một trong những nhà cung cấp dịch vụ trung gian thanh toán hàng đầu tại Việt Nam, nổi lên như một giải pháp tối ưu. Nền tảng này không chỉ sở hữu một hệ sinh thái đa dạng các phương thức thanh toán—from quét mã VNPAY-QR, thẻ ATM và tài khoản ngân hàng nội địa, đến các loại thẻ thanh toán quốc tế như Visa, MasterCard, JCB—mà còn được cộng đồng và các đối tác lớn tin tưởng nhờ vào các tiêu chuẩn bảo mật nghiêm ngặt.

Do đó, bài toán đặt ra cho dự án là xây dựng và tích hợp thành công chức năng thanh toán trực tuyến thông qua cổng VnPay, nhằm giải quyết các thách thức hiện hữu và đạt được các mục tiêu cốt lõi sau:

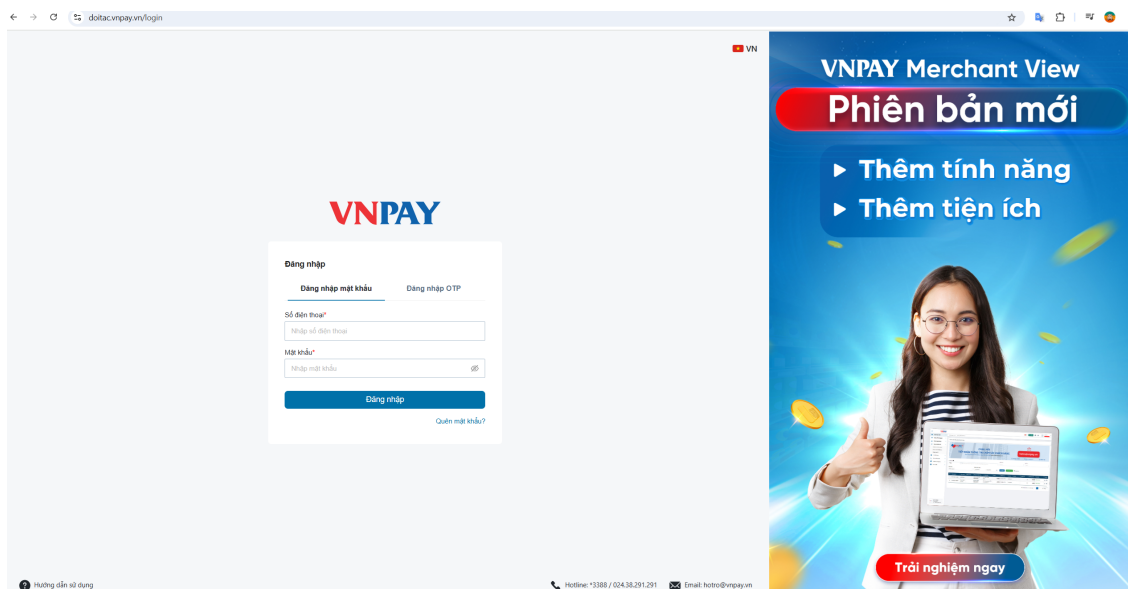
- **Tối ưu hóa trải nghiệm người dùng:** Xây dựng một luồng thanh toán (payment flow) đơn giản, trực quan, giảm thiểu số bước thao tác và giúp người dùng hoàn tất giao dịch một cách dễ dàng.
- **Đảm bảo an toàn và bảo mật tuyệt đối:** Tuân thủ các tiêu chuẩn bảo mật trong ngành tài chính, mã hóa dữ liệu trên đường truyền và không lưu trữ các thông tin nhạy cảm của người dùng (như số thẻ) trên hệ thống của website.
- **Linh hoạt và đa dạng hóa phương thức thanh toán:** Đáp ứng nhu cầu thanh toán đa dạng của mọi đối tượng khách hàng, từ người dùng trong nước đến du khách quốc tế.

Việc giải quyết thành công bài toán này không chỉ là một nâng cấp về mặt công nghệ mà còn là một bước tiến quan trọng trong việc hoàn thiện mô hình kinh doanh, nâng cao năng lực cạnh tranh và khẳng định vị thế chuyên nghiệp của hệ thống trên thị trường du lịch số.

5.2.2 Thực hiện giải pháp

Việc tích hợp chức năng thanh toán được thực hiện thông qua môi trường kiểm thử (Sandbox) do VnPay cung cấp, cho phép xây dựng và kiểm tra luồng hoạt động một cách an toàn trước khi triển khai thực tế.

Bước 1: Cấu hình môi trường kiểm thử



Hình 5.1: Giao diện đăng ký đối tác VnPay

Hình 5.1 là là giao diện để đăng ký đối tác VN. Để bắt đầu, dự án sử dụng tài khoản kiểm thử (Merchant Test) được cung cấp trên trang đối tác của VnPay. Từ tài khoản này, hệ thống nhận được các thông tin cấu hình cốt lõi, bao gồm:

- **TmnCode:** Mã website kiểm thử.
- **HashSecret Key:** Khóa bí mật dùng để tạo và xác thực chữ ký số trong môi trường test.
- **URL API:** Các đường dẫn API dành riêng cho môi trường Sandbox.

Bước 2: Xây dựng luồng thanh toán

Luồng hoạt động chính của chức năng được thiết kế qua 3 bước chính:

1. **Tạo yêu cầu thanh toán:** Khi người dùng chọn thanh toán, máy chủ (backend) sẽ thu thập thông tin đơn hàng và tập hợp các tham số theo yêu cầu của tài liệu API VnPay. Bước quan trọng nhất là dùng HashSecret để tạo chữ ký số `vnp_SecureHash` cho dữ liệu, đảm bảo tính toàn vẹn và chống giả mạo.
2. **Điều hướng đến Cổng thanh toán:** Hệ thống tạo một URL thanh toán hoàn chỉnh (đã bao gồm chữ ký số) và điều hướng trình duyệt của người dùng đến cổng thanh toán Sandbox của VnPay. Tại đây, người dùng có thể sử dụng các thông tin thẻ/tài khoản test để thực hiện giao dịch giả lập.
3. **Xử lý kết quả trả về:** Sau khi giao dịch hoàn tất, VnPay thông báo kết quả qua một yêu cầu IPN (Instant Payment Notification) tới máy chủ. Hệ thống sẽ thực hiện hai nhiệm vụ chính:
 - ****Xác thực lại chữ ký số**** đi kèm trong yêu cầu IPN để đảm bảo thông

tin là hợp lệ và đến từ VnPay.

- Dựa vào mã phản hồi 'vnp_ResponseCode' ('00' là thành công), hệ thống cập nhật trạng thái đơn hàng trong cơ sở dữ liệu (ví dụ: "Paid") và hiển thị thông báo tương ứng cho người dùng.

5.2.3 Kết quả đạt được

Việc tích hợp hệ thống thanh toán VnPay vào nền tảng mang lại một số lợi ích đáng kể như:

- Đảm bảo đáp ứng nhanh chóng và thuận tiện nhu cầu thanh toán của người dùng.
- Tăng cường trải nghiệm của người dùng thông qua việc cung cấp một hệ thống thanh toán bảo mật và đa dạng phương thức.
- Nâng cao tính tin cậy và chuyên nghiệp của hệ thống, từ đó cải thiện khả năng cạnh tranh trên thị trường.
- Giúp doanh nghiệp tối ưu hóa quy trình thanh toán, đồng thời giảm thiểu các rủi ro liên quan đến giao dịch tiền mặt.

5.3 Xây dựng và Tích hợp Module Xác thực Phân quyền bằng JWT

5.3.1 Phân tích bài toán và bối cảnh ứng dụng

Trong bối cảnh kiến trúc phần mềm hiện đại, đặc biệt là với các hệ thống được xây dựng theo mô hình Single Page Application (SPA) với frontend (ReactJS) và backend (Spring Boot) tách biệt, việc thiết lập một cơ chế xác thực và phân quyền hiệu quả, an toàn là yêu cầu nền tảng. Các phương pháp xác thực truyền thống dựa trên Session-Cookie, vốn mang bản chất "stateful" (lưu trạng thái phiên của người dùng trên server), bộc lộ nhiều hạn chế khi áp dụng vào kiến trúc này. Cụ thể, việc quản lý session gây khó khăn cho quá trình cân bằng tải (load balancing) và làm tăng độ phức tạp khi hệ thống cần mở rộng hoặc chuyển đổi sang kiến trúc microservices.

Do đó, bài toán đặt ra cho dự án là phải tìm kiếm và triển khai một giải pháp xác thực "stateless" (phi trạng thái), đáp ứng được các tiêu chí khắt khe sau:

- **Bảo mật:** Phải đảm bảo an toàn cho dữ liệu định danh người dùng trên đường truyền và chống lại các hình thức tấn công phổ biến.
- **Linh hoạt và Độc lập:** Cơ chế xác thực không được phụ thuộc vào một nền tảng cụ thể, cho phép frontend và backend phát triển độc lập và dễ dàng tích hợp với các client khác trong tương lai (ví dụ: ứng dụng di động).
- **Hiệu năng cao:** Giảm thiểu độ trễ trong mỗi yêu cầu API do quá trình xác

thực gây ra.

Dựa trên các yêu cầu trên, JSON Web Token (JWT) đã được lựa chọn làm giải pháp kỹ thuật. JWT là một tiêu chuẩn mở (RFC 7519), cho phép tạo ra các token tự chứa (self-contained) thông tin định danh đã được mã hóa và ký số. Điều này giúp server có thể xác thực yêu cầu mà không cần truy vấn cơ sở dữ liệu hay bộ nhớ đệm để tìm kiếm thông tin phiên, hoàn toàn phù hợp với kiến trúc phi trạng thái mà dự án hướng tới.

Hình 5.2 miêu tả ba thành phần chính của một token JWT được phân tách bằng dấu chấm (.), bao gồm: Header, Payload và Signature.



Hình 5.2: Cấu trúc chi tiết của JWT

5.3.2 Thiết kế và triển khai luồng xác thực với JWT

Dự án đã triển khai một quy trình xác thực và quản lý vòng đời token toàn diện, sử dụng Spring Security ở phía backend và ReactJS ở phía frontend.

1. Luồng Đăng nhập và Cấp phát Token Khi người dùng gửi thông tin đăng nhập (username và password) từ giao diện ReactJS, backend Spring Boot sẽ thực hiện các bước sau:

- **Xác thực thông tin:** So khớp username và password được gửi lên với dữ liệu trong cơ sở dữ liệu. Mật khẩu của người dùng được lưu dưới dạng băm bằng thuật toán BCrypt, do đó quá trình so sánh cũng được thực hiện thông qua BCryptPasswordEncoder để đảm bảo an toàn.
- **Sinh Token:** Nếu xác thực thành công, backend sẽ tạo ra một JWT. Token này

có cấu trúc gồm 3 phần (Header, Payload, Signature). Phần Payload chứa các "claims" quan trọng như: sub (chủ thể - username), iat (thời điểm cấp phát), exp (thời điểm hết hạn), và các thông tin tùy chỉnh như vai trò (roles) để phục vụ cho việc phân quyền.

- **Ký và trả về Token:** JWT sau đó được ký bằng một khóa bí mật (secret key) với thuật toán HMAC-SHA512. Token hoàn chỉnh được trả về cho client. Frontend sẽ lưu trữ token này (thường trong localStorage) để sử dụng cho các yêu cầu tiếp theo.

2. Gửi và Xác thực Token trong các Yêu cầu API Với mỗi yêu cầu cần được bảo vệ gửi đến API sau khi đăng nhập, frontend phải đính kèm JWT vào header Authorization theo chuẩn Bearer:

```
Authorization: Bearer <your_jwt_token>
```

Ở phía backend, một bộ lọc xác thực tùy chỉnh (CustomJwtDecoder) sẽ chặn các yêu cầu này và thực hiện quy trình xác thực token:

- Giải mã và kiểm tra chữ ký với secret key để đảm bảo token không bị giả mạo.
- Kiểm tra thời gian hết hạn để chắc chắn token vẫn còn hiệu lực.
- Kiểm tra xem token có nằm trong danh sách bị thu hồi hay không (xem mục quản lý vòng đời token).

Nếu tất cả các bước trên đều hợp lệ, hệ thống sẽ trích xuất thông tin người dùng từ token và cho phép yêu cầu được tiếp tục xử lý.

3. Phân quyền Dựa trên Vai trò (Role-Based Authorization) Sau khi xác thực thành công, hệ thống tiến hành phân quyền. Dựa trên thông tin vai trò (roles) đã được nhúng trong payload của JWT, Spring Security sẽ quyết định xem người dùng có đủ quyền để truy cập vào endpoint được yêu cầu hay không. Việc này được cấu hình một cách tường minh thông qua các annotation như @PreAuthorize("hasRole('ADMIN')") tại các controller, giúp bảo vệ tài nguyên một cách chi tiết và an toàn.

4. Quản lý Vòng đời Token (Logout và Refresh) Một trong những thách thức của JWT là bản chất phi trạng thái của nó. Để giải quyết vấn đề này, dự án đã triển khai:

- **Cơ chế thu hồi Token:** Khi người dùng đăng xuất, token hiện tại không thể bị xóa khỏi client một cách tuyệt đối. Để ngăn chặn việc token cũ bị sử dụng lại

(replay attack), ID của token này sẽ được lưu vào một bảng "danh sách đen" trong cơ sở dữ liệu (InvalidatedToken). Bộ lọc xác thực sẽ luôn kiểm tra token trong danh sách này trước khi chấp nhận.

- **Cơ chế làm mới Token:** Khi access token hết hạn, thay vì bắt người dùng đăng nhập lại, hệ thống có thể cung cấp một cơ chế làm mới. Người dùng gửi yêu cầu đến một endpoint đặc biệt, hệ thống xác thực token cũ (dù đã hết hạn), thu hồi nó bằng cách thêm vào danh sách đen, và cấp phát một cặp token mới.

5. Xử lý Lỗi và Cấu hình Bảo mật Toàn cục Hệ thống được cấu hình để xử lý các trường hợp xác thực thất bại một cách nhất quán. JwtAuthenticationEntryPoint được sử dụng để bắt các AuthenticationException và trả về mã trạng thái 401 Unauthorized cùng một thông điệp lỗi rõ ràng cho client. Ngoài ra, lớp cấu hình Spring Security định nghĩa rõ các URL công khai (public) không cần xác thực (/api/auth/login, /api/tours, etc.) và các URL còn lại bắt buộc phải có JWT hợp lệ.

5.3.3 Đánh giá kết quả và lợi ích

Việc áp dụng thành công kiến trúc xác thực bằng JWT đã mang lại những kết quả tích cực và thiết thực cho hệ thống:

- **Bảo mật toàn diện:** Cơ chế chữ ký số đảm bảo tính toàn vẹn của thông tin định danh, trong khi việc băm mật khẩu bằng BCrypt bảo vệ dữ liệu người dùng ở tầng lưu trữ. Kiến trúc này cũng giúp giảm thiểu nguy cơ tấn công CSRF (Cross-Site Request Forgery) so với phương pháp dựa trên cookie.
- **Tăng cường khả năng tích hợp và tách biệt (Decoupling):** JWT tạo ra một "hợp đồng" xác thực rõ ràng giữa frontend và backend, cho phép hai đội ngũ phát triển song song. Đồng thời, việc tích hợp với các client mới như ứng dụng di động hay các hệ thống của bên thứ ba trở nên đơn giản hơn rất nhiều.
- **Nâng cao trải nghiệm người dùng:** Quá trình xác thực diễn ra nhanh chóng, gần như tức thời với mỗi yêu cầu API, giúp duy trì trạng thái đăng nhập một cách liền mạch ngay cả khi người dùng làm mới trang, mang lại một trải nghiệm mượt mà và không bị gián đoạn.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong khuôn khổ thời gian thực hiện, đồ án tốt nghiệp "Xây dựng website marketplace chuyên du lịch" đã hoàn thành các mục tiêu trọng tâm đã đề ra, tạo ra một sản phẩm phần mềm có tính ứng dụng thực tiễn.

Cụ thể, một hệ thống quản lý chuyên du lịch toàn diện đã được xây dựng, cho phép quản trị viên cấu hình chi tiết thông tin chuyên du lịch, lịch trình, giá cả và hình ảnh một cách trực quan. Đi kèm với đó là module đặt chuyên du lịch được tự động hóa, có khả năng sinh mã đặt chỗ (booking code) duy nhất và theo dõi trạng thái đơn hàng qua các giai đoạn xử lý như đang chờ xử lý (PENDING), đã xác nhận (CONFIRMED), đã thanh toán (PAID) và đã hủy (CANCELLED). Đặc biệt, chức năng thanh toán trực tuyến và hệ thống đánh giá, phản hồi của khách hàng sau chuyến đi cũng đã được tích hợp thành công, góp phần hoàn thiện quy trình kinh doanh khép kín.

Để hiện thực hóa các chức năng trên, đồ án đã áp dụng một ngăn xếp công nghệ hiện đại và phổ biến trong ngành. Phía backend được phát triển bằng Spring Boot, xây dựng theo kiến trúc RESTful API để đảm bảo sự linh hoạt và tách biệt với frontend. Phía frontend sử dụng thư viện ReactJS để tạo ra một giao diện người dùng năng động với các thành phần có khả năng tái sử dụng cao. Về mặt bảo mật, cơ chế xác thực bằng JSON Web Token (JWT) đã được triển khai để bảo vệ các tài nguyên API, kết hợp với mô hình phân quyền chi tiết. Các tài nguyên hình ảnh được quản lý thông qua dịch vụ lưu trữ đám mây Cloudinary, trong khi hệ thống thông báo được xử lý qua email và notification trong ứng dụng.

Về mặt an toàn và hiệu năng, đồ án đã thành công trong việc triển khai một mô hình phân quyền rõ ràng cho các vai trò khác nhau như ADMIN, TOUR_MANAGER và USER, đảm bảo mỗi vai trò chỉ có thể truy cập các tài nguyên được cho phép. Mọi yêu cầu API đều được bảo vệ và kiểm tra quyền truy cập một cách nghiêm ngặt. Các truy vấn cơ sở dữ liệu được tối ưu hóa thông qua việc sử dụng JPA, đồng thời các giao dịch quan trọng như đặt chuyên du lịch và thanh toán đều được xử lý an toàn để đảm bảo tính toàn vẹn dữ liệu.

Bên cạnh những kết quả đạt được, do giới hạn về thời gian và kiến thức, đồ án vẫn còn một số điểm có thể cải thiện trong tương lai. Các thách thức chính bao gồm việc tối ưu hóa hiệu năng hệ thống khi phải xử lý lưu lượng truy cập lớn, cải tiến thuật toán tìm kiếm và lọc chuyên du lịch để cho ra kết quả nhanh và chính

xác hơn, hoàn thiện trải nghiệm người dùng trên các thiết bị di động, cũng như mở rộng và đa dạng hóa các phương thức thanh toán.

6.2 Hướng phát triển

Để tiếp tục nâng cao giá trị và hoàn thiện hệ thống, một số định hướng phát triển trong tương lai đã được vạch ra, tập trung vào việc cải thiện nền tảng hiện tại và bổ sung các tính năng mới.

Trước hết, việc tối ưu hóa hệ thống là ưu tiên hàng đầu. Về mặt hiệu năng, có thể triển khai các cơ chế caching cho các dữ liệu tĩnh hoặc ít thay đổi nhằm giảm tải cho cơ sở dữ liệu, tiếp tục tinh chỉnh các truy vấn phức tạp, và cân nhắc sử dụng các giải pháp bộ nhớ đệm như Redis để quản lý phiên đăng nhập hiệu quả hơn. Song song đó, việc nâng cao trải nghiệm người dùng (UX) cũng rất quan trọng, bao gồm việc hoàn thiện thiết kế đáp ứng (Responsive Design) để tương thích tốt hơn với các thiết bị di động và tinh giản các biểu mẫu đặt chuyến du lịch, giúp quy trình trở nên nhanh chóng và thân thiện hơn.

Về mặt tính năng, hệ thống có thể được mở rộng với nhiều module mới mang lại giá trị gia tăng. Chức năng thanh toán có thể được đa dạng hóa bằng việc tích hợp thêm các cổng thanh toán phổ biến khác tại Việt Nam như Momo, hoặc cung cấp các lựa chọn thanh toán trả góp để tăng khả năng tiếp cận khách hàng. Đồng thời, việc xây dựng một module quản lý quan hệ khách hàng (CRM) với các chương trình tích điểm, voucher khuyến mãi, và hệ thống gửi email marketing tự động sẽ giúp tăng cường sự gắn kết và lòng trung thành của khách hàng cũ.

Một hướng phát triển khác là tăng cường khả năng kết nối của hệ thống thông qua việc tích hợp với các dịch vụ của bên thứ ba. Chẳng hạn, hệ thống có thể sử dụng API thời tiết để cung cấp thông tin hữu ích cho điểm đến, nhúng Google Maps để hiển thị lộ trình chuyến du lịch một cách trực quan, hay tích hợp các chatbot thông minh để hỗ trợ khách hàng 24/7. Về lâu dài, hệ thống có thể được mở rộng quy mô bằng cách phát triển ứng dụng di động chuyên biệt cho nền tảng Android và iOS, xây dựng API cho các đối tác du lịch, và hỗ trợ đa ngôn ngữ để hướng đến thị trường quốc tế.

Cuối cùng, việc đảm bảo an ninh và độ tin cậy của hệ thống là một quá trình liên tục. Các công việc cần thực hiện bao gồm thường xuyên cập nhật các bản vá bảo mật, triển khai cơ chế giới hạn tần suất truy cập (rate limiting) để chống tấn công từ chối dịch vụ, thiết lập hệ thống sao lưu dữ liệu tự động, cũng như xây dựng một hệ thống giám sát (monitoring) và ghi log (logging) chi tiết để chủ động phát hiện và xử lý sự cố. Những định hướng này khi được thực hiện sẽ giúp sản phẩm trở nên hoàn thiện, chuyên nghiệp và có khả năng cạnh tranh cao trên thị trường du

lịch trực tuyến.

TÀI LIỆU THAM KHẢO

- [1] M. Jones, J. Bradley, and N. Sakimura, *Json web token (jwt), rfc 7519*, [Online]. Available: <https://tools.ietf.org/html/rfc7519> (visited on 2015), 2015.
- [2] Meta and the React team, *React documentation*, [Online]. Available: <https://react.dev/> (visited on 2024-06-01), 2023.
- [3] VNPAY, *Tài liệu tích hợp cổng thanh toán vnpay*, [Online]. Available: <https://sandbox.vnpayment.vn/apis/> (visited on 2024-06-01), 2024.
- [4] Cloudinary, *Cloudinary documentation*, [Online]. Available: <https://cloudinary.com/documentation> (visited on 2024-06-01), 2024.
- [5] Supabase, *Supabase documentation*, [Online]. Available: <https://supabase.com/docs> (visited on 2024-06-01), 2024.
- [6] IBM, *What is three-tier architecture?* [Online]. Available: <https://www.ibm.com/topics/three-tier-architecture> (visited on 2024-06-10), 2024.
- [7] Postgres, *Postgres documentation*, [Online]. Available: <https://www.postgresql.org/docs/> (visited on 2024-06-10), 2024.